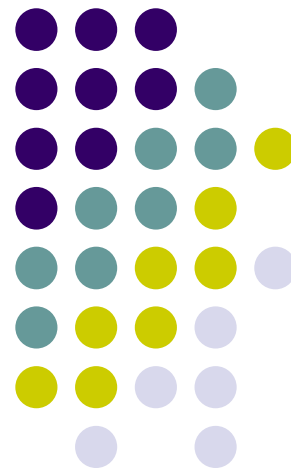


第 4 章

结构化设计方法



软件生存期

软件定义时期

问题定义

可行性研究

需求分析

软件开发时期

概要设计

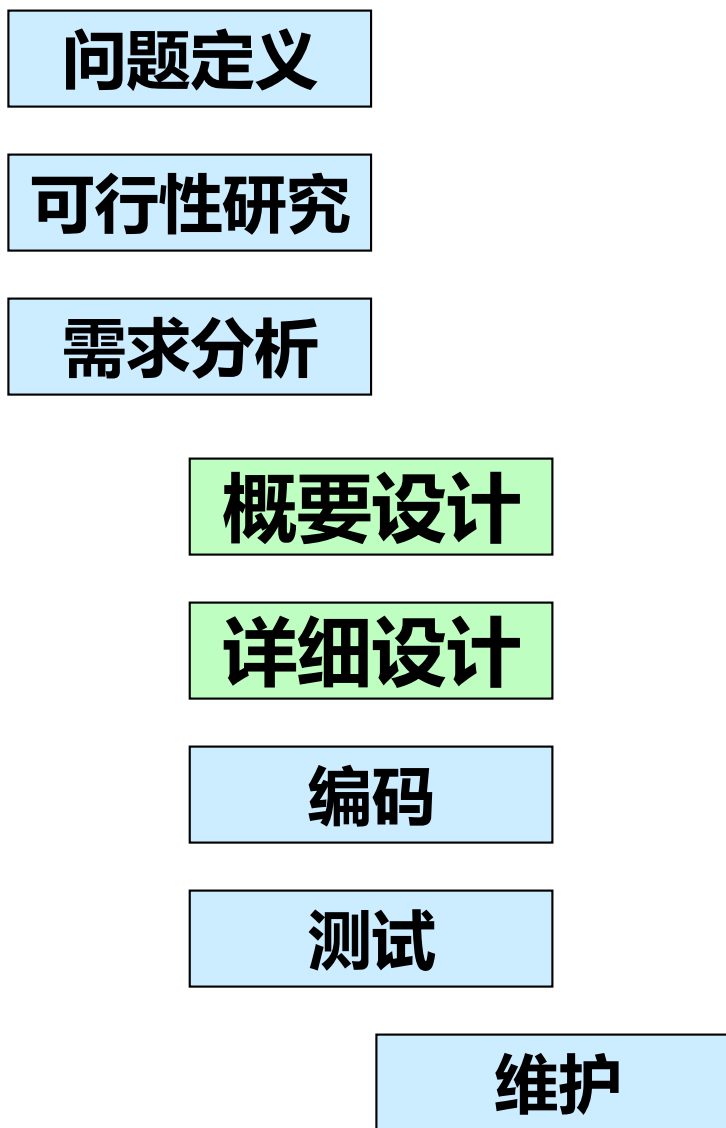
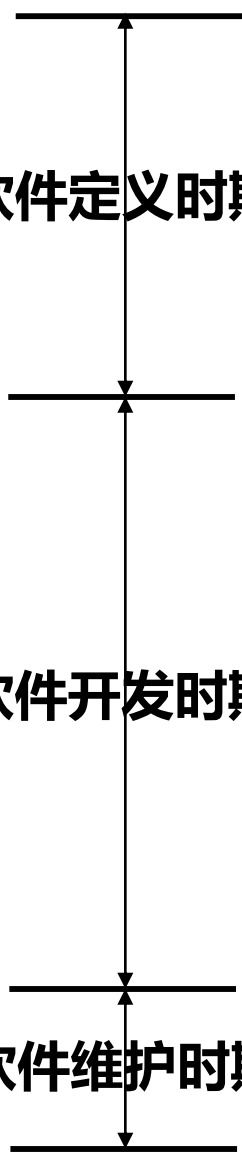
详细设计

编码

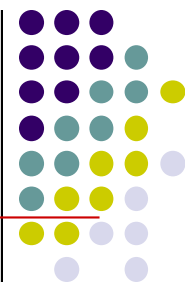
测试

软件维护时期

维护

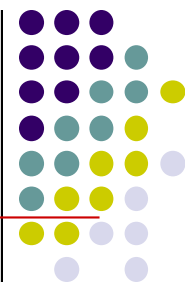


4.1 软件设计的概念及原则

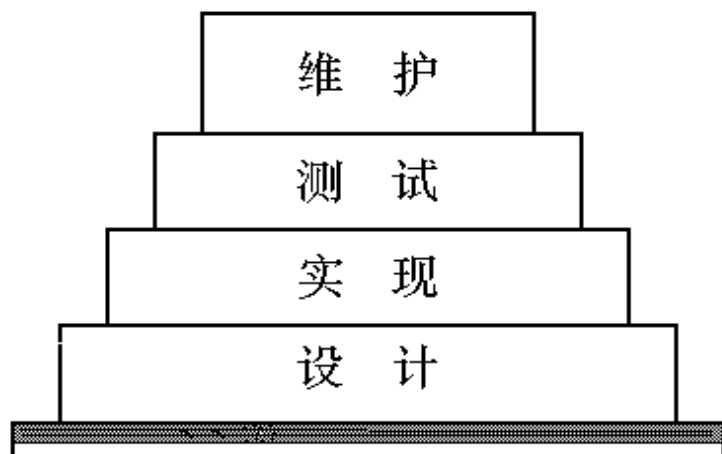


- 设计是一项核心的工程活动。
- 在20世纪90年代早期，Lotus 1-2-3的发明人 Mitch Kapor在《Dr. Dobbs》杂志上发表了“软件设计宣言”，其中指出：
- “什么是设计？设计是你站在两个世界——技术世界和人类的目标世界——而你尝试将这两个世界结合在一起……”。

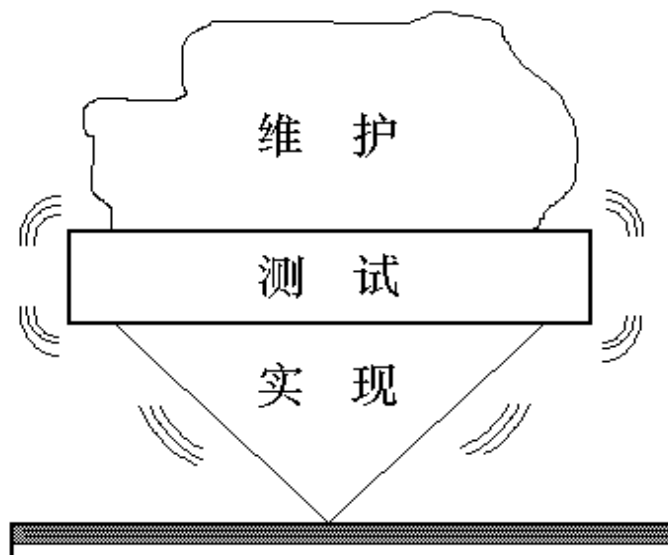
4.1 软件设计的概念及原则



- 软件设计是后续开发步骤及软件维护工作的基础。
- 如果没有设计，只能建立一个不稳定的系统结构。

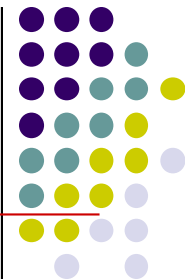


有软件设计



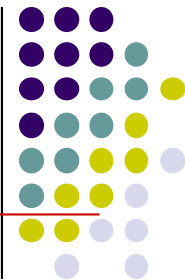
没有软件设计

4.1.2 软件设计的原则



- 分而治之——模块化
- 模块独立性
- 自顶向下逐步求精
- 复用性设计
- 灵活性设计

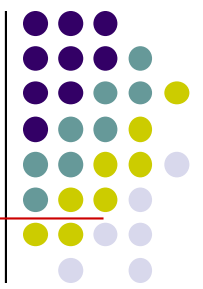
4.1.2 软件设计的原则



■ 分而治之

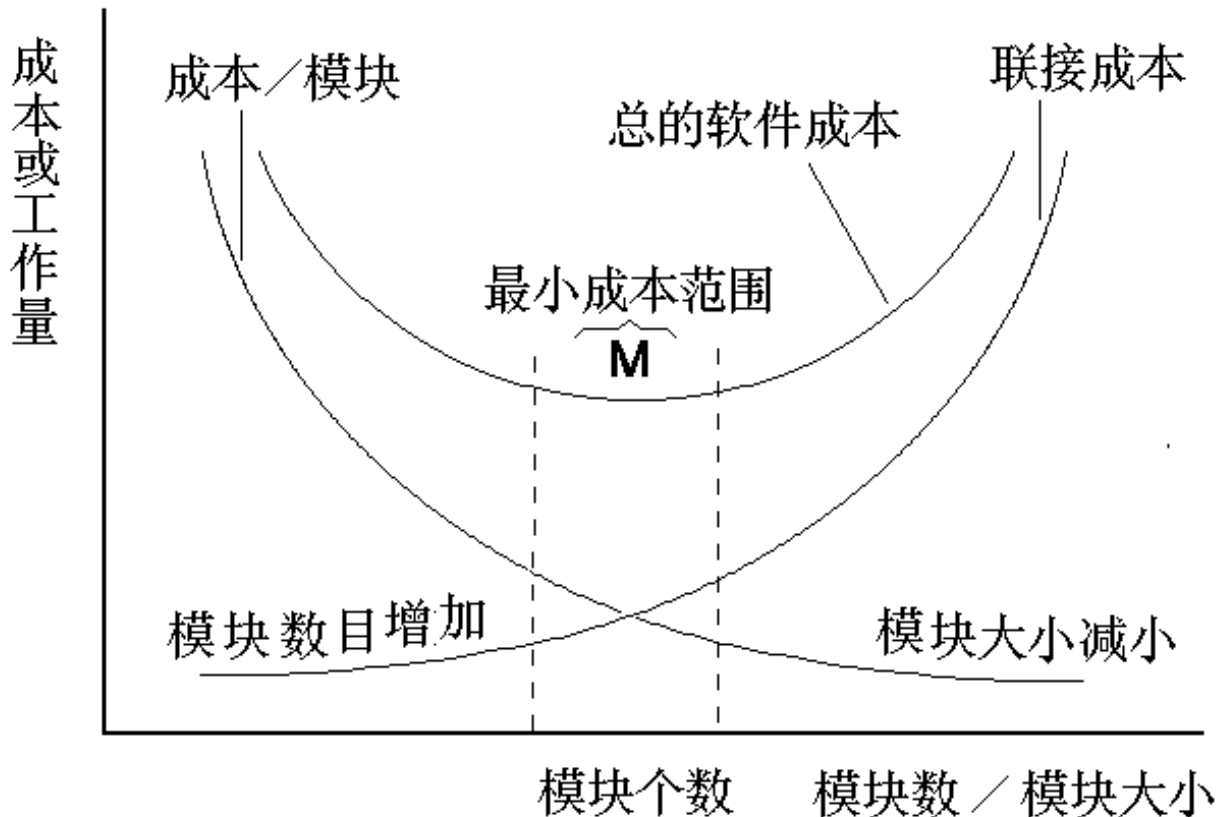
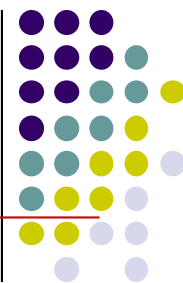
- 分而治之是人们解决大型复杂问题时通常采用的策略。
- 将大型复杂的问题分解为许多容易解决的小问题，原来的问题也就容易解决了。
- 软件的体系结构设计、模块化设计都是分而治之策略的具体表现。

模块化



- **模块化就是把程序划分成独立命名且可独立访问的模块，每个模块完成一个子功能，把这些模块集成起来构成一个整体，可以完成指定的功能满足用户的需求。**

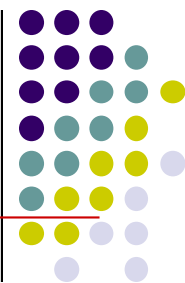
模块化



随着模块数增加

- 每个模块规模减小，开发单个模块的成本降低；
- 模块间关系更复杂，设计模块接口的工作量升高。
- 总成本先降低，后升高。

自顶向下，逐步求精

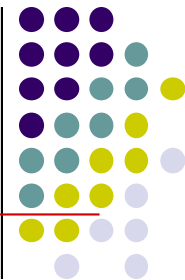


- **逐步求精**：为了能集中精力解决主要问题而尽量推迟对问题细节的考虑。

Miller法则（人类的认知规律）：一个人在任何时候都只能把注意力集中在 7 ± 2 个知识块上。

- 将软件的体系结构按自顶向下方式，对各个层次的过程细节和数据细节逐层细化，直到用程序设计语言的语句能够实现为止，从而最后确立整个体系结构。

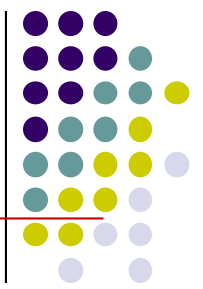
信息隐蔽



- **每个模块的实现细节对于其它模块来说是隐蔽的。**
- **模块中所包含的信息（包括数据和过程）不允许其它不需要这些信息的模块使用。**

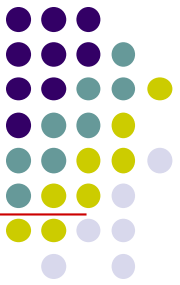


模块的独立性



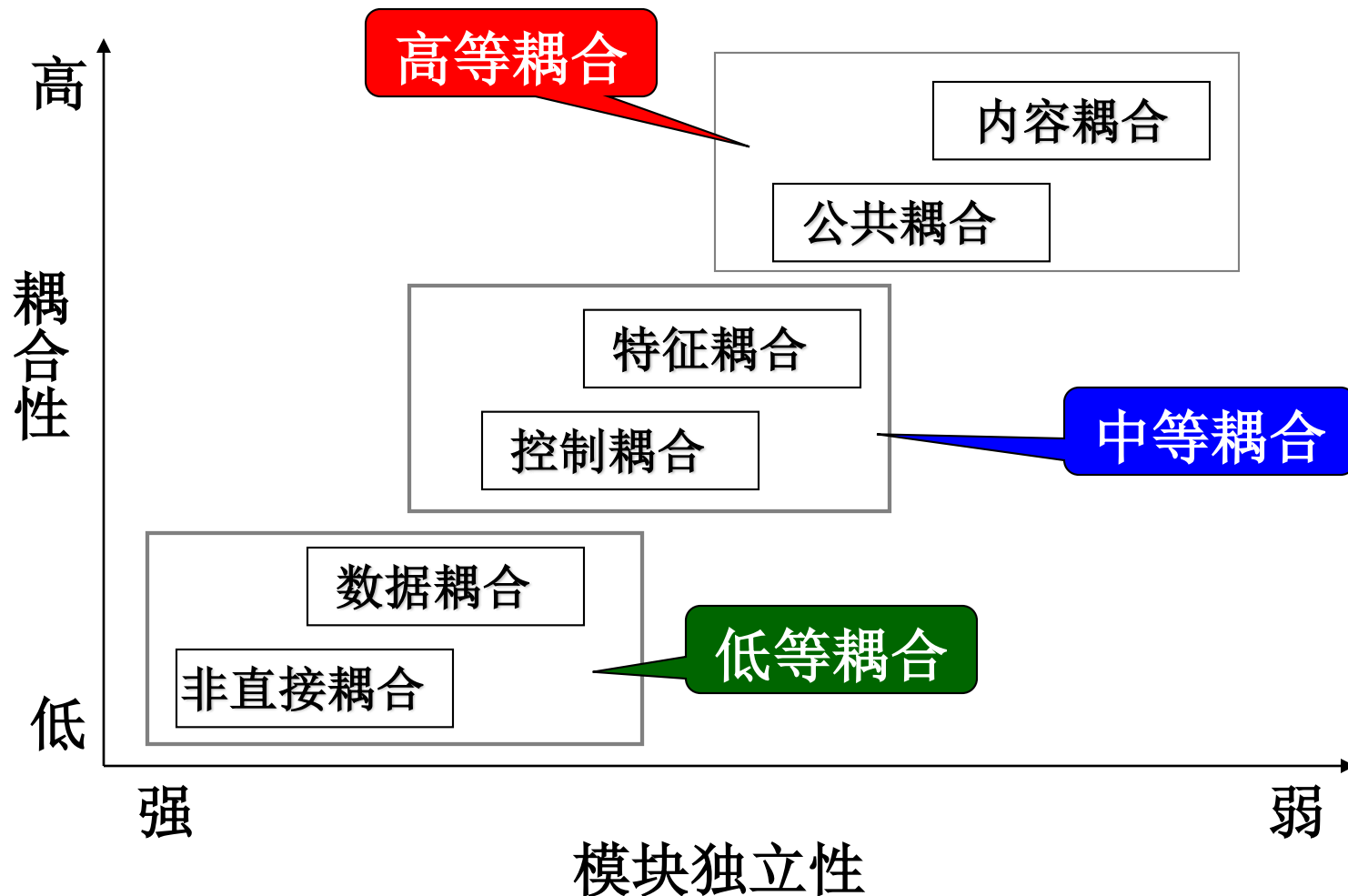
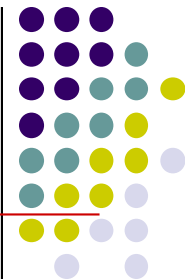
- **定义:** 是指软件系统中每个模块只涉及软件要求的具体的子功能，而和软件系统中其它模块的接口是简单的。
- **有效的模块化使软件便于分工协作开发。**
- **独立的模块比较容易测试和维护。**

模块独立性的度量准则

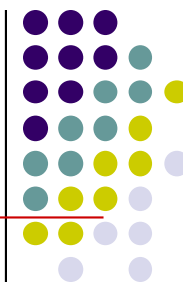


- **耦合**：是模块之间的互相连接的紧密程度的度量。
- **内聚**：是模块功能强度(一个模块内部各个元素彼此结合的紧密程度)的度量。
- 模块独立性比较强的模块应是**高内聚低耦合**的模块。

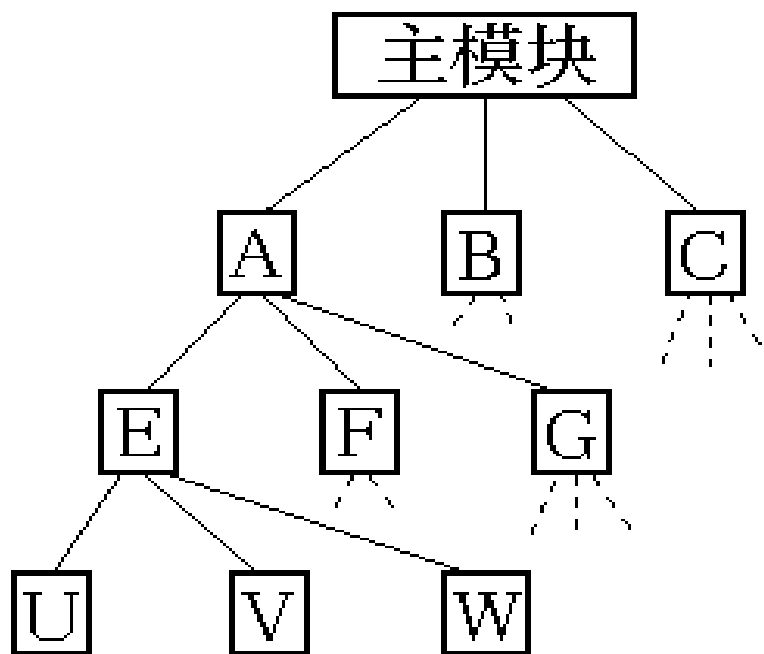
模块间的耦合



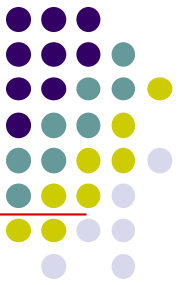
非直接耦合(Nondirect Coupling)



- 两个模块之间没有直接关系，它们之间的联系完全是通过主模块的控制和调用来实现的。
- 非直接耦合的模块独立性最强。



数据耦合 (Data Coupling)



一个模块访问另一个模块时，彼此之间是通过**简单数据参数**（不是控制参数、公共数据结构或外部变量）来交换输入、输出信息的。也是较理想的耦合。

例：计算圆面积

```
# include <stdio.h>
```

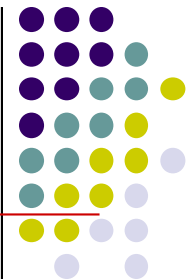
```
float area (float radius)
```

```
{    return 3.14159*radius*radius ;  
}
```

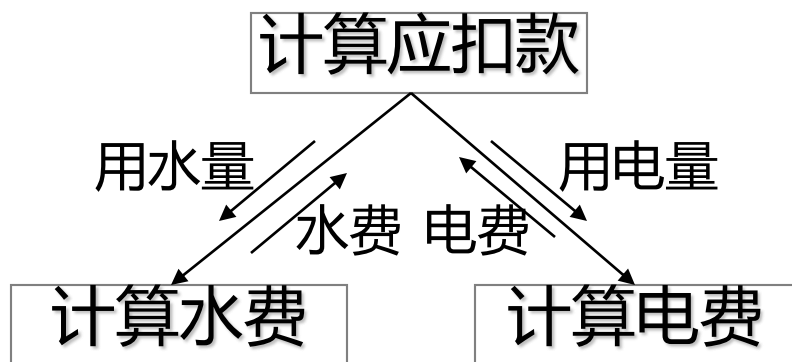
```
void main( )
```

```
{    float s, r;  
    scanf("%f ", &r);  
    s = area( r );  
    printf("area is : %f\n", s);  
}
```


特征耦合 (Stamp Coupling)



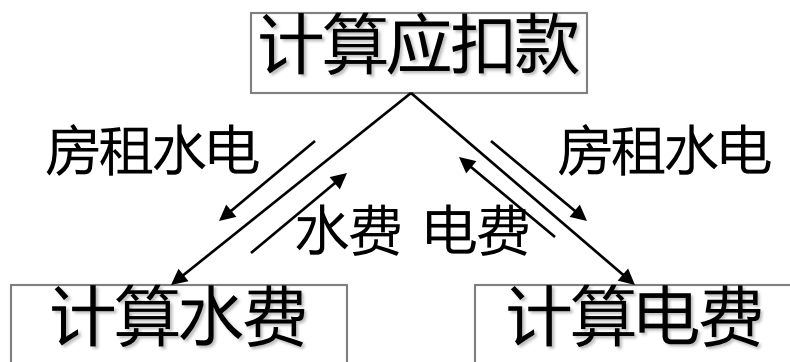
- 一组模块通过参数表传递**记录信息**，就是特征耦合。
- 这个记录是某一**数据结构**的子结构，而不是简单变量。



图a

■ 图a

上下模块间传递的是用水量和用电量、水费和电费。属于数据耦合。



图b

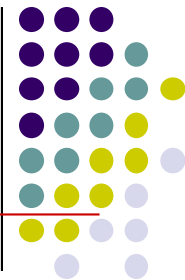
■ 图b

事先定义了数据结构：

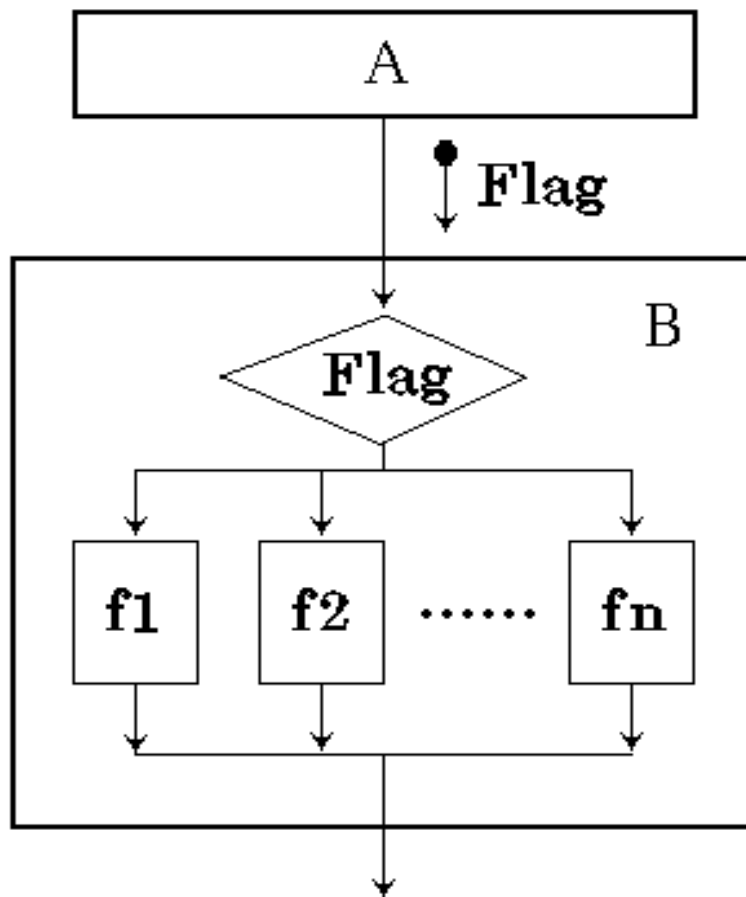
房租水电 = 房租 + 用水量 + 用电量

则上下模块间属于特征耦合

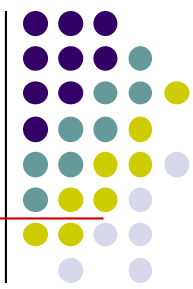
控制耦合 (Control Coupling)



**如果一个模块通过
传送开关、标志、名
字等控制信息，控制
选择另一模块的功能
，就是控制耦合。**



公共耦合（Common Coupling）



- 若一组模块都访问**同一个公共数据环境**，则它们之间的耦合就称为公共耦合。
- 公共的数据环境可以是全局数据结构、共享的通信区、内存的公共覆盖区等。

float Max , Min ;

float a[10]={ 99,45,78,97,100,67.5,89,92,66,43};

void main()

```
{  
    maxmin( );  
    echoa() ;  
    printf( “max=%6.2f \nmin=%6.2f \n” , Max , Min ) ;  
}
```

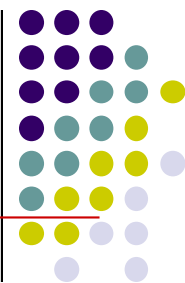
void echoa()

```
{  
    int i;  
    for(i=0;i<10;i++)  printf(“%f ”,a[i]);  
}
```

void maxmin()

```
{  
    int i ;  
    Max=Min=a[0] ;  
    for ( i=1; i<n; i++)  
    {  
        if( a[i] > Max )      Max=a[i] ;  
        else if( a[i] < Min)  Min=a[i] ;  
    }  
}
```

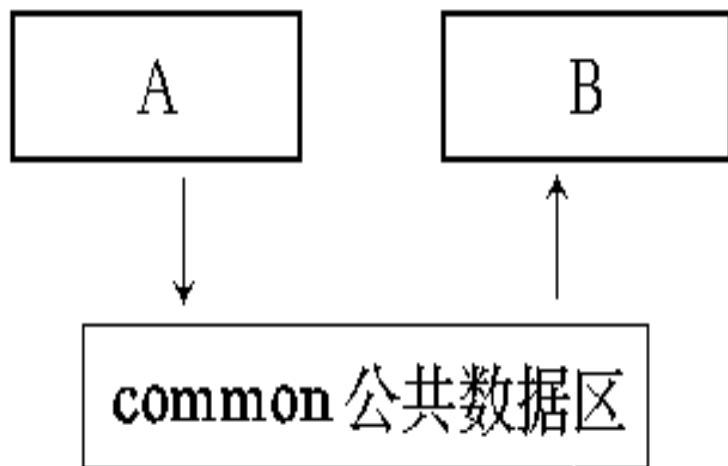
公共耦合（Common Coupling）



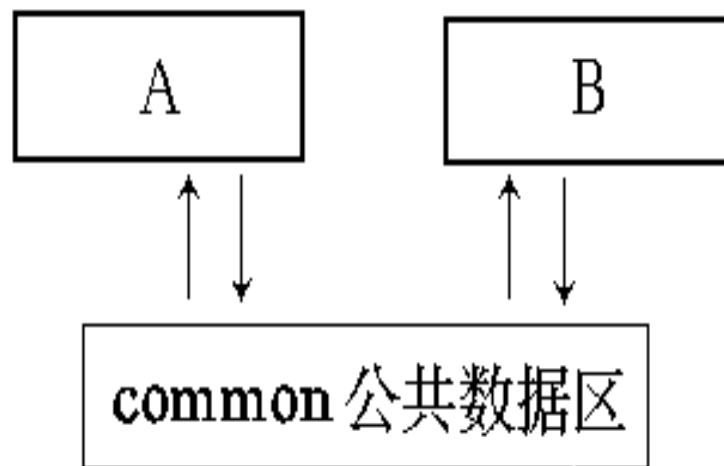
■ 公共耦合的缺点

- **修改某个数据，将会影响所有模块。**
- **无法控制各个模块对公共数据的存取，影响软件模块的可靠性和适应性。**
- **公共数据名的使用，明显降低了程序的可读性。**
- **只有在模块之间共享的数据很多，且通过参数表传递不方便时，才使用公共耦合。**

- 公共耦合的复杂程度随耦合模块的个数增加而显著增加。
- 若只是两模块间有公共数据环境，则公共耦合有两种情况。松散公共耦合和紧密公共耦合。

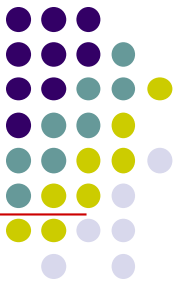


(a) 松散的公共耦合

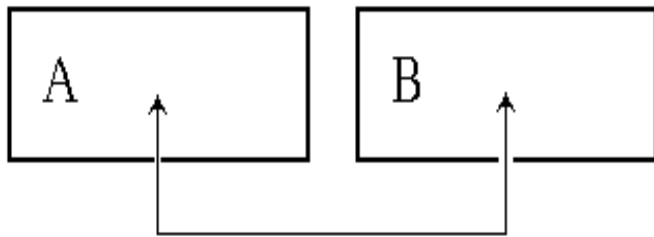


(b) 紧密的公共耦合

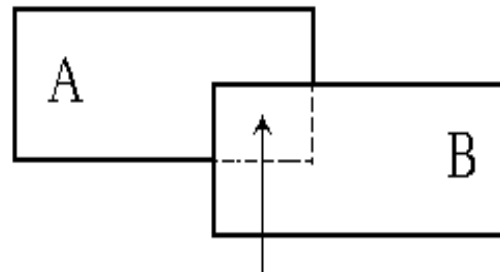
内容耦合 (Content Coupling)



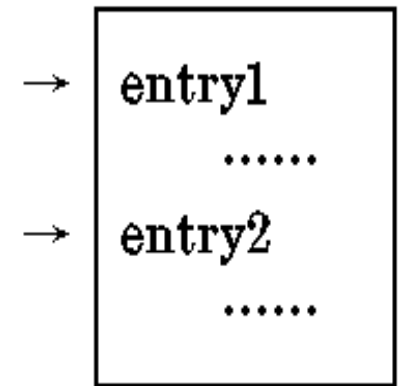
- 一个模块直接访问另一个模块的内部数据。
- 一个模块不通过正常入口转到另一模块内部。
- 两个模块有一部分程序代码重迭。
(只可能出现在汇编语言中)。
- 一个模块有多个入口。



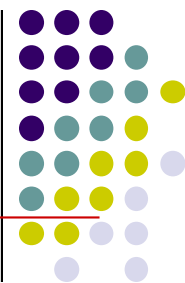
(a) 进入另一模块内部



(b) 模块代码重迭



模块间的耦合

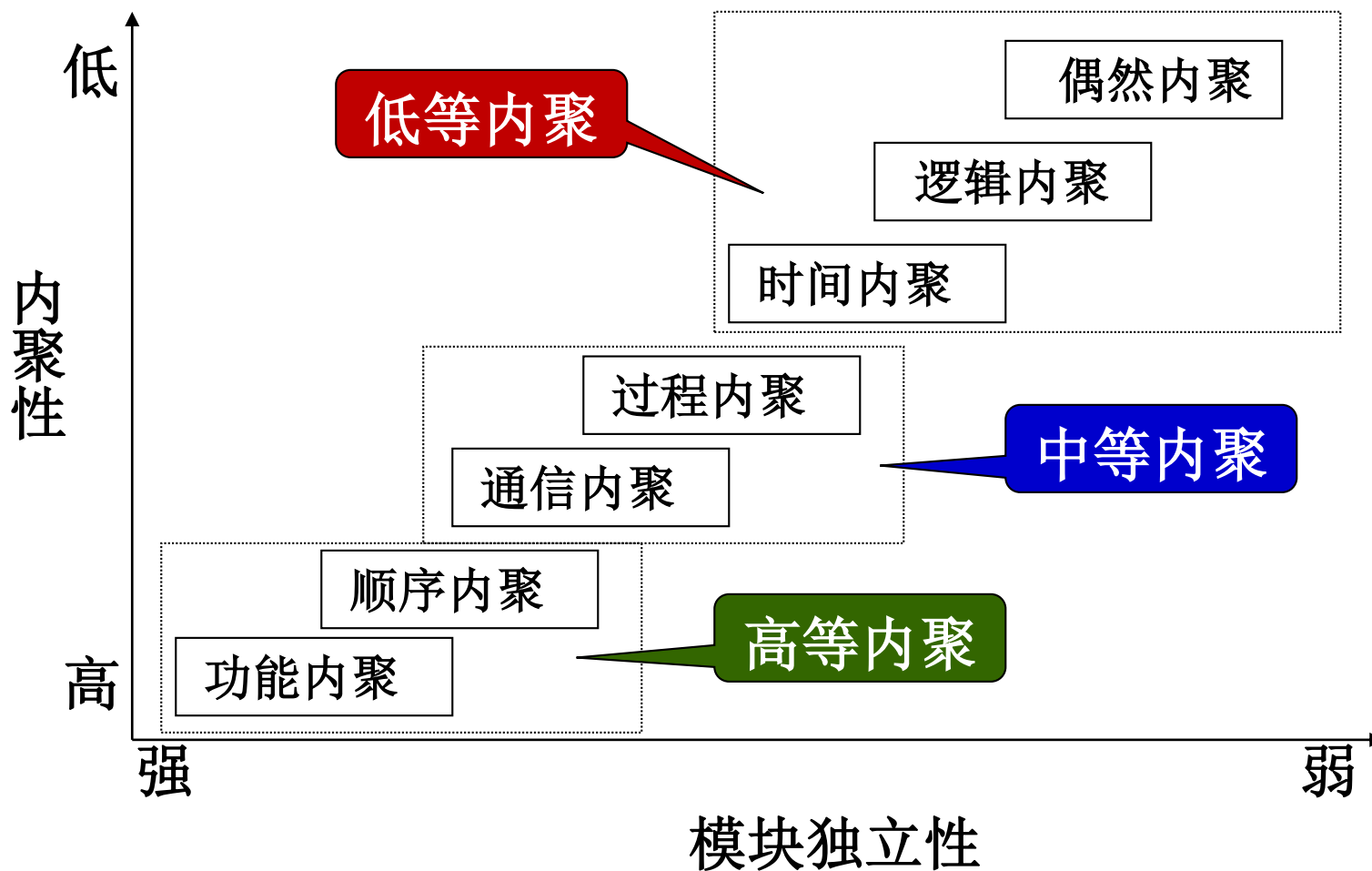
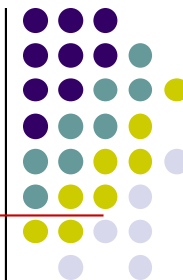


■ 设计原则

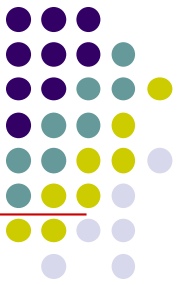
- 尽量使用数据耦合
- 少用控制耦合
- 限制使用公共耦合（除非传递大量数据）
- 完全不用内容耦合

- 实际上，两个模块之间的耦合不只是一种类型，而是多种类型的混合。这就要求设计人员进行分析、比较，逐步加以改进，以提高模块的独立性。

模块内聚



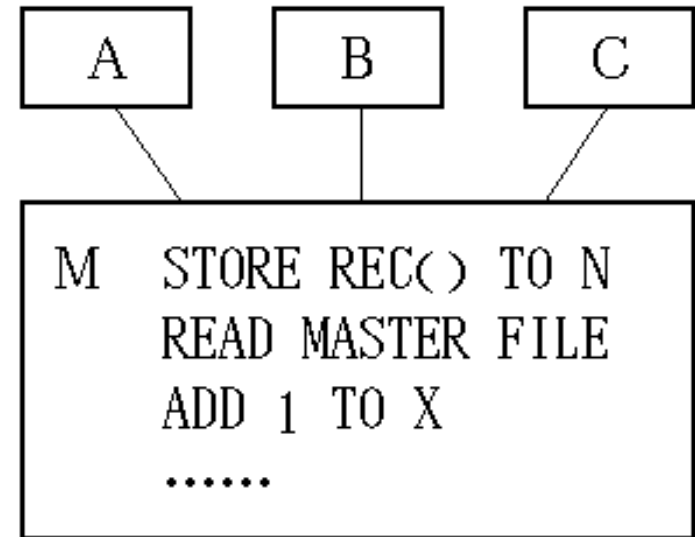
偶然内聚 (Coincidental Cohesion)



- 当模块内各部分之间没有联系，或者即使有联系，这种联系也很松散，则称这种模块为偶然内聚模块，内聚程度最低。

- **缺点**

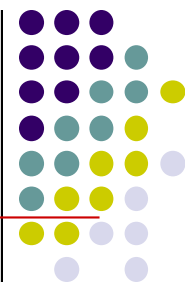
- 1) 内容不易理解，很难描述其功能。
- 2) 把完整的程序分割到多个模块中，在程序运行时会频繁地互相调用。



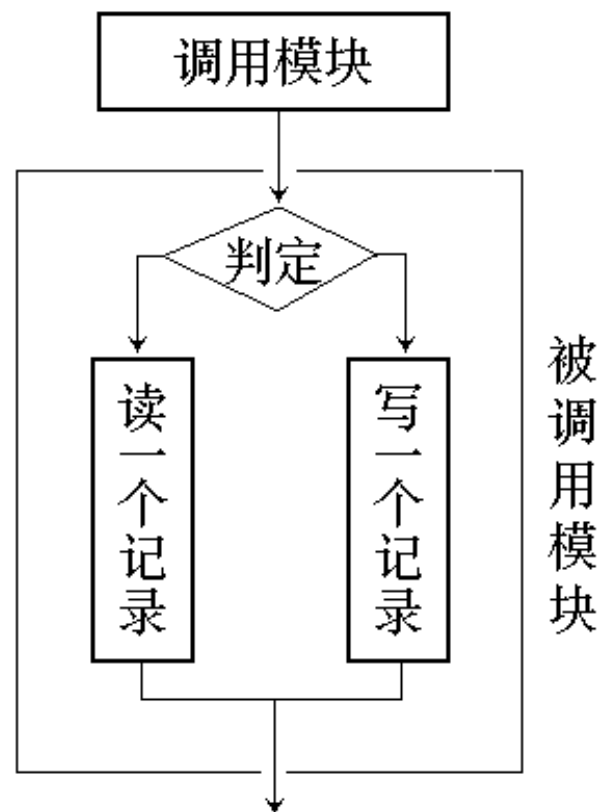
- **识别**

如果描述模块功能时很难用一句话说清，则可能是偶然内聚。

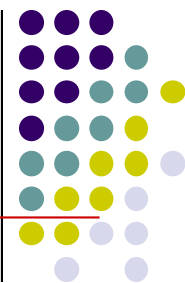
逻辑内聚 (Logical Cohesion)



- 把几种**相关的功能**组合在一起，每次被调用时，由传送给模块的判定参数来确定该模块应执行哪个功能。
- **缺点**
 - 1) 不易修改，因包含多个功能
 - 2) 需传递控制参数——控制耦合
 - 3) 未用部分调入内存，影响效率
- 逻辑内聚模块比偶然内聚模块的内聚程度要高。因为它表明了各部分之间在功能上的相关关系。



时间内聚（Classical Cohesion）

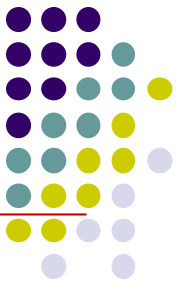


- 时间内聚模块大多为**多功能**模块，但模块的各个功能的执行**与时间有关**，通常要求**所有功能必须在同一时间段内执行**。

例如：初始化模块和终止模块。

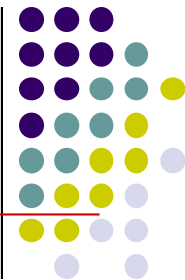
- 时间内聚模块比逻辑内聚模块的内聚程度又稍高一些。在一般情形下，各部分可以以任意的顺序执行，所以它的内部逻辑更简单。

过程内聚（Procedural Cohesion）

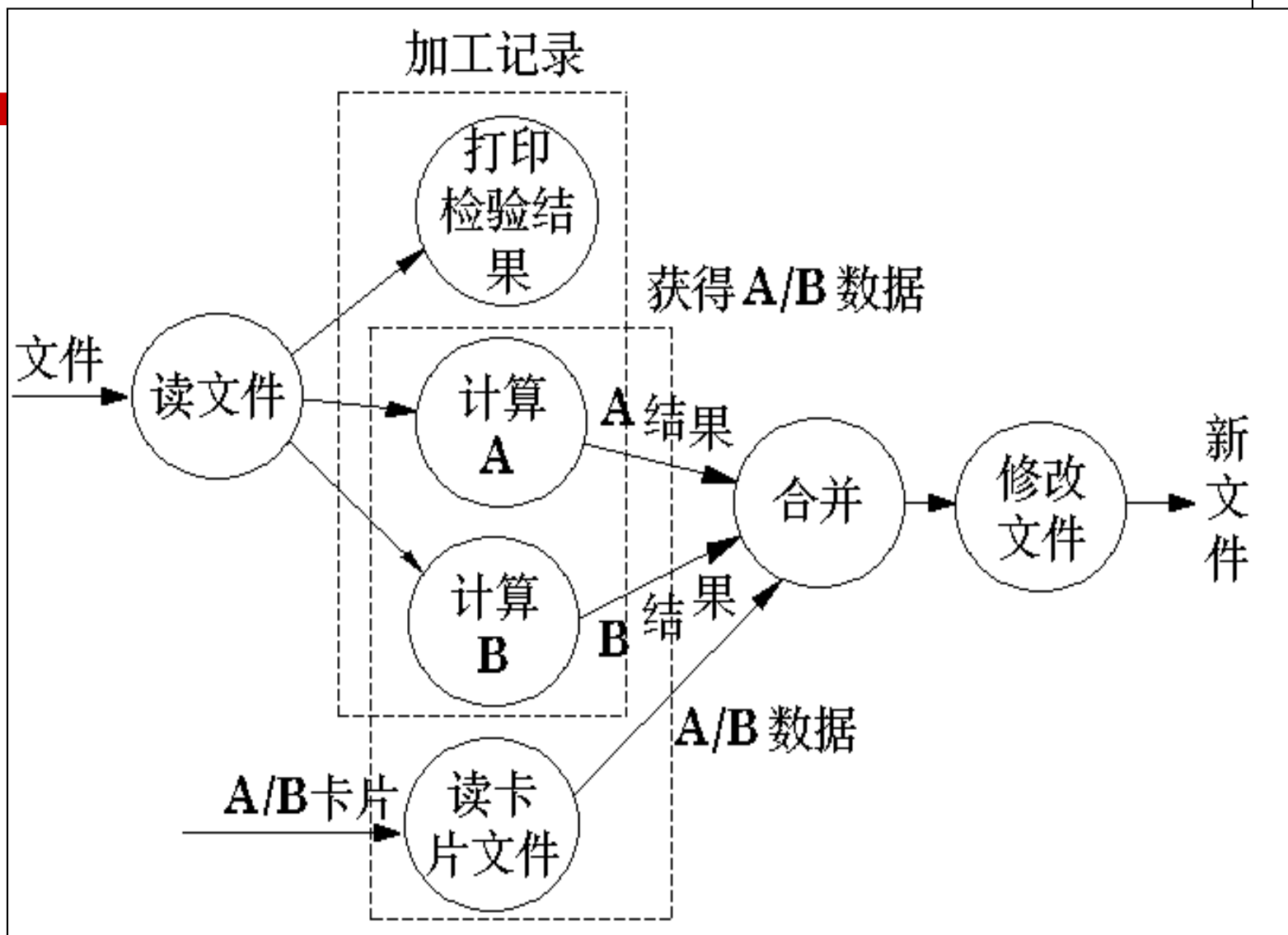
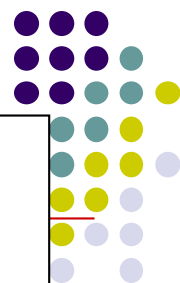


- 如果一个模块内的**处理是相关的**，而且必须以**特定次序执行**，则是过程内聚。
- **过程内聚仅包含完整功能的一部分**
- 使用流程图做为工具设计程序时，把流程图中的某一部分划出组成模块，就得到过程内聚模块。
- 例如，把流程图中的循环部分、判定部分、计算部分分成三个模块，这三个模块都是过程内聚模块。

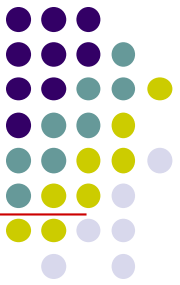
通信内聚 (Communication Cohesion)



- 如果一个模块内各功能部分都使用了**相同的输入数据**，或产生了**相同的输出数据**，则称之为通信内聚模块。
- 通常，通信内聚模块是通过数据流图来定义的。
- 比过程内聚高，因通讯内聚模块中包含了**许多独立的功能**。



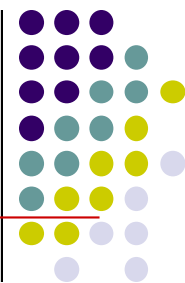
顺序内聚(Sequential Cohesion)



- n 一个模块中的处理元素和**同一功能**密切相关，而且这些处理必须**顺序执行**，则称为顺序内聚。
- n 根据数据流图划分模块时，通常得到顺序内聚的模块。

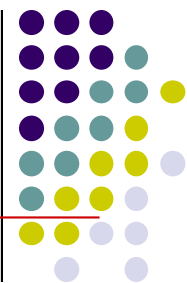


功能内聚 (Functional Cohesion)



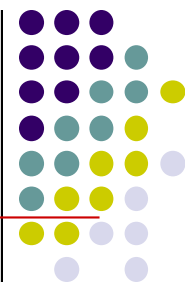
- 一个模块中各个部分都是**完成某一具体功能**必不可少的组成部分，或者说模块中所有部分都是为了完成一项具体功能而协同工作，紧密联系，不可分割的。则称该模块为功能内聚模块。
- 优点：容易修改和维护
- 识别方法
只用一个动宾词组就能准确地描述其功能。

模块的独立性



- 模块之间的连接越紧密，联系越多，耦合性就越高，而其模块独立性就越弱。
- 一个模块内部各个元素之间的联系越紧密，则它的内聚性就越高，相对地，它与其它模块之间的耦合性就会减低，而模块独立性就越强。
- 因此，模块**独立性比较强**的模块应是**高内聚低耦合**的模块。

4.1 软件设计的概念及原则

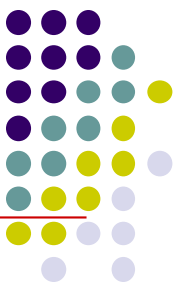


■ 复用性设计

- 复用是指同一事物不做修改或稍加修改就可以多次重复使用。将复用的思想用于软件开发，称为软件复用。
- 将软件的重用部分称为软构件。
- 在构造新的软件系统时不必从零做起，可以直接使用已有的软构件即可组装（或加以合理修改）成新的系统。

■ 复用性设计有两方面的含义

- 软件设计中尽量使用已有构件；
- 若需开发新的构件，也要考虑将来的可复用性。



4.1 软件设计的概念及原则

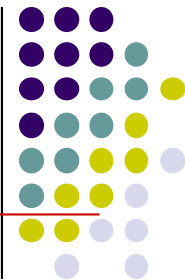
■ 灵活性设计

- **灵活性**：修改一个地方，不会影响到其他，代码修改平稳地发生。
- 保证软件灵活性设计的关键是**抽象**。

■ 在设计中引入灵活性的方法

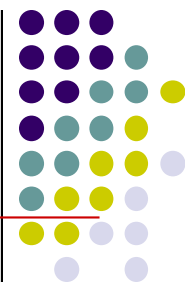
- **降低耦合并提高内聚**（易于提高替换能力）；
- **建立抽象**（创建有多态操作的接口和父类）；
- 不要将代码写死（消除代码中的常数）；
- **抛出异常**（由操作的调用者处理异常）；
- 使用并创建可复用的代码。

4.2 结构化设计



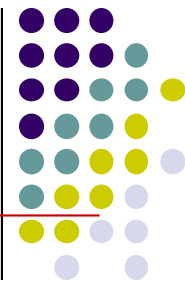
- 结构化设计的任务
- 结构化设计与结构化分析的关系
- 模块结构及表示
- 数据结构及表示

4.2.1 软件设计的任务



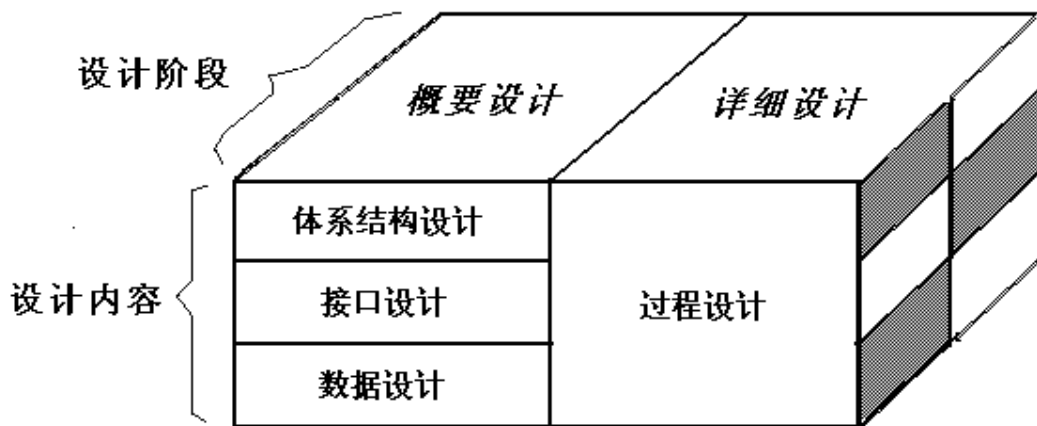
- 软件设计的主要任务是要**解决如何做的问题**，要在需求分析的基础上，建立各种设计模型，并通过对设计模型的分析 and 评估，来确定这些模型是否能够满足需求。
- 软件设计是将用户需求准确地转化成为最终的软件产品的唯一途径，在需求到构造之间起到了桥梁作用。
- 在软件设计阶段，往往存在多种设计方案，通常需要在多种设计方案之中进行决策和折中，并使用选定的方案进行后续的开发活动。

4.2.1 软件设计的任务

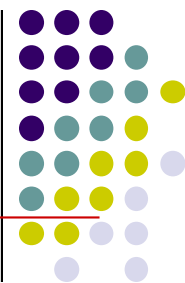


■ 软件设计的阶段与任务

- 从工程管理的角度，将设计分为**概要设计阶段**和**详细设计阶段**。
- 从技术的角度，结构化方法将软件设计划分为**体系结构设计、数据设计、接口设计**和**过程设计**4部分。

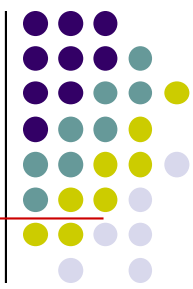


4.2.1 软件设计的任务

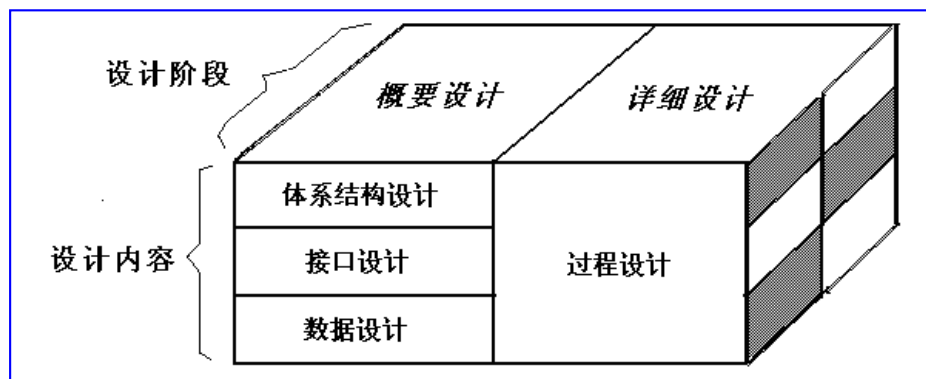
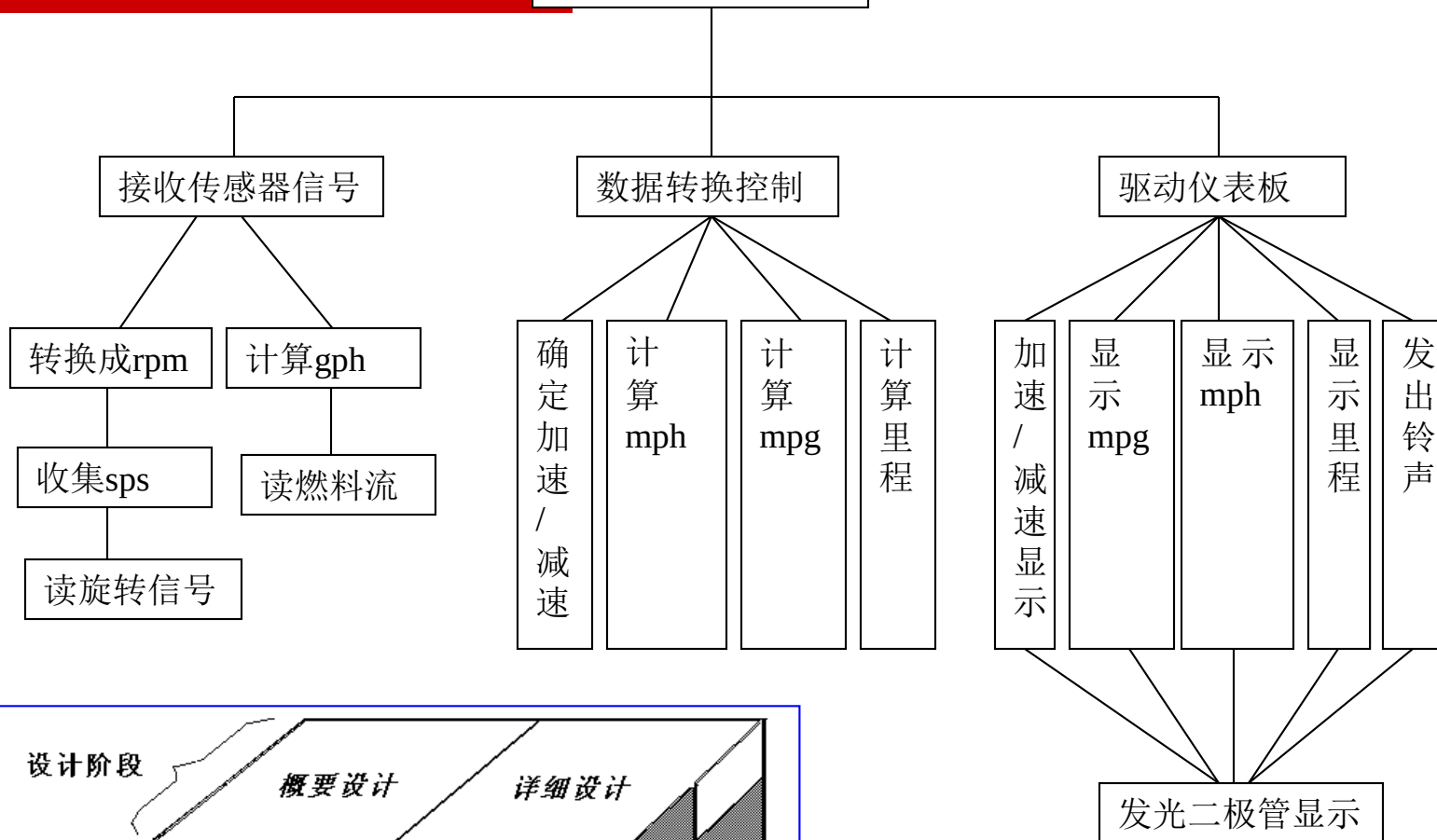


■ 软件设计的阶段与任务

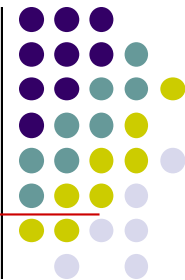
- **体系结构设计**：定义软件的主要结构元素及其之间的关系。
- **接口设计**：描述用户界面，软件和其他硬件设备、其他软件系统及使用人员的外部接口，以及各种构件之间的内部接口。
- **数据设计**：根据需求阶段所建立的数据模型来确定所涉及的文件系统结构及数据库结构。
- **过程设计**：确定软件各个组成部分内的算法及内部数据结构，并选定某种过程的表达形式来描述各种算法。



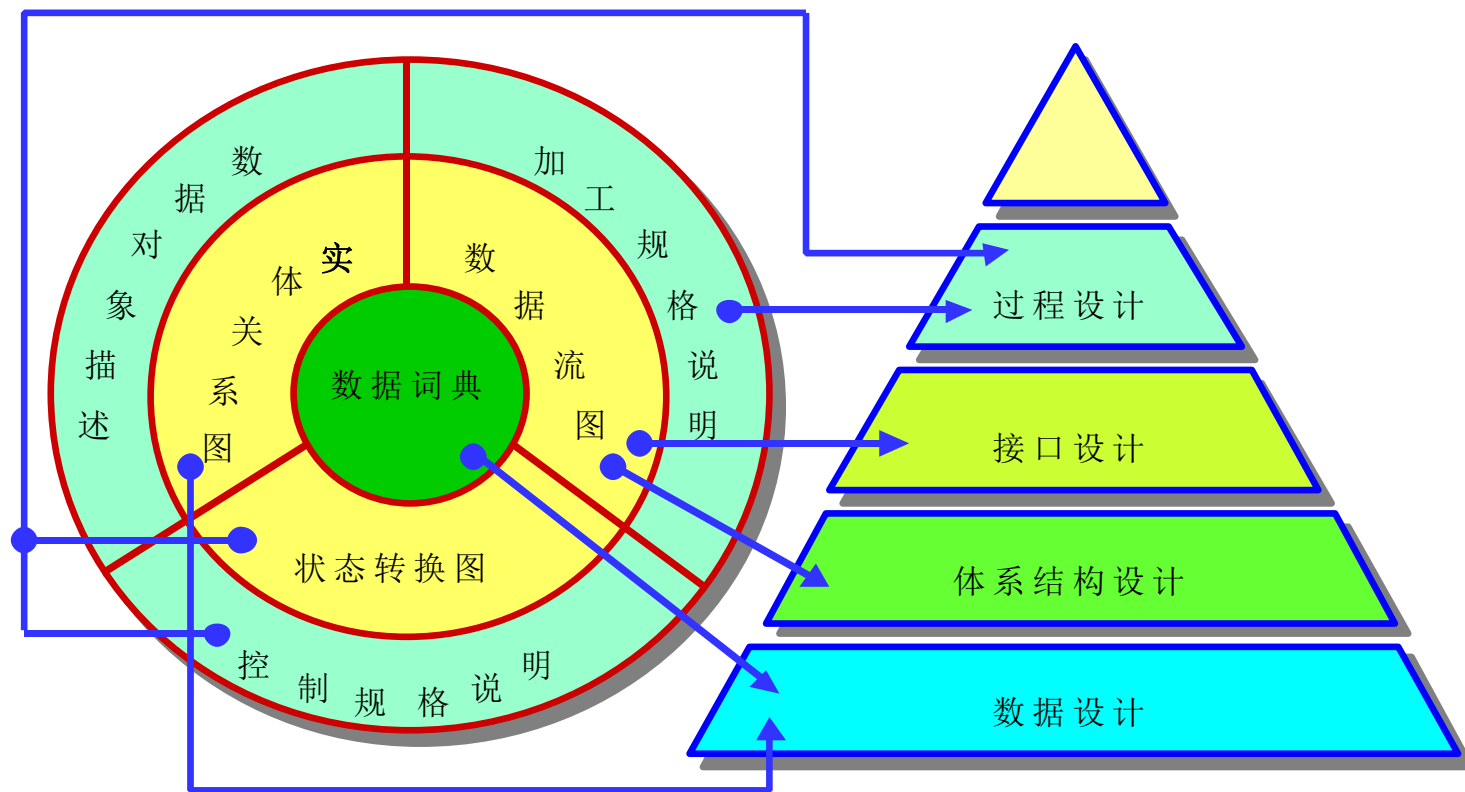
数字仪表板控制



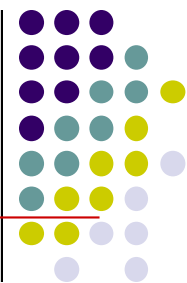
4.2.2 结构化分析与结构化设计的关系



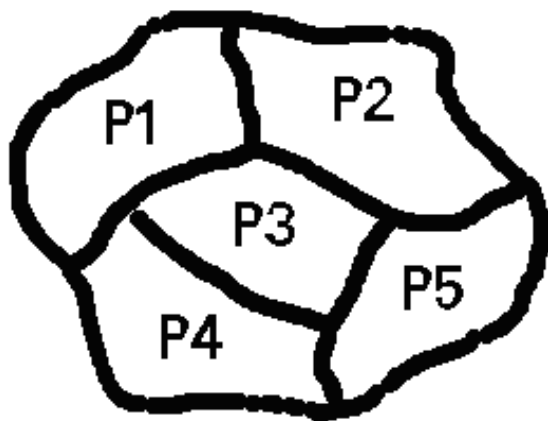
- 结构化分析的结果为结构化设计提供了最基本的输入信息，两者的关系如下图所示。



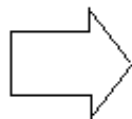
4.2.3 模块结构及表示



- 一般通过功能划分过程来完成软件结构设计。
- 功能划分过程从需求分析确立的目标系统的模型出发，对**整个问题进行分割**，使其每一部分用一个或几个**软件模块**加以解决，整个问题就解决了。

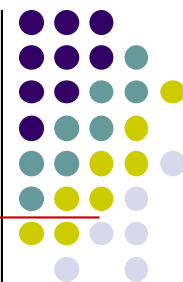


需要通过软件解决的问题



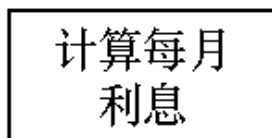
通过功能划分得到的软件模块

4.2.3 模块结构及表示

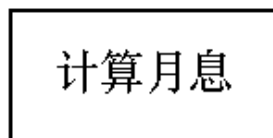


■ 模块

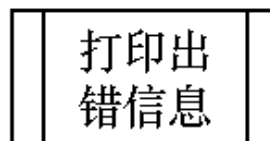
- 一个软件系统通常由很多模块组成；
- 结构化程序设计中的函数和子程序都可称为模块；
- 模块是程序语句按逻辑关系建立起来的组合体。
- 模块用矩形框表示，并用模块的名字标记它。



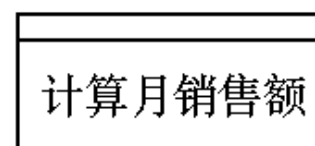
以功能做模块名



以功能的缩写
做模块名

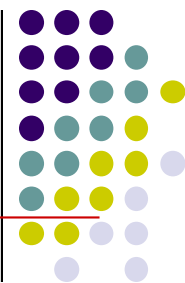


已定义模块



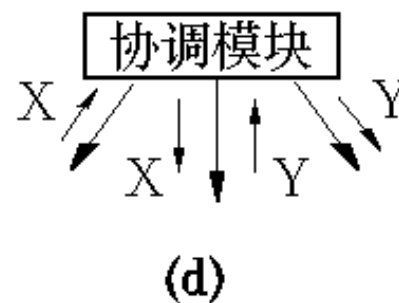
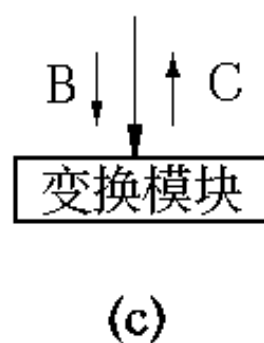
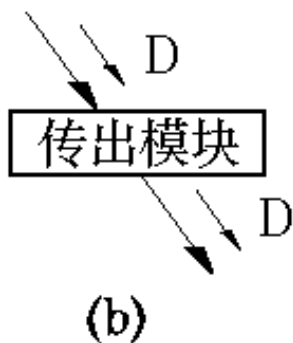
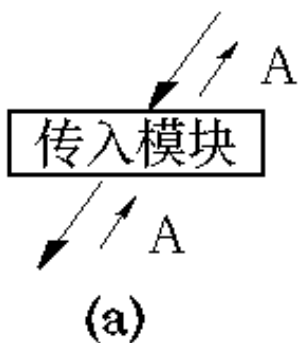
子程序(或过程)

4.2.3 模块结构及表示



■ 模块的分类

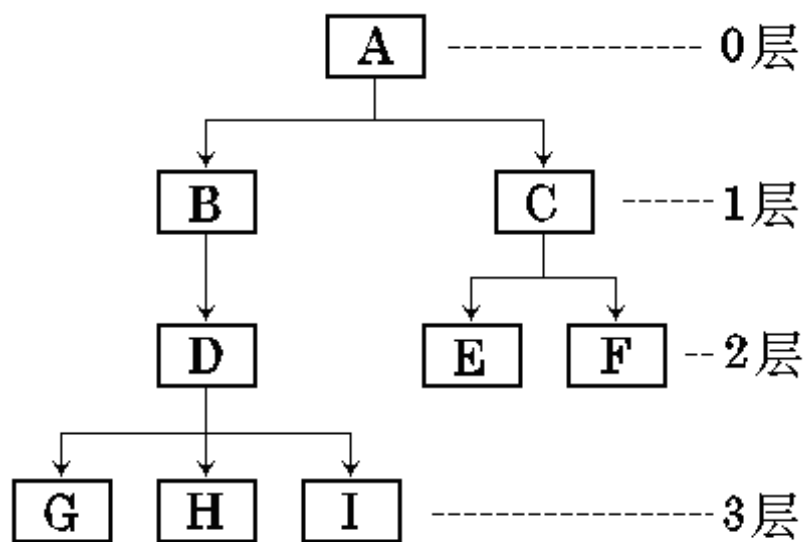
- **传入模块**：取得数据或输入数据的模块；
- **传出模块**：输出数据的模块；
- **变换模块**：从上级模块取得数据，经过处理后，将处理结果返回给上级调用模块。
- **协调模块**：通过调用、协调和管理其他模块来完成特定功能。



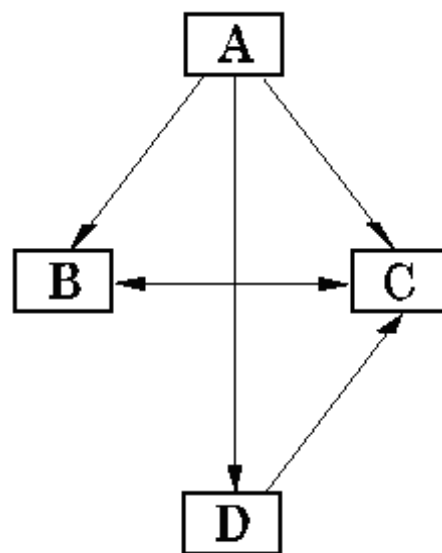
4.2.3 模块结构及表示

■ 模块结构

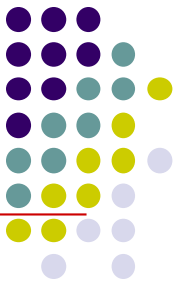
- 模块结构最普通的形式就是树状结构和网状结构。



(a) 树状结构



(b) 网状结构



4.2.3 表示软件结构的图形工具

■ 层次图

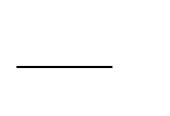
- 描绘软件的层次结构。

矩形框代表一个模块

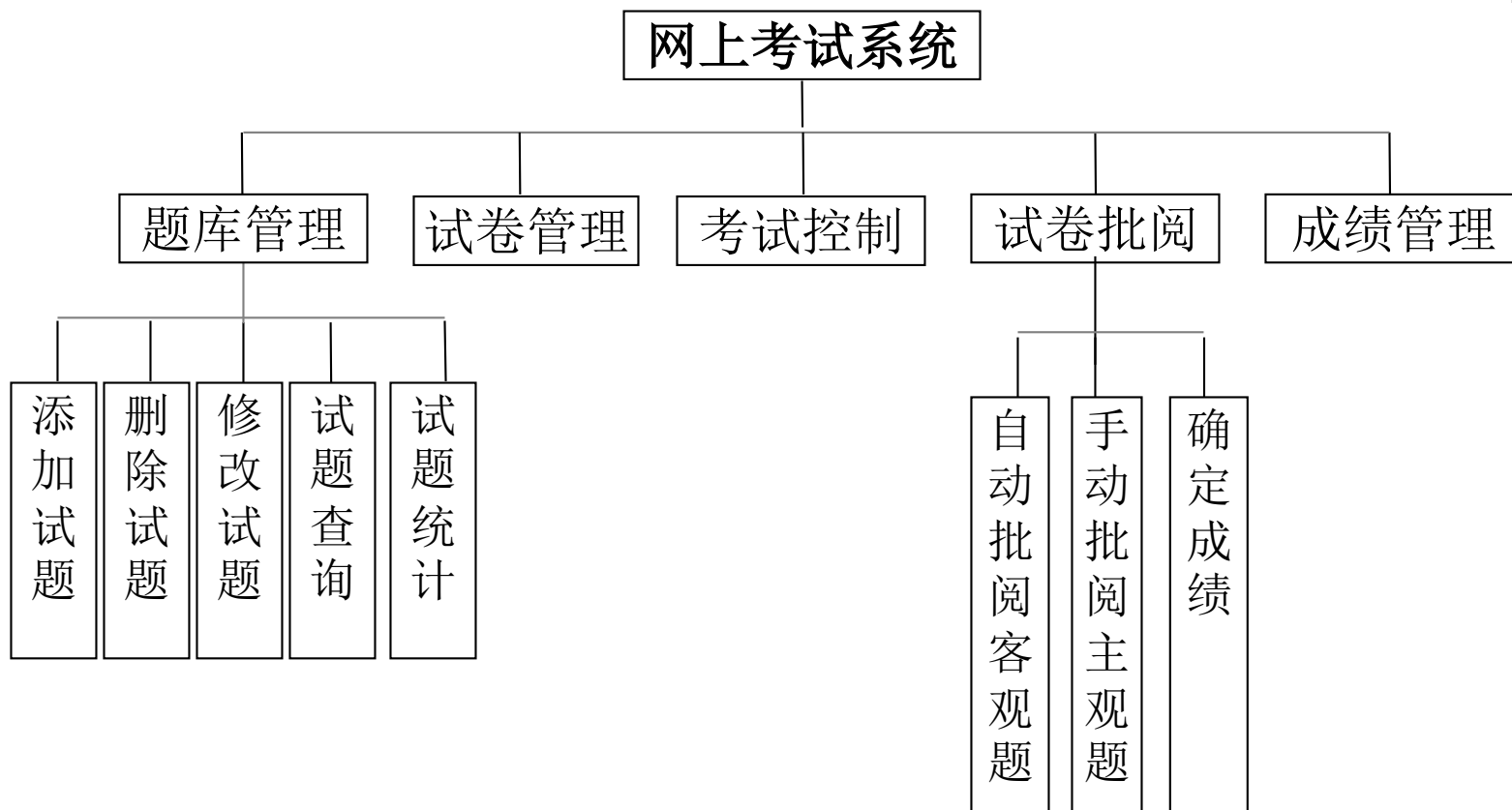
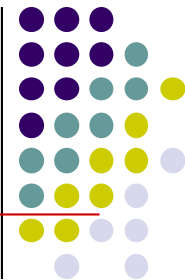
框间的连线表示调用关系

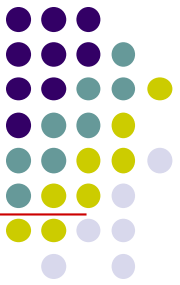
位于上方的模块调用下方的模块

求根



4.2.3 表示软件结构的图形工具



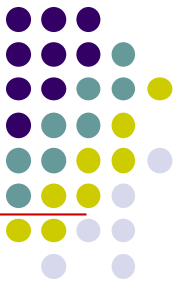


4.2.3 表示软件结构的图形工具

■ 结构图（SC图）

结构图反映程序中模块之间的层次调用关系和联系；

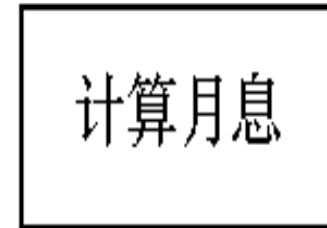
它以特定的符号表示模块、模块间的调用关系和模块间信息的传递。



4.2.3 表示软件结构的图形工具

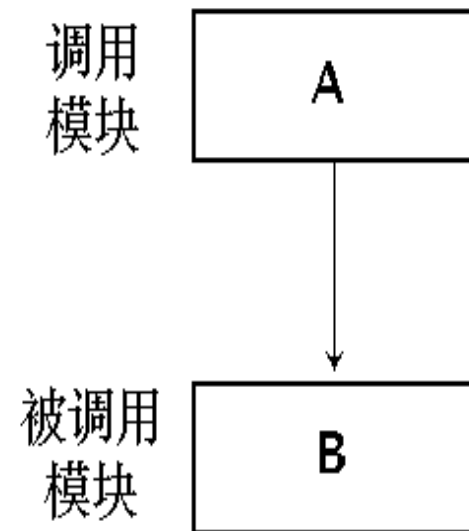
■ 模块

模块用矩形框表示，并用模块的名字标记它。

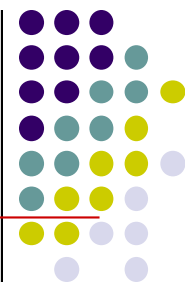


● 模块的调用关系和接口

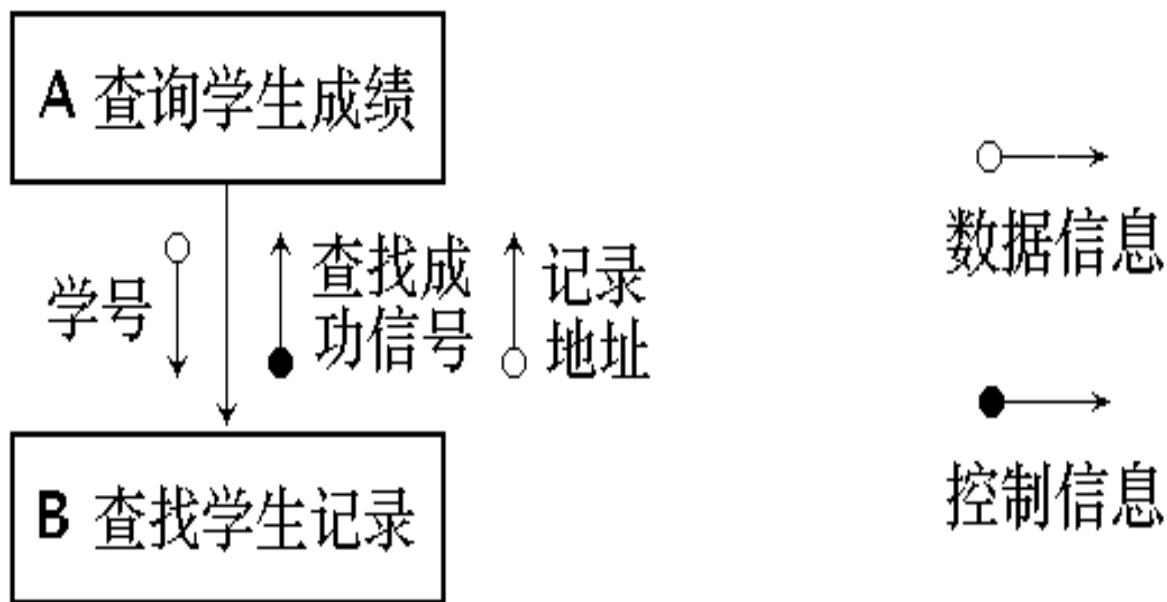
模块之间用单向箭头联结，箭头从调用模块指向被调用模块



4.2.3 表示软件结构的图形工具

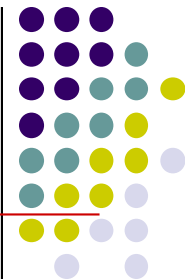


■ 模块间的信息传递



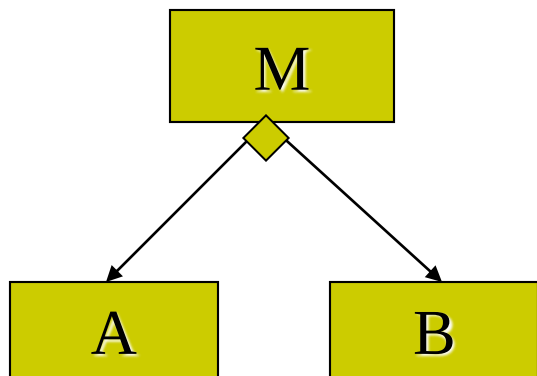
(b) 模块间接口的表示

4.2.3 表示软件结构的图形工具

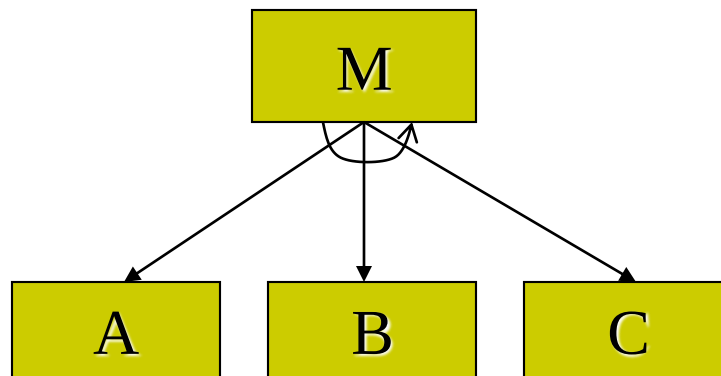


■ 附加符号

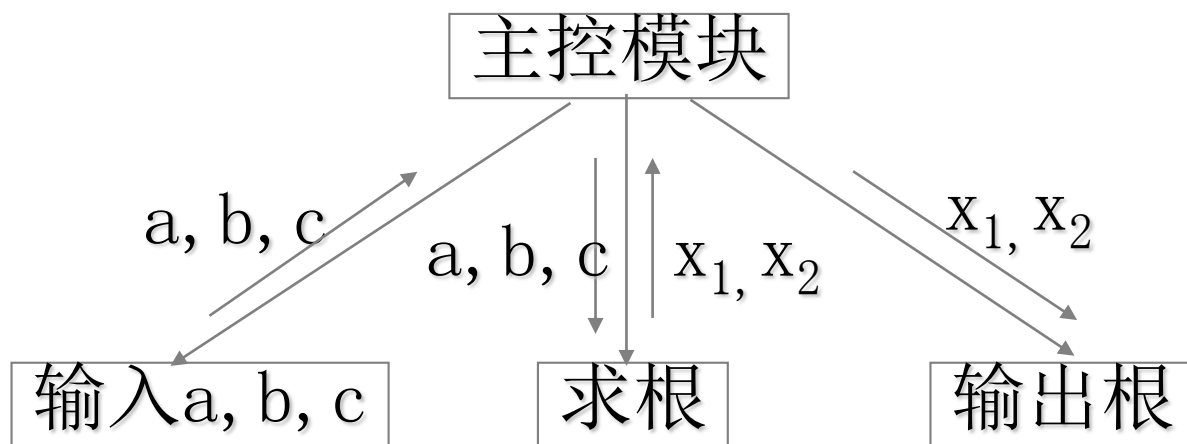
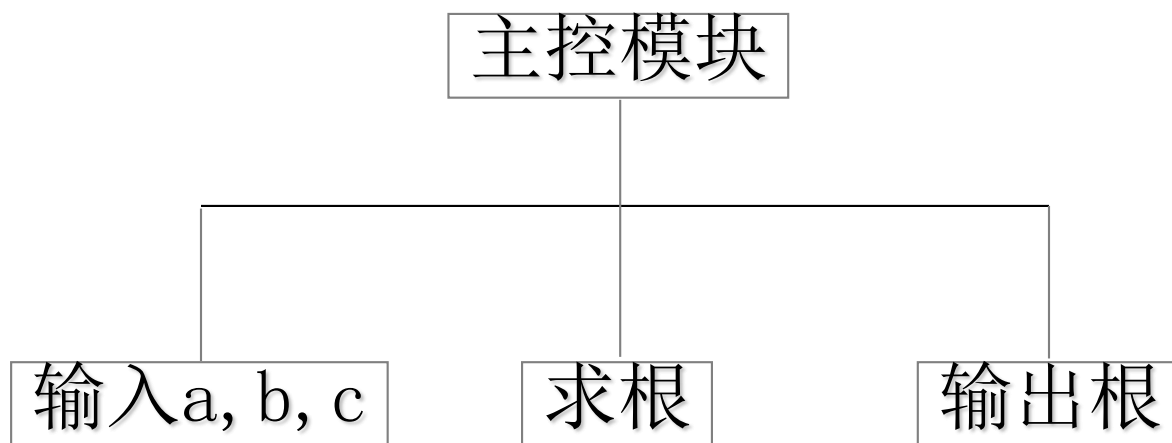
条件调用



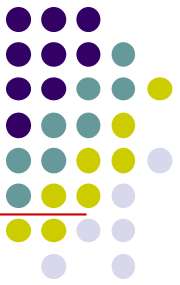
循环调用



SC图和HC图的联系和区别

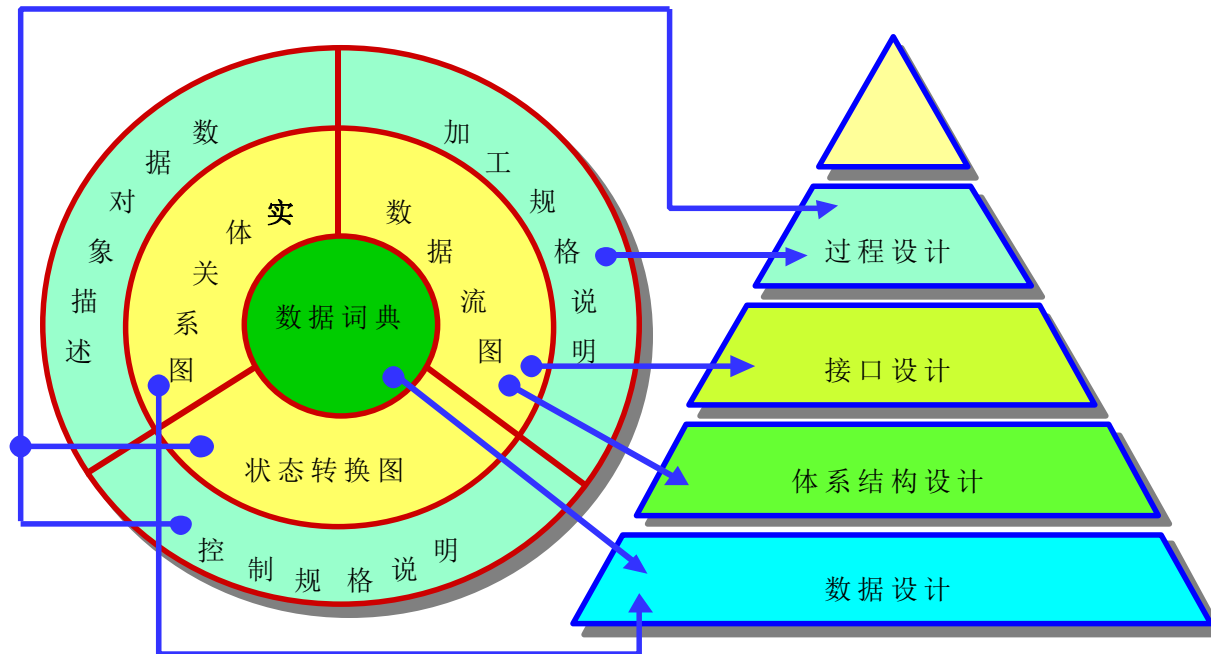


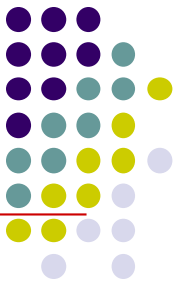
求一元二次方程的根



4.3 面向数据流的设计方法

- 面向数据流的设计方法是把信息流映射成软件结构。（DFD图 $\longrightarrow$ SC图）





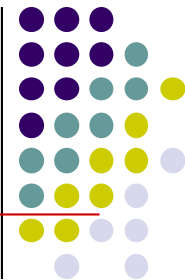
4.3 面向数据流的设计方法

- 信息流的类型决定了映射的方法。
- 信息流有下述两种类型

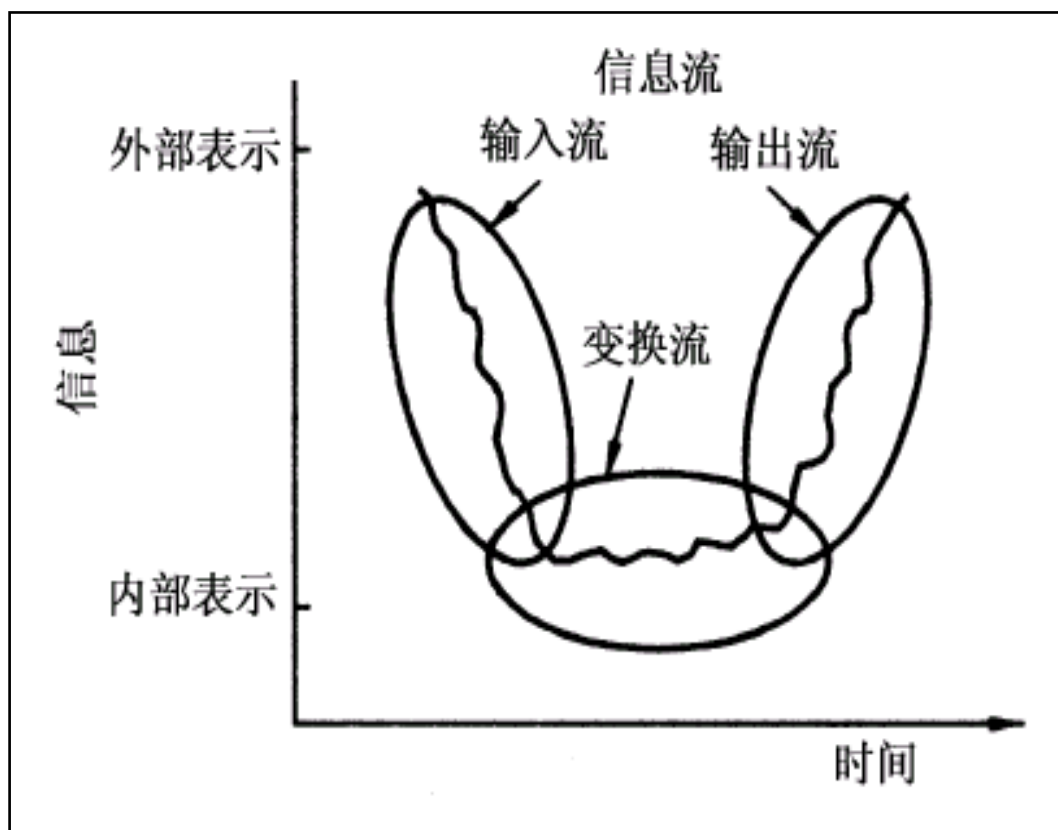
1.变换流

2.事务流

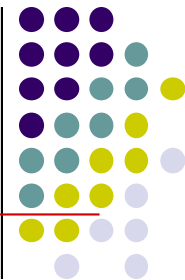
4.3 面向数据流的设计方法



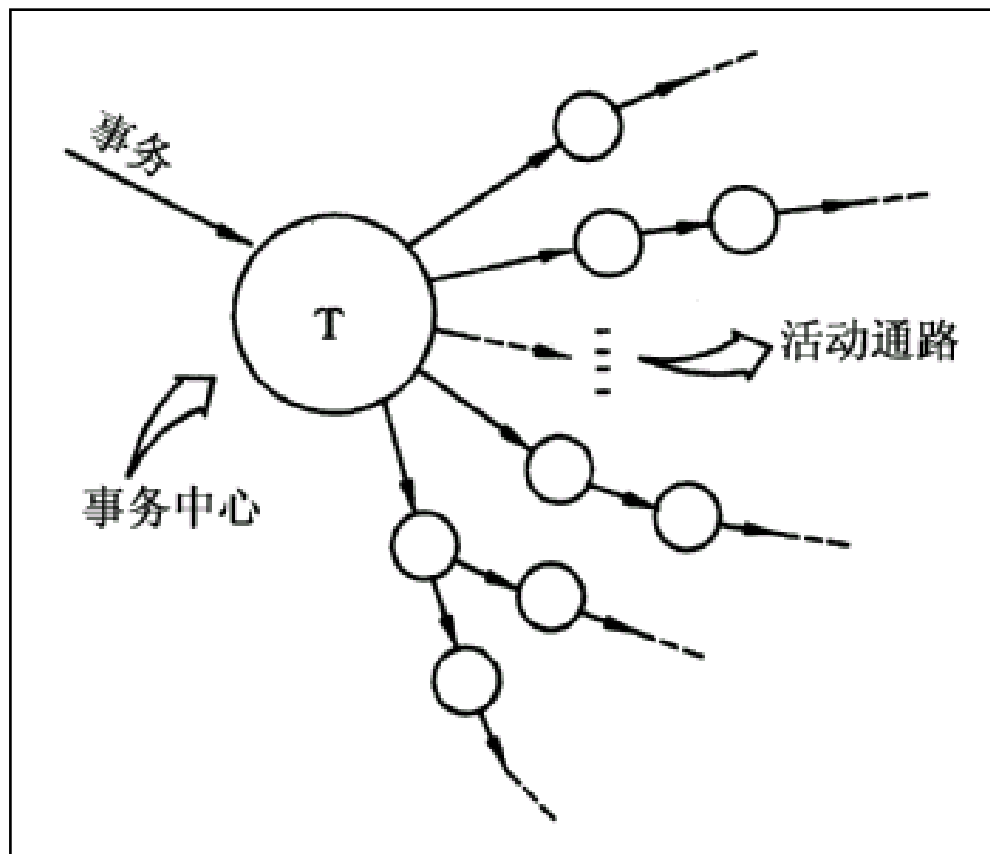
- 变换流

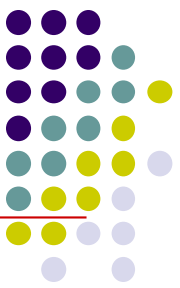


4.3 面向数据流的设计方法



● 事务流





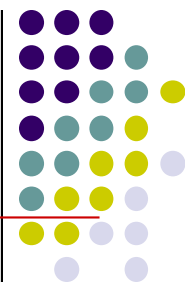
4.3 面向数据流的设计方法

■ 结构化设计的实施要点

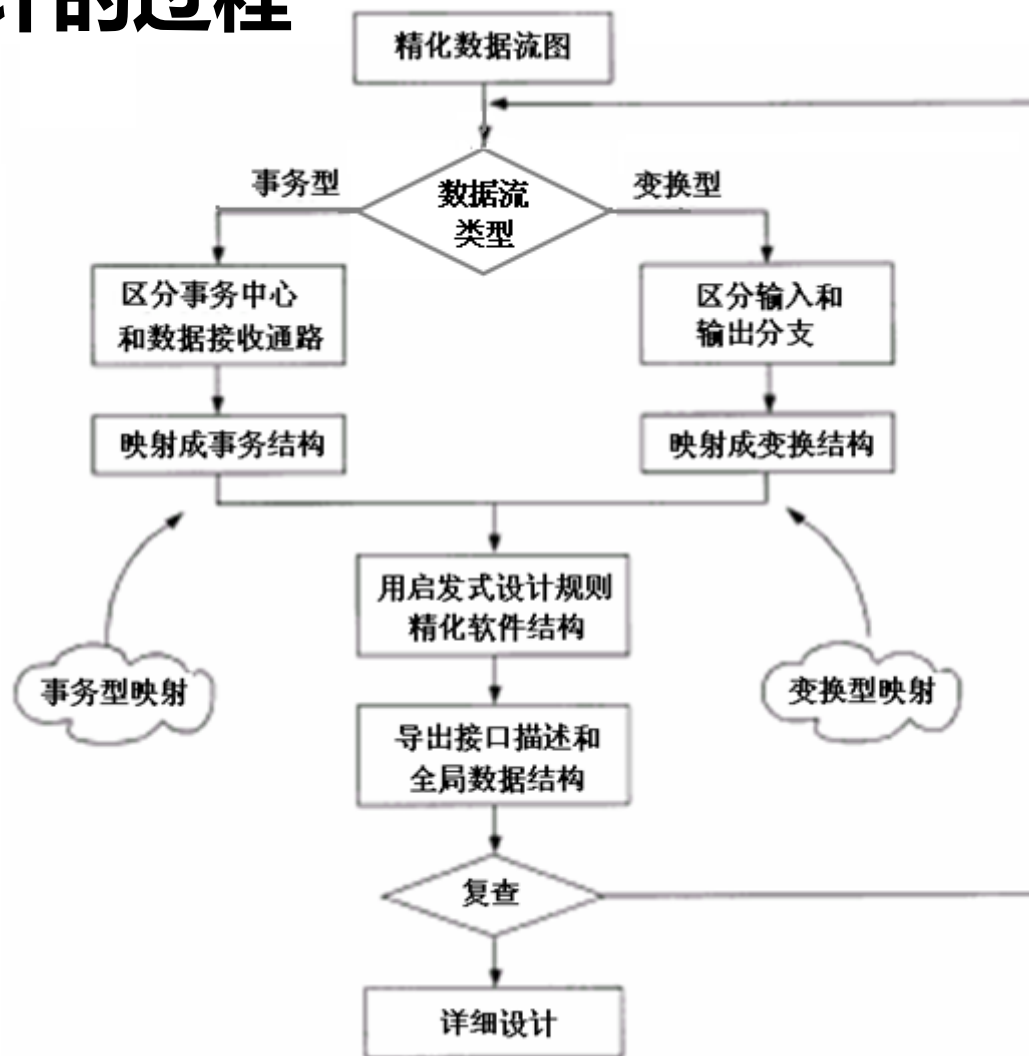
- 研究、分析和审查数据流图。
- 根据数据流图决定问题的类型：**变换型和事务型**。
针对两种不同的类型分别进行分析处理。
- 由数据流图**推导出**系统的**初始结构图**。
- 利用一些启发式原则来**改进**系统的**初始结构图**。
- 根据实体关系图和数据字典进行**数据设计**，包括数据库设计或数据文件的设计。
- 依据分析模型中的加工规格说明、状态转换图进行**过程设计**。
- 制定测试计划。



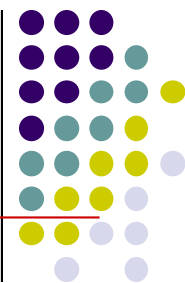
4.3 面向数据流的设计方法



■ 结构化设计的过程

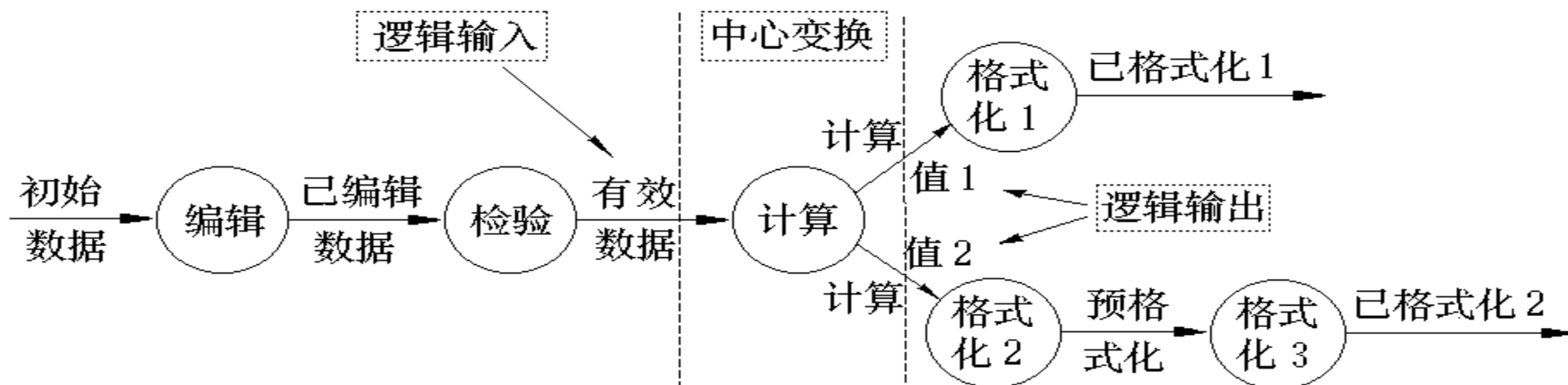


4.3.3 变换型数据流映射方法

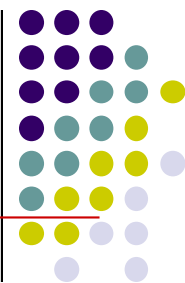


■ 变换分析方法由以下四步组成

- 1) 重画数据流图，确定物理输入和物理输出；
- 2) 区分逻辑输入、逻辑输出和中心变换部分；
 - 逻辑输入：离物理输入端最远，但仍可以看作系统输入的数据流。
 - 逻辑输出：离物理输出端最远，但仍可以看作系统输出



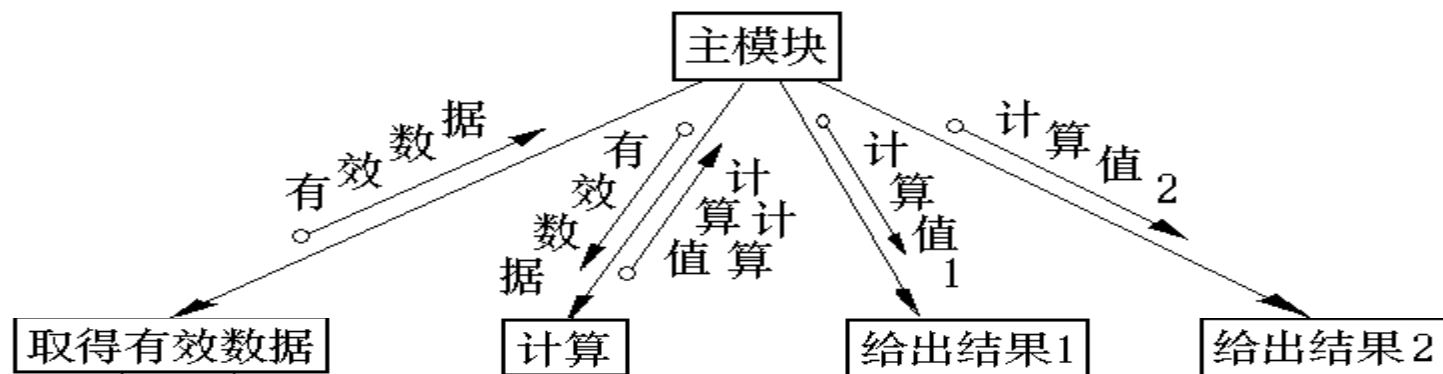
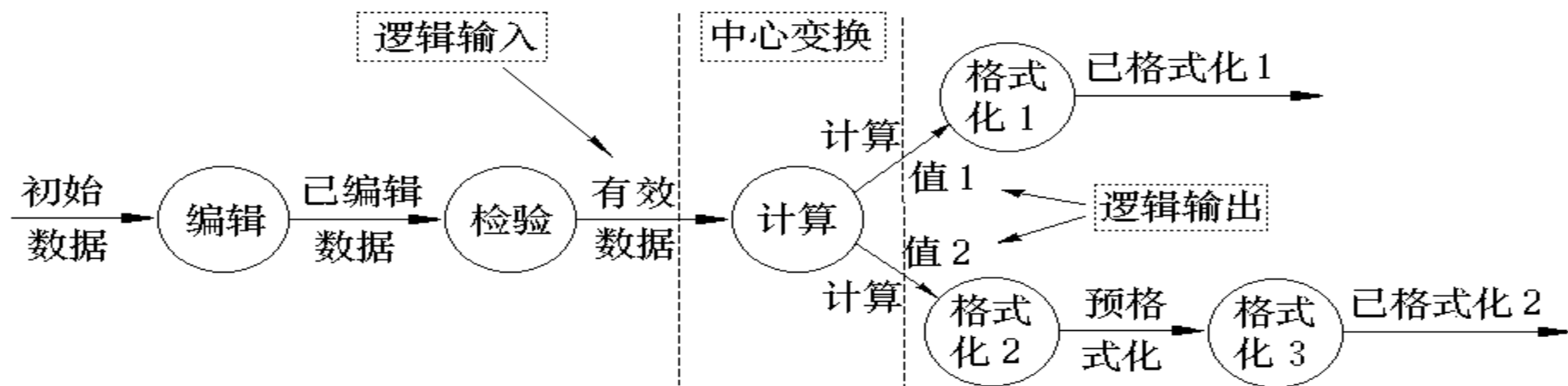
4.3.3 变换型数据流图映射方法



■ 变换分析方法由以下四步组成

3) 进行一级分解，设计上层模块；

- 为每一个逻辑输入设计一个输入模块，它为主模块提供数据；
- 为每一个逻辑输出设计一个输出模块，它将主模块提供的数据输出；
- 为中心变换设计一个变换模块，它将逻辑输入转换成逻辑输出。



■ 变换分析方法由以下四步组成

4) 进行二级分解：设计输入、输出和中心变换部分的中、下层模块。

● 输入模块

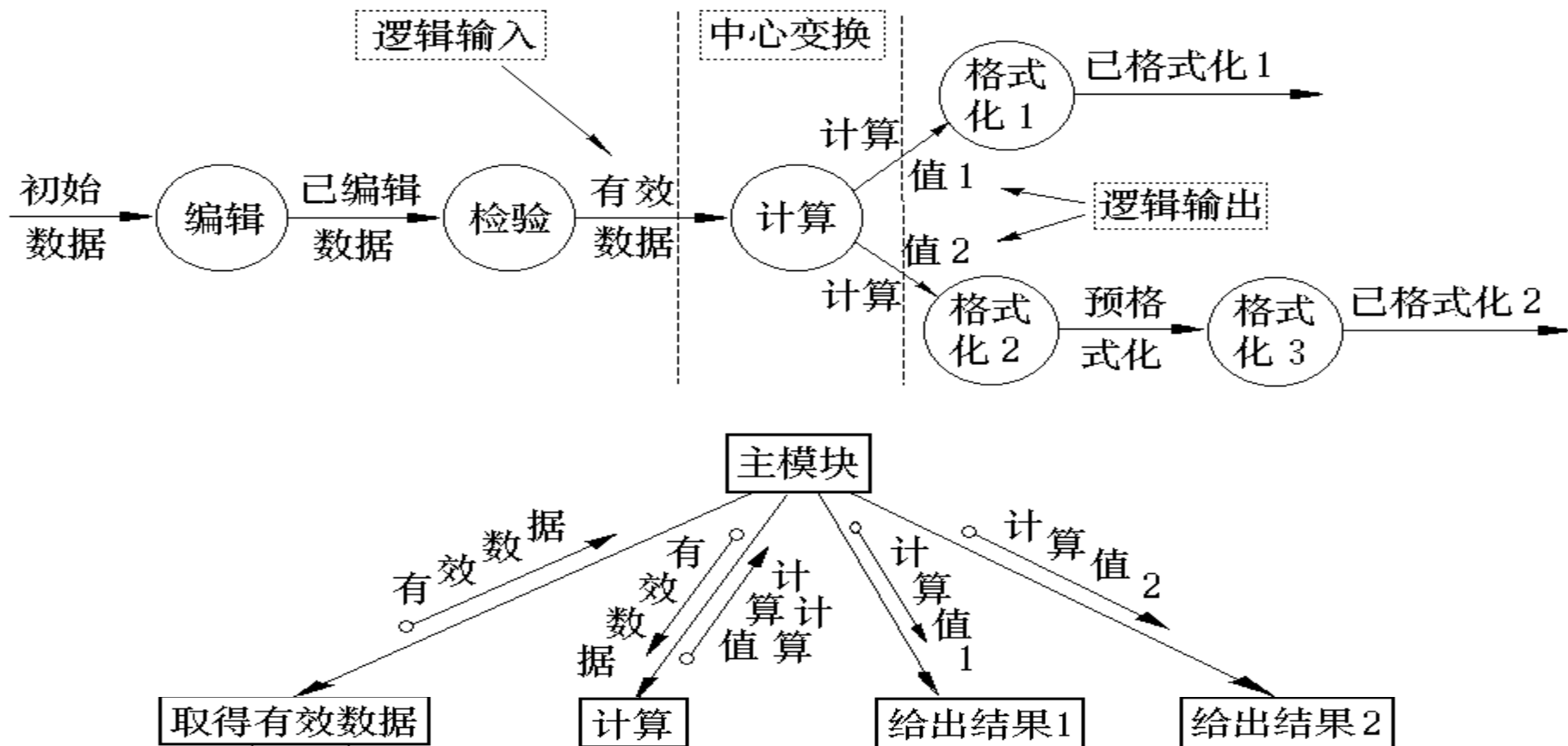
输入模块要向调用它的上级模块提供数据，因而它必须有两个下属模块：**一个是接收数据（子输入模块）；另一个是把这些数据变换成它的上级模块所需的数据（子变换）。**

● 输出模块

输出模块是从调用它的上级模块接收数据，用以输出，因而也应当有两个下属模块：**一个是将上级模块提供的数据变换成输出的形式（子变换模块）；另一个是将它们输出（子输出模块）。**

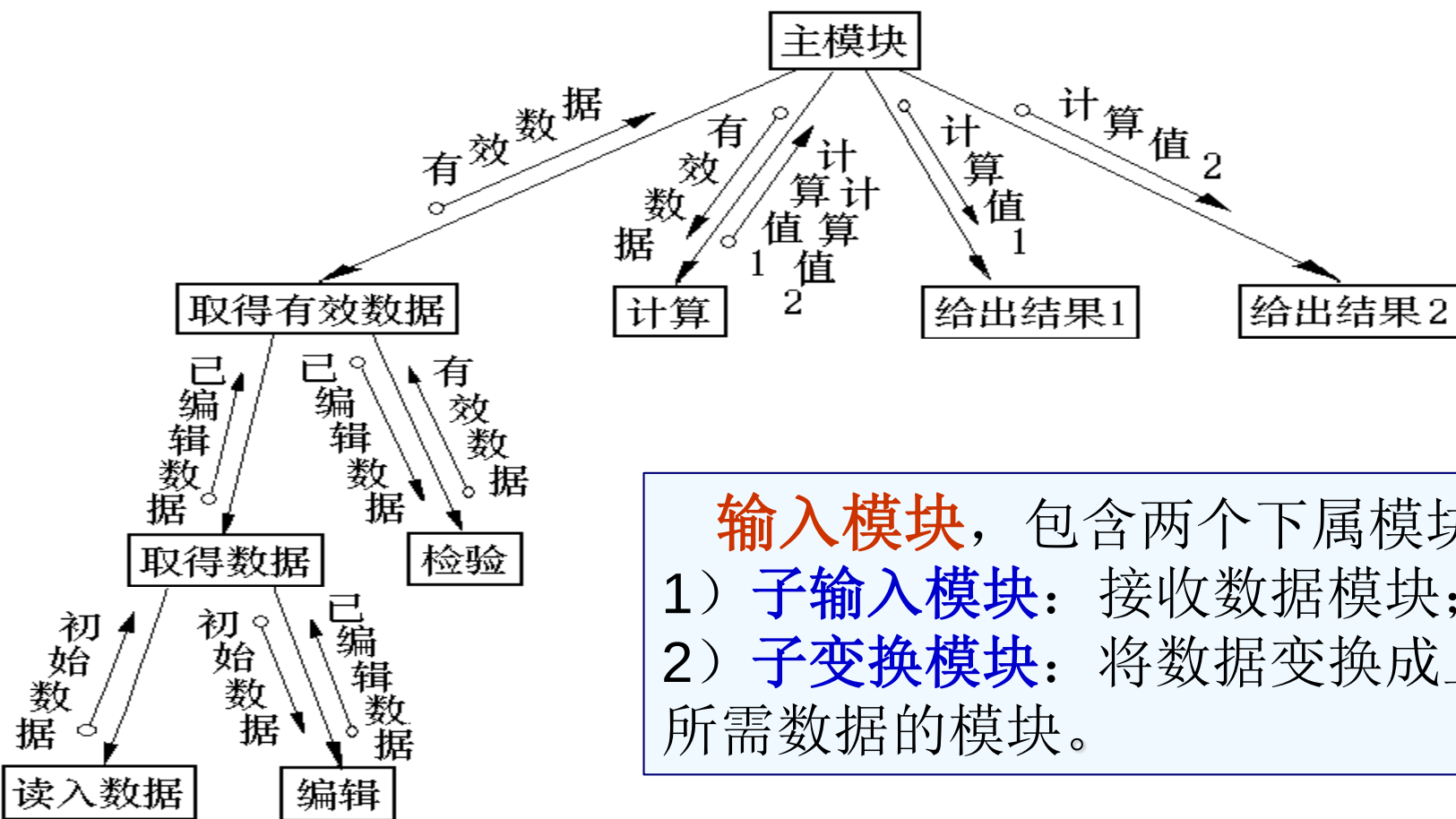
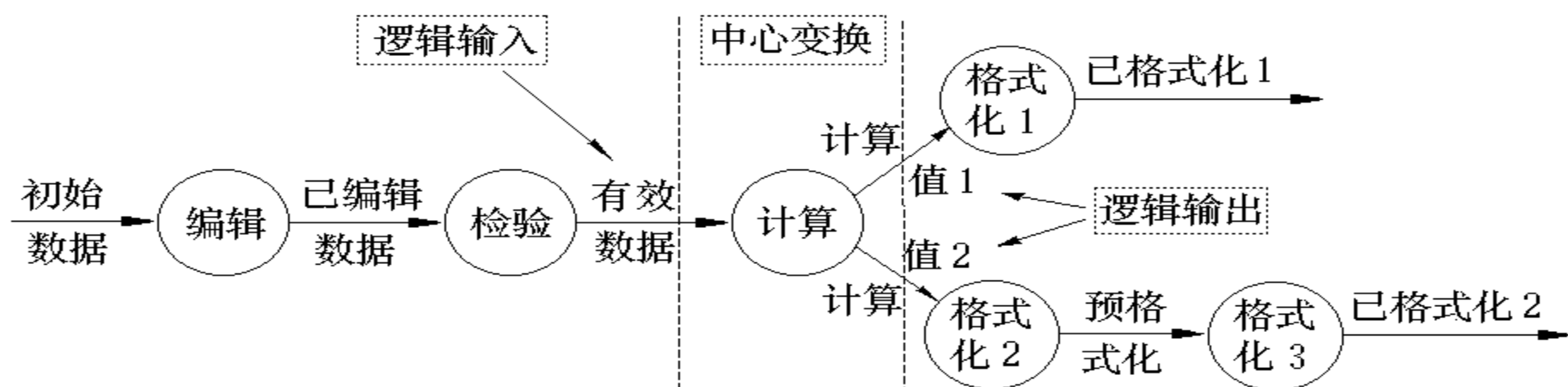
● 中心变换模块

没有通用的设计方法，一般应参照数据流图的中心变换部分和功能分解的原则来考虑如何对中心变换模块进行分解。



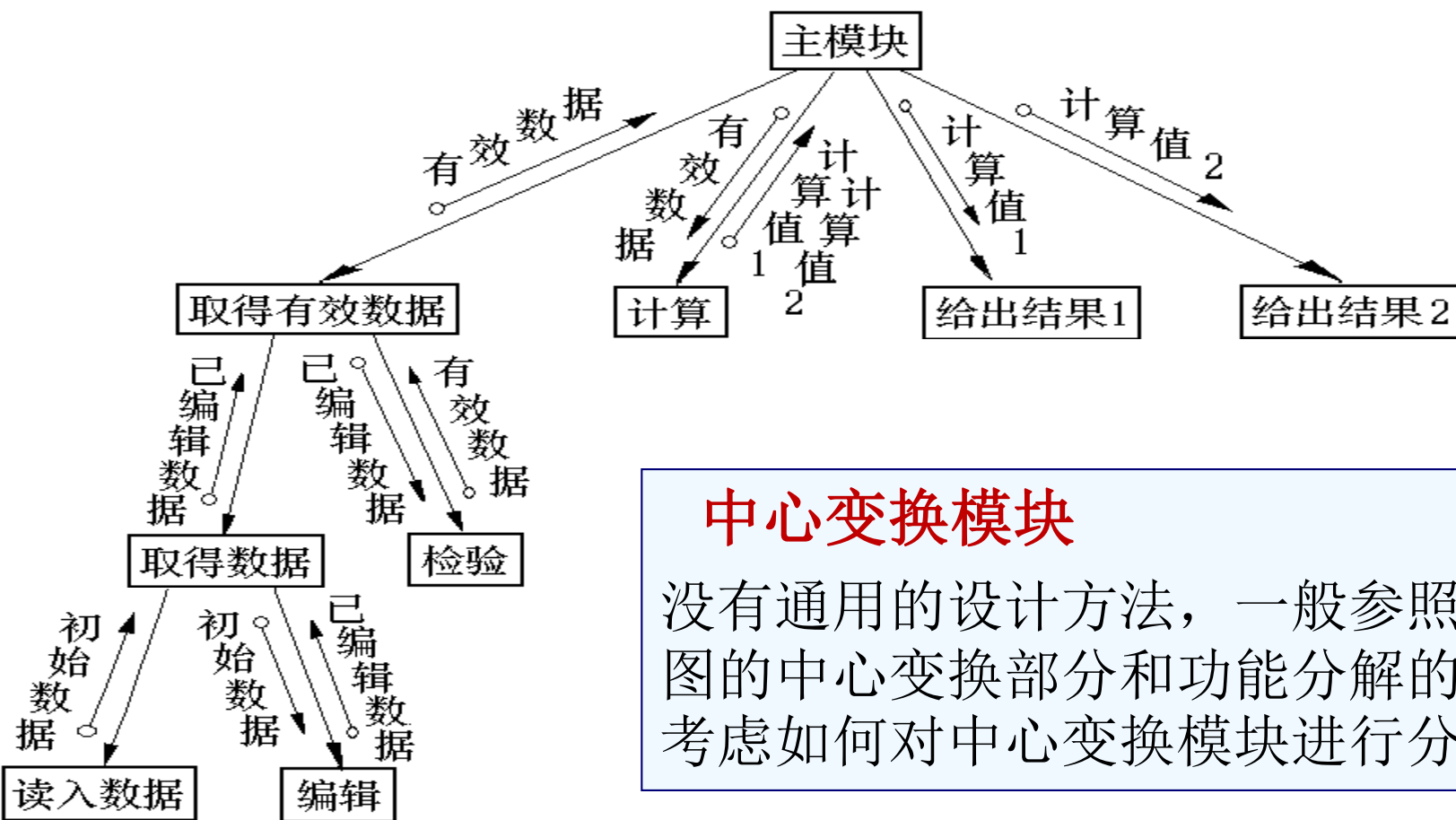
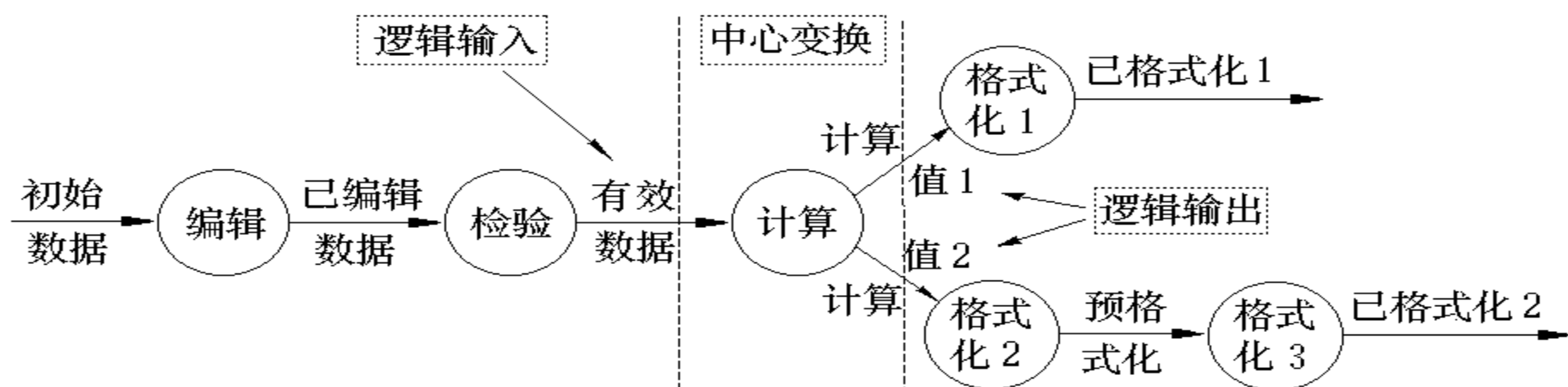
输入模块包含两个两个下属模块：

- 1) **子输入模块**：接收数据模块；
- 2) **子变换模块**：将数据变换成上级模块所需数据的模块。



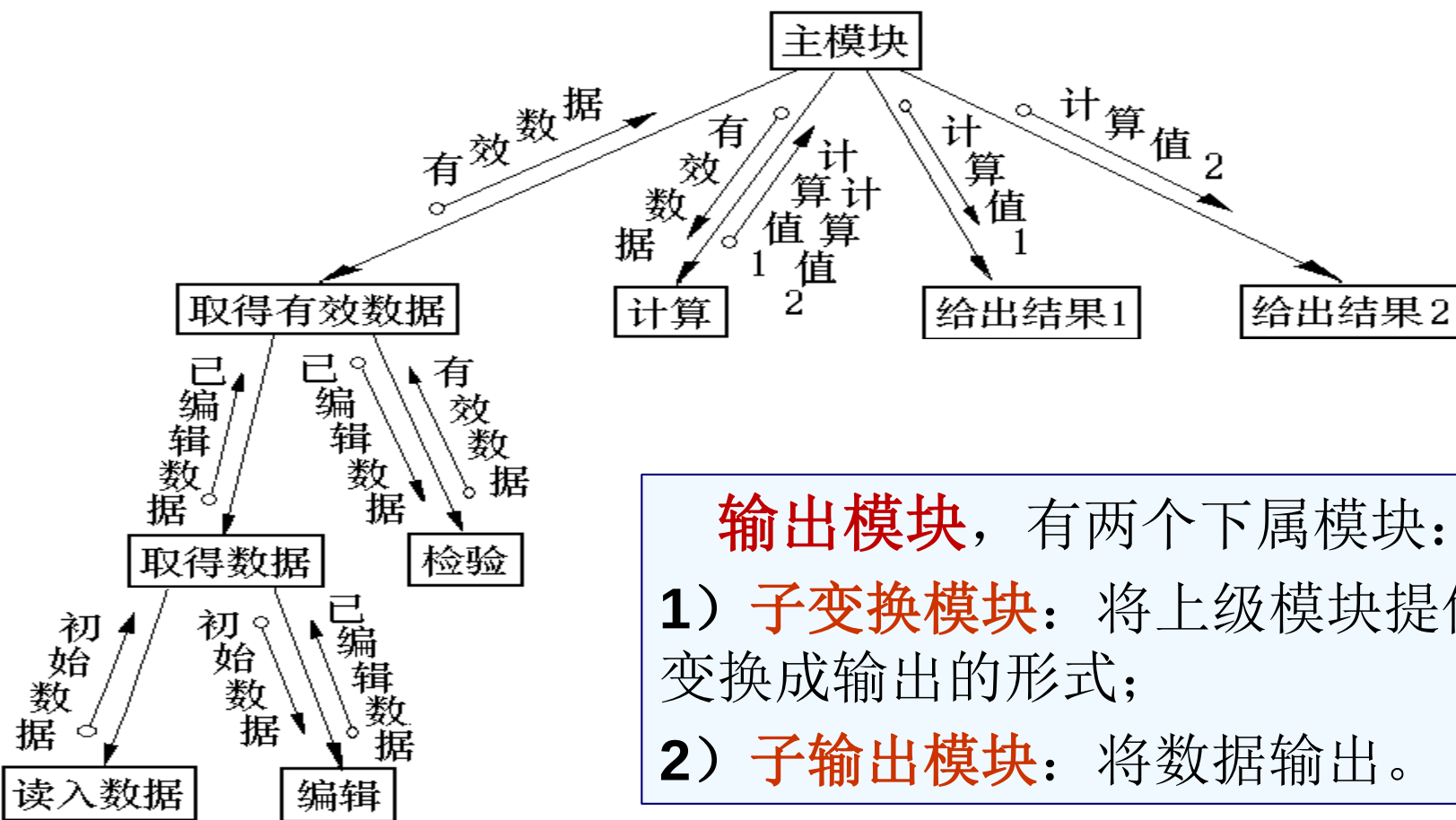
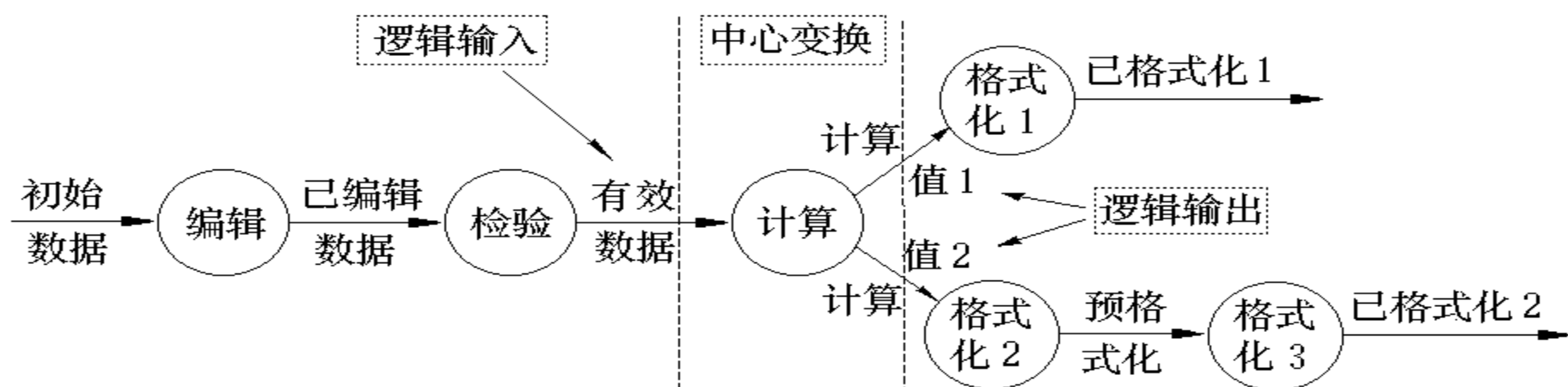
输入模块，包含两个下属模块：

- 1) **子输入模块**：接收数据模块；
- 2) **子变换模块**：将数据变换成上级模块所需数据的模块。



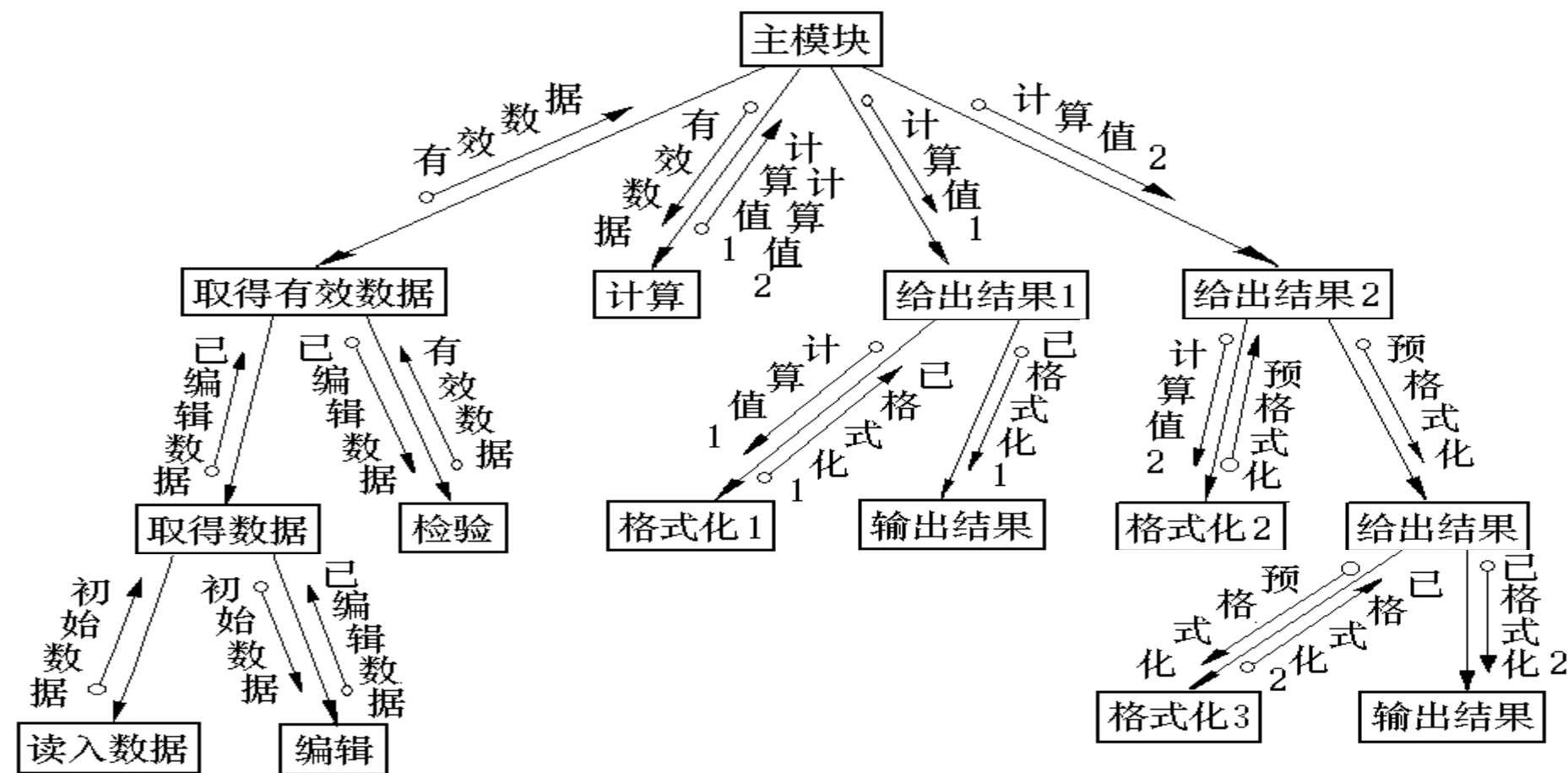
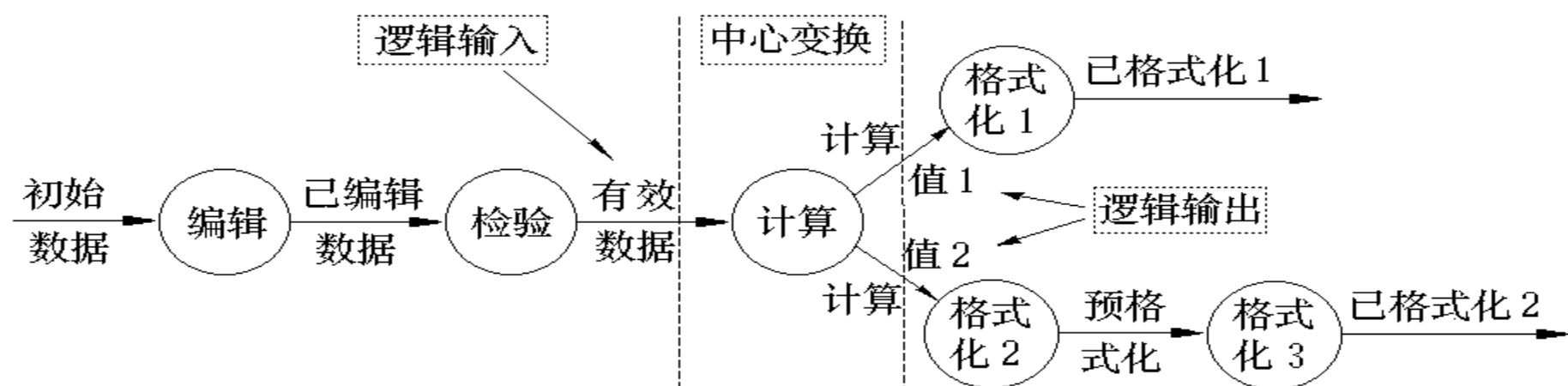
中心变换模块

没有通用的设计方法，一般参照数据流图的中心变换部分和功能分解的原则来考虑如何对中心变换模块进行分解。

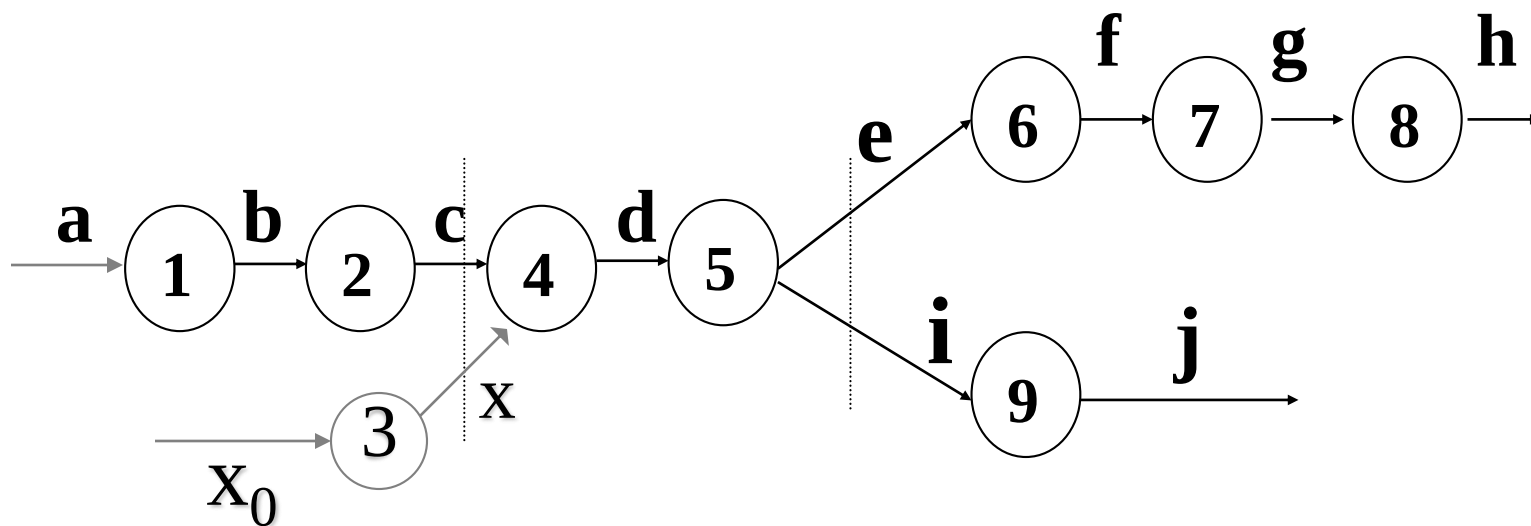
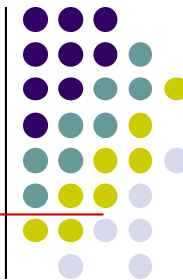


输出模块，有两个下属模块：

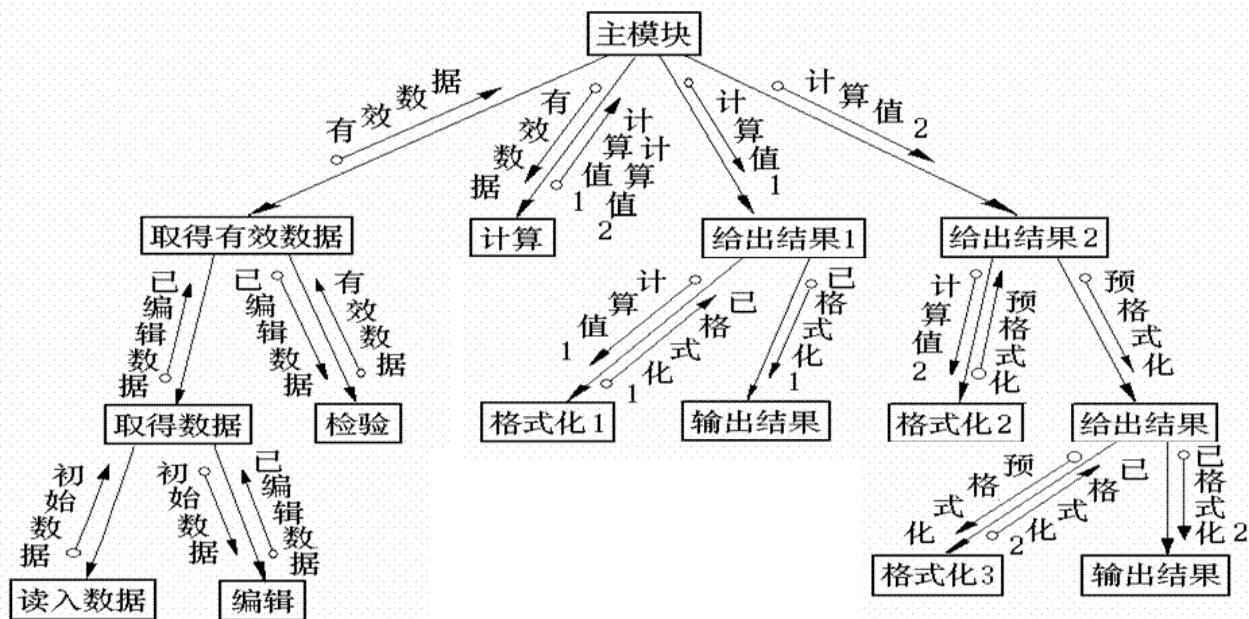
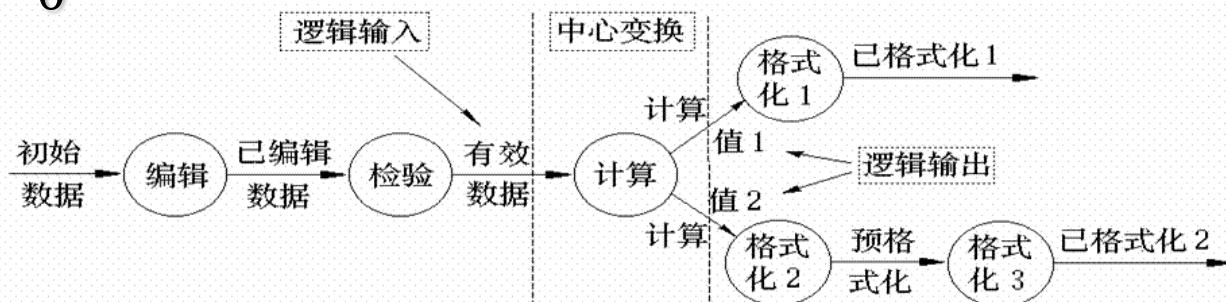
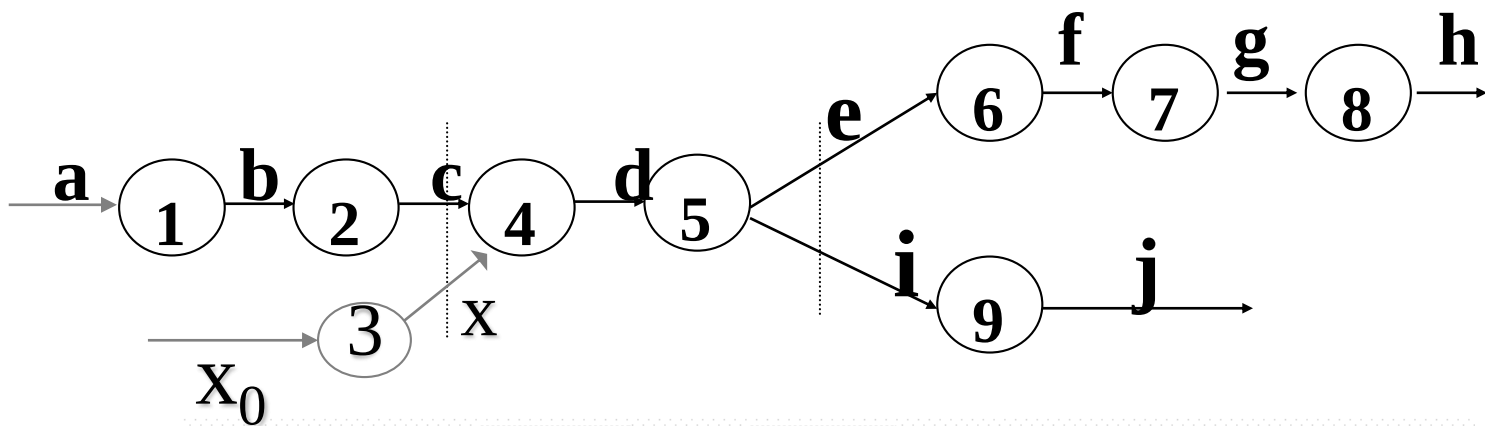
- 1) **子变换模块**：将上级模块提供的数据变换成输出的形式；
- 2) **子输出模块**：将数据输出。

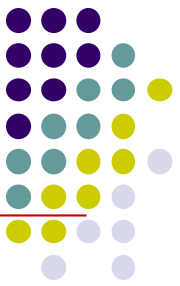


将下图所示的DFD图转换为SC图（变换分析）



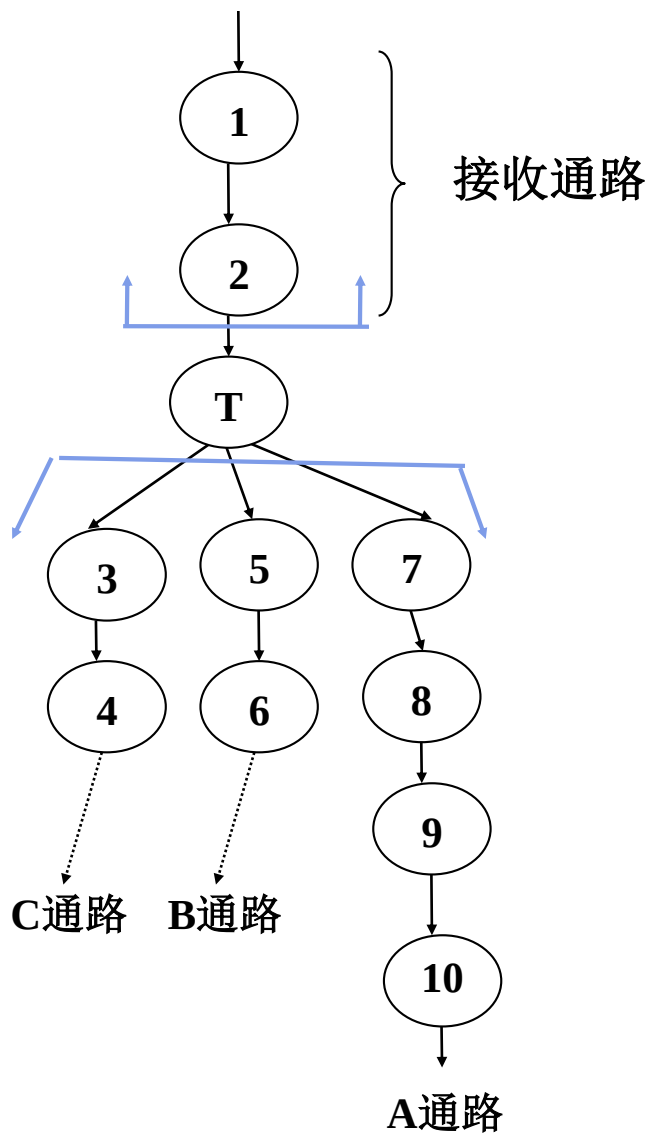
（注：虚线表示逻辑输入和逻辑输出）



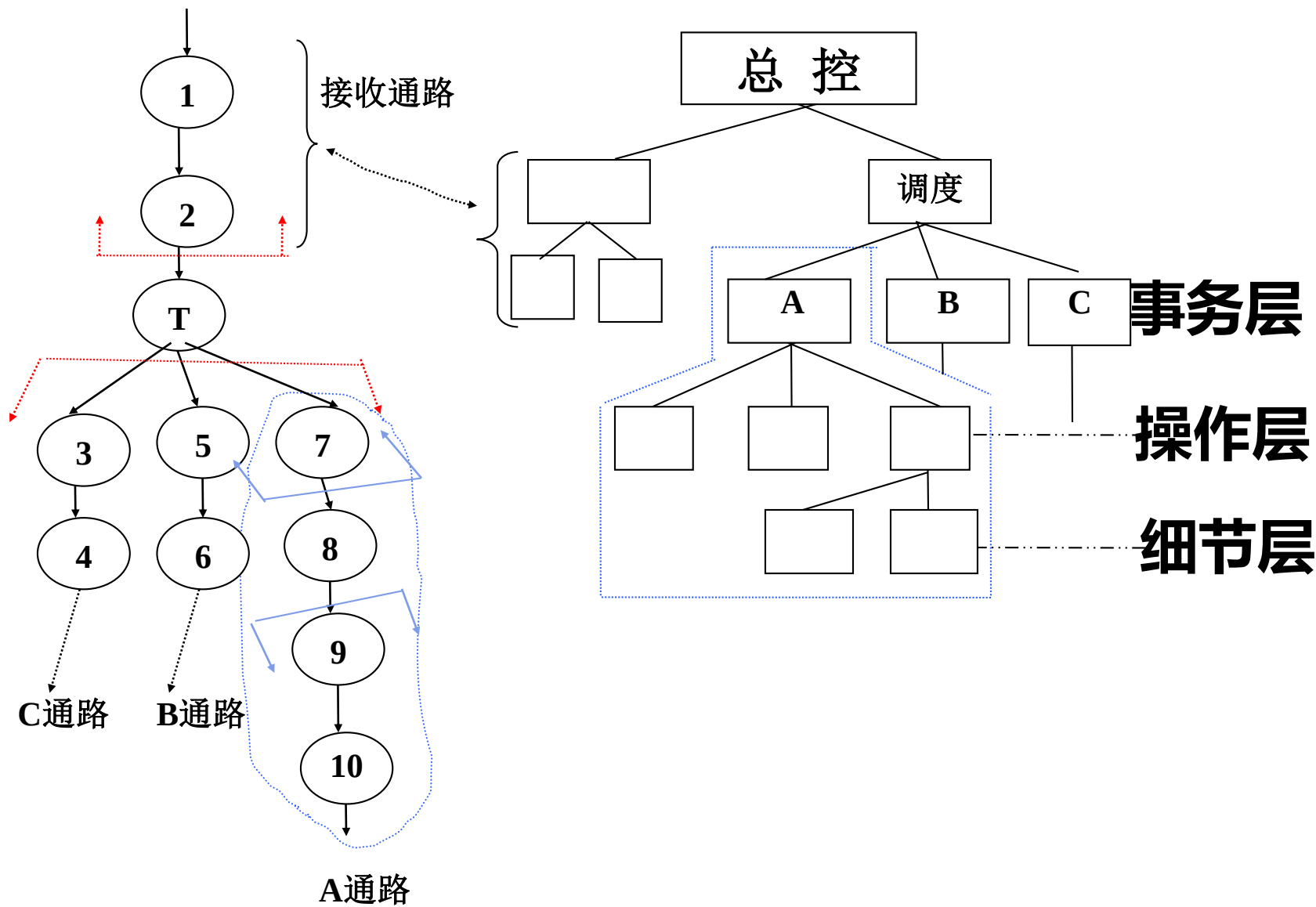


4.3.4 事务型映射方法

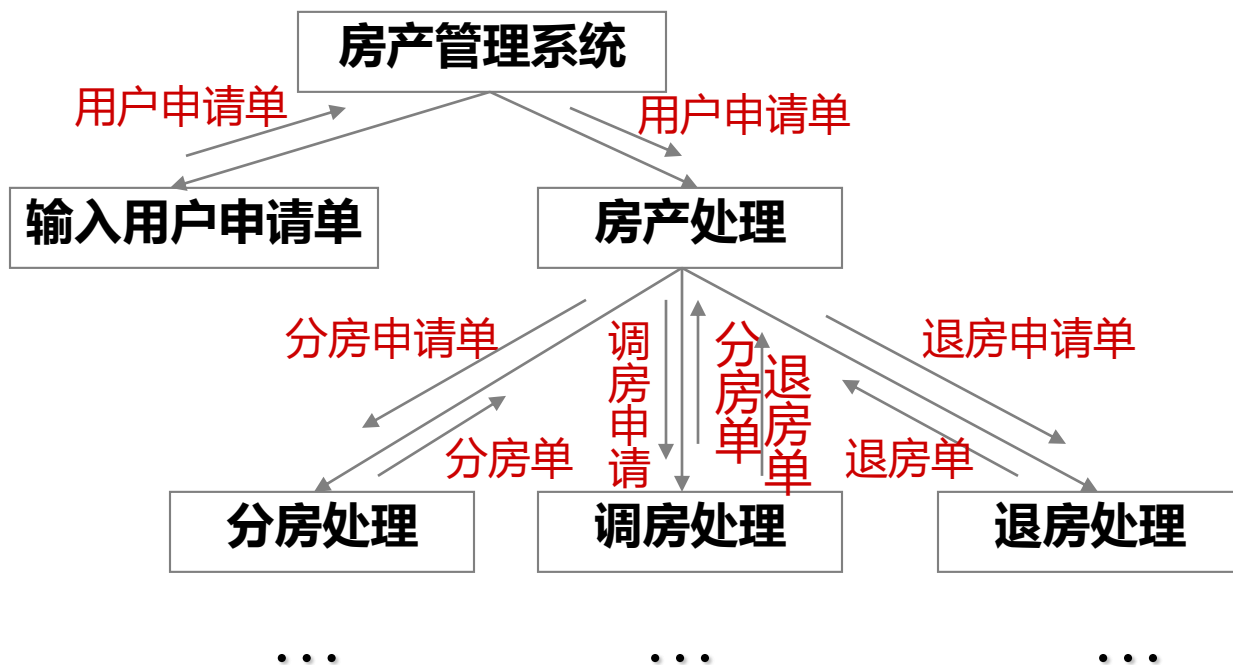
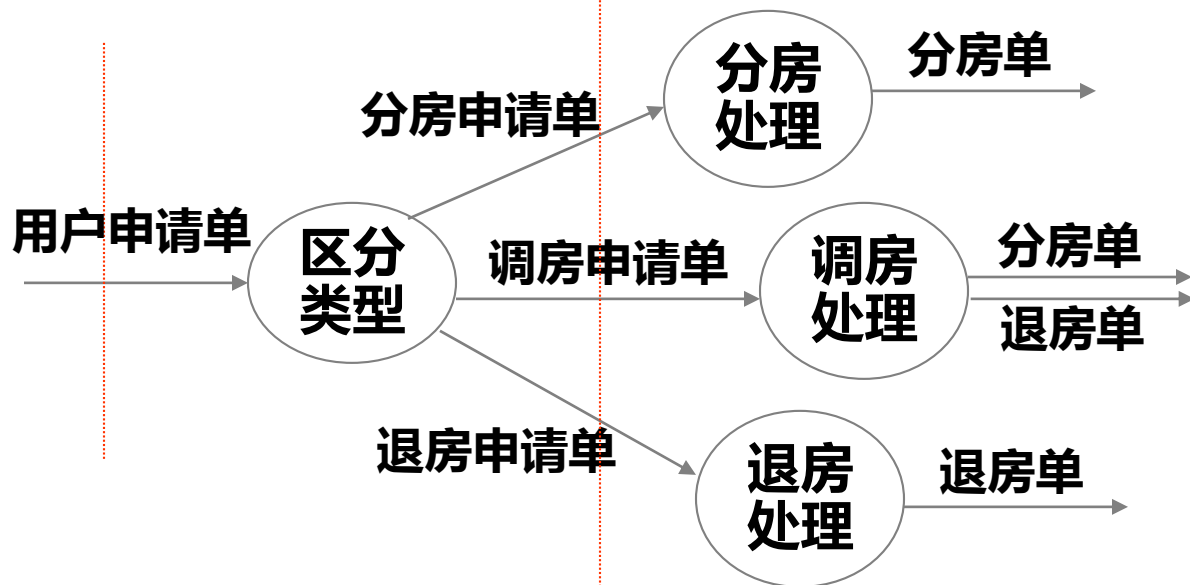
- 与变换分析一样，事务分析也是从分析数据流图开始，自顶向下，逐步分解，建立系统结构图。
- 由事务流映射的软件结构包括一个接收分支和一个发送分支。
- 从事务中心的边界开始，把沿着接收流通路的处理映射成模块。
- 发送分支的结构包含一个调度模块，它控制下层的所有活动模块；然后把数据流图中的每一个活动流通路映射成与它的流特征相对应的结构。



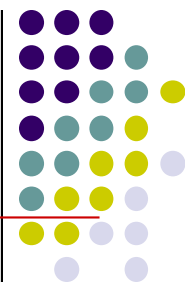
事务分析的映射方法



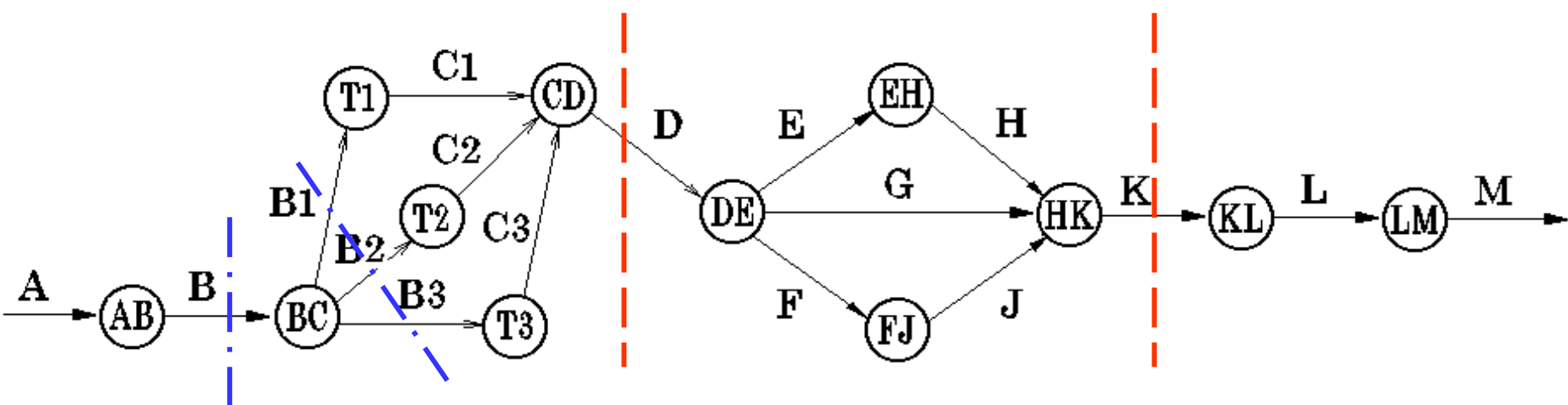
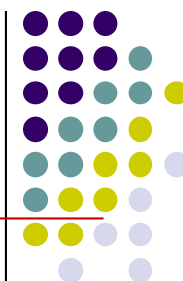
事务分析的映射方法

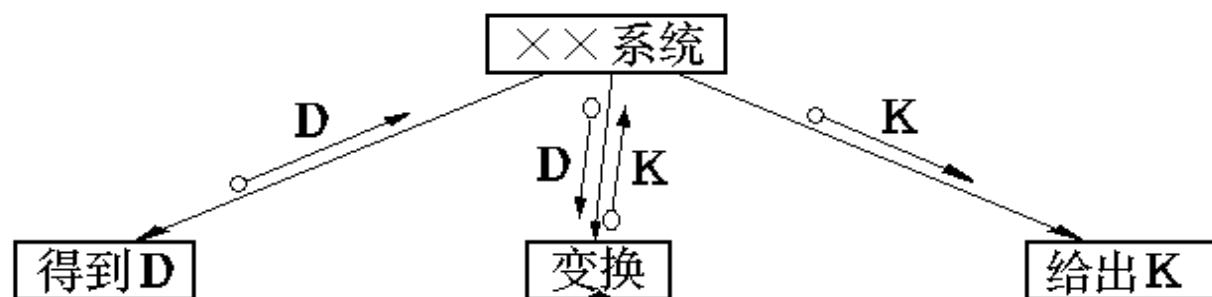
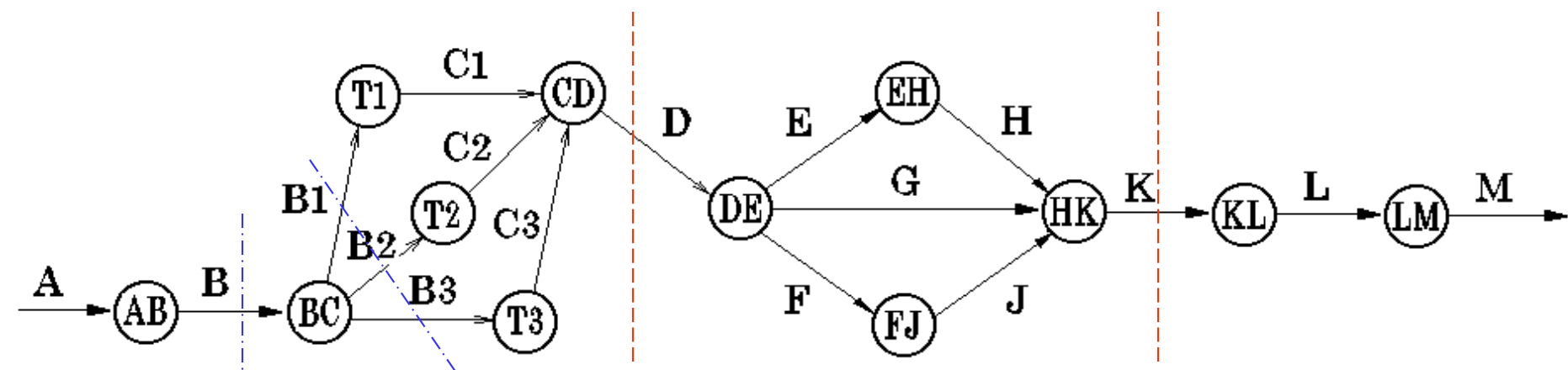


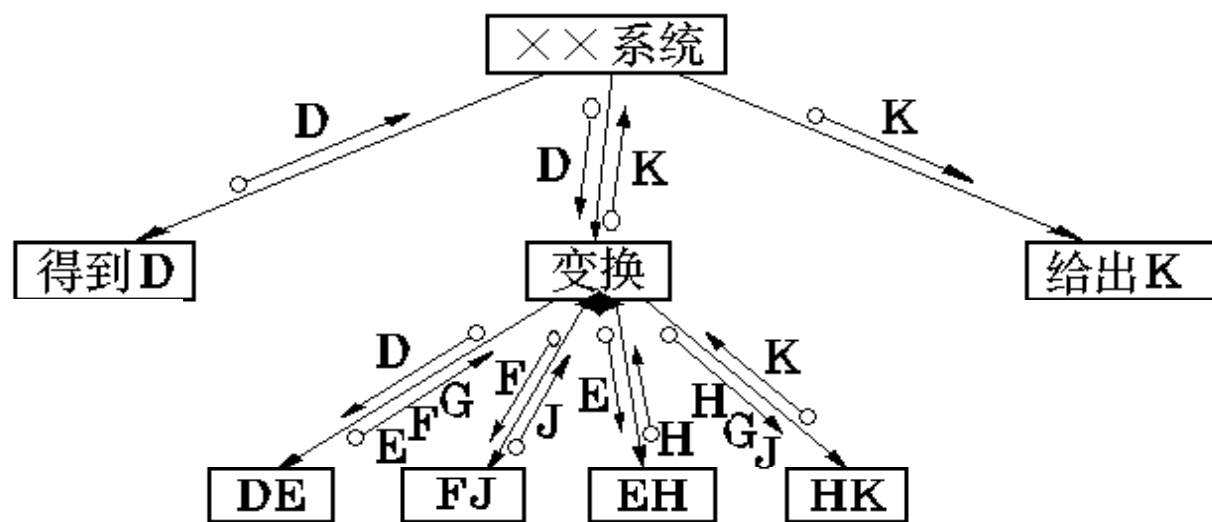
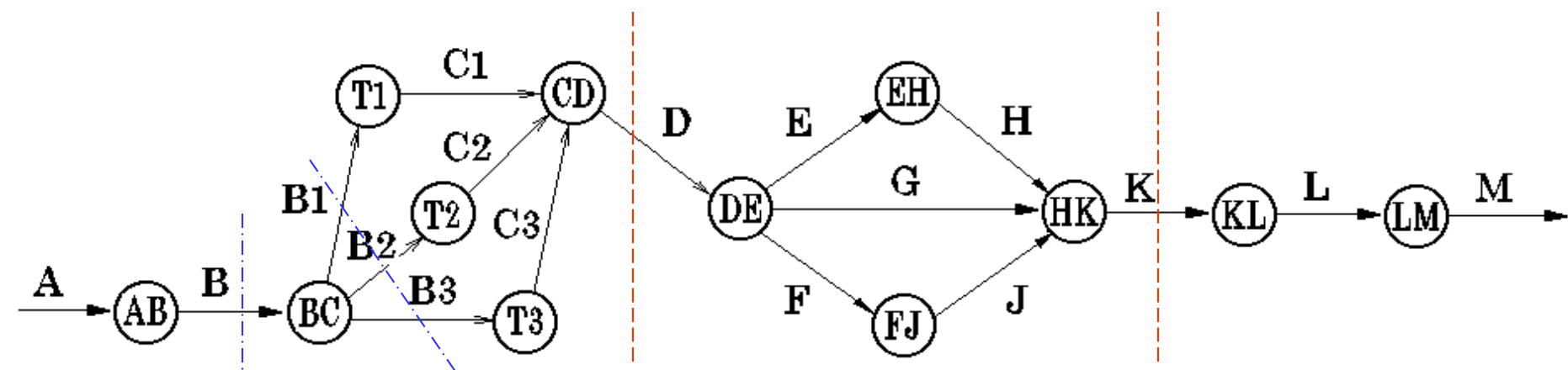
混合型结构的映射方法

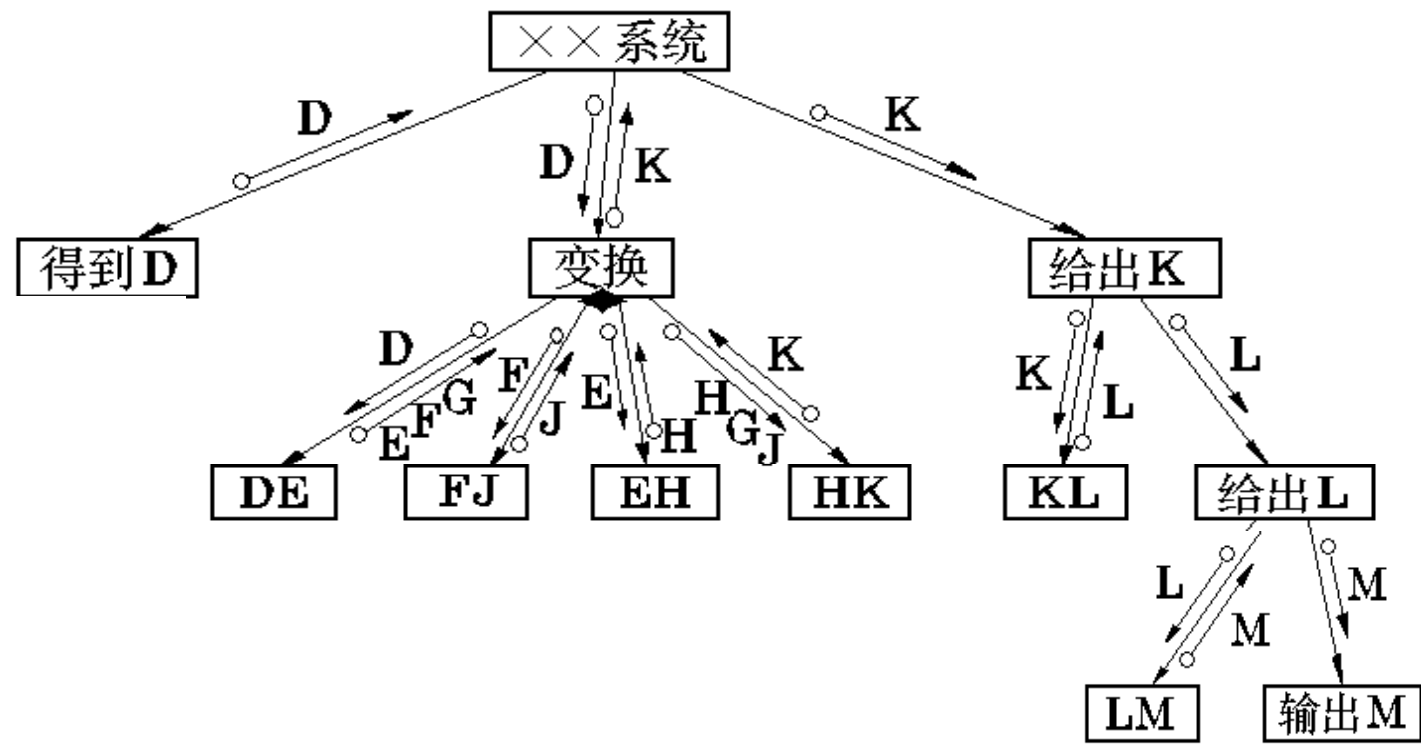
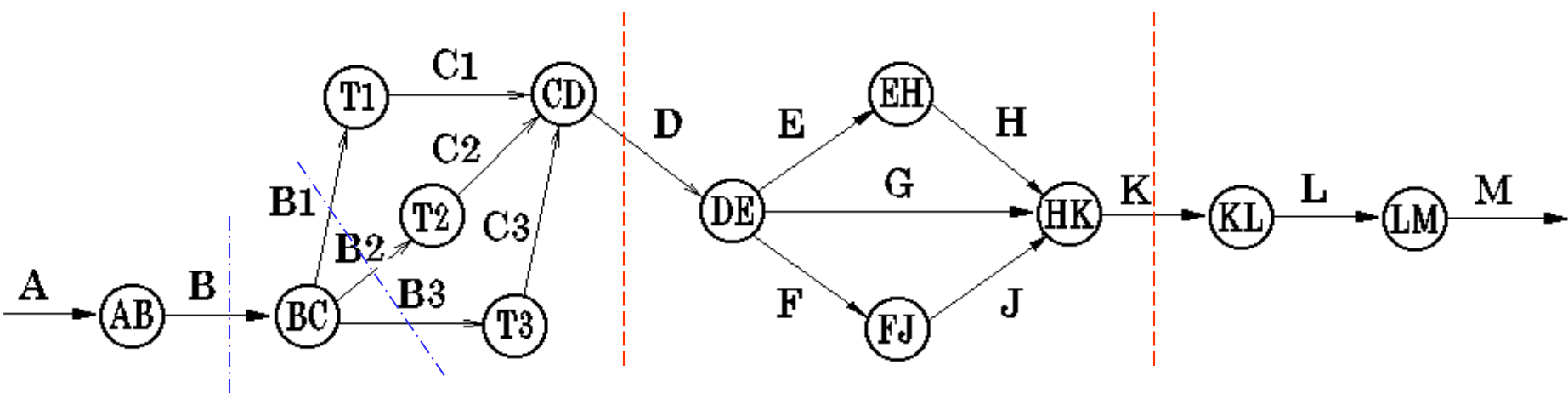


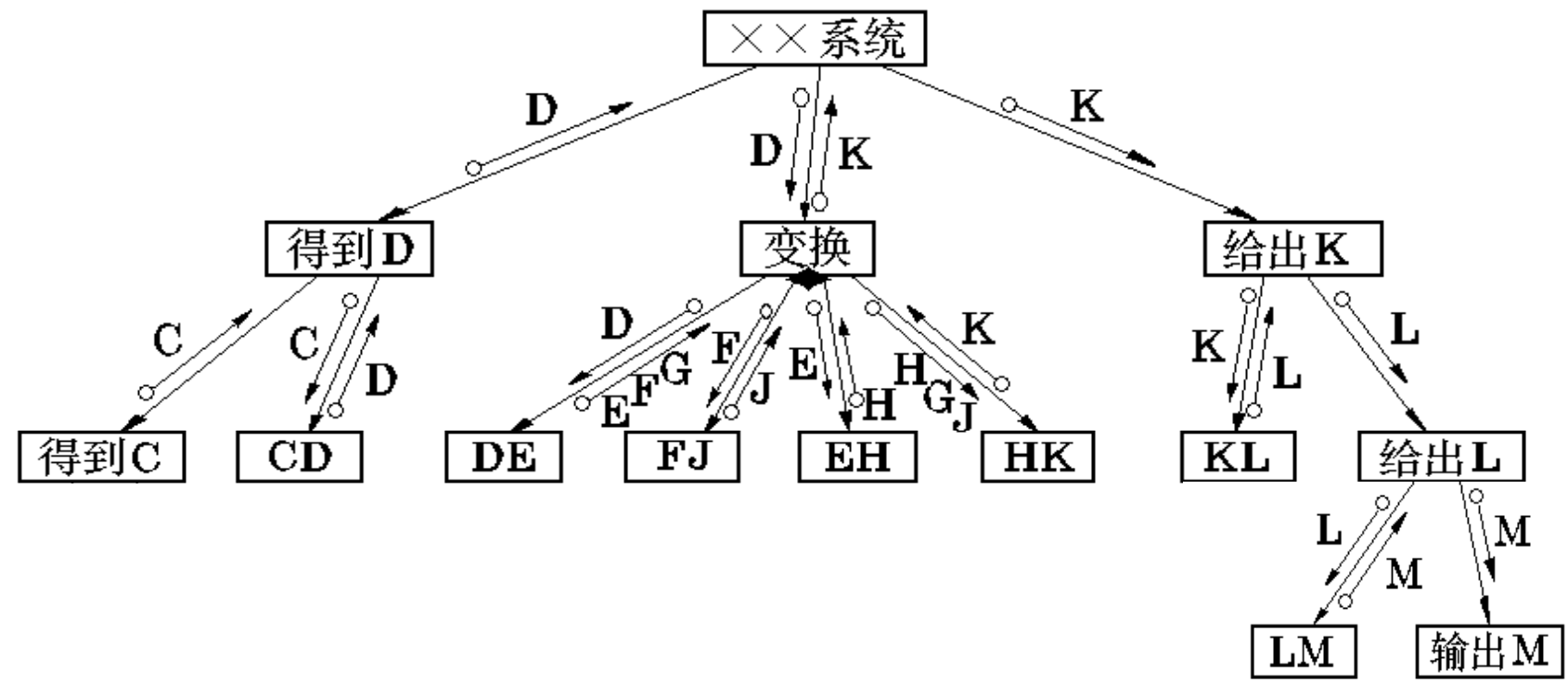
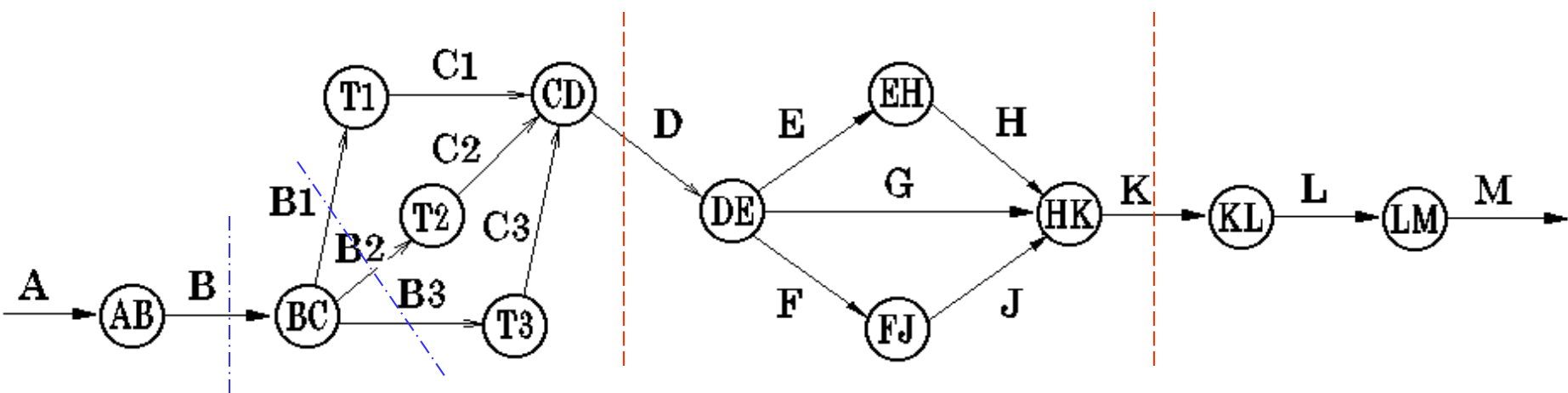
- 大型的软件系统往往是**变换型结构**和**事务型结构的混合结构**。
- 根据DFD中占优势的属性，确定DFD的全局属性。
- 把和全局属性不同的局部区域孤立出来，以后按照这些子系统的特点精化根据全局属性得出的软件结构。
- 通常以**变换分析为主，事务分析为辅**的方式进行软件结构设计。

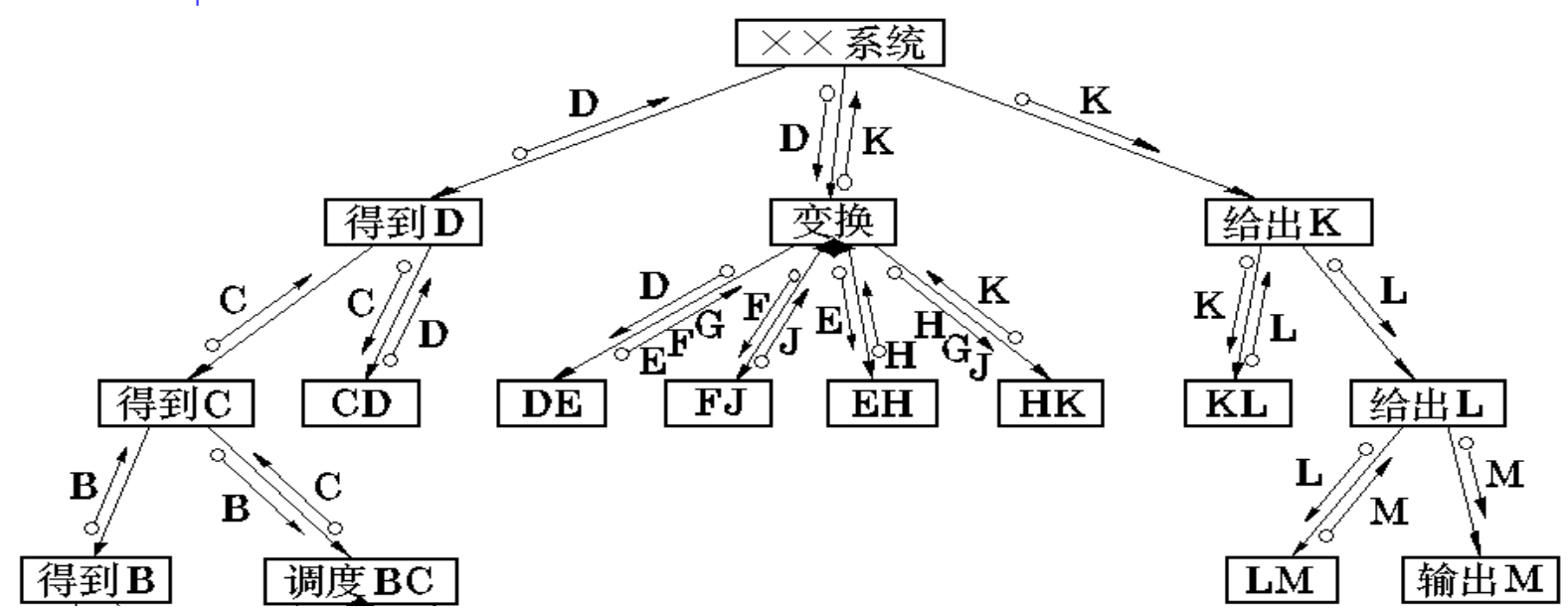
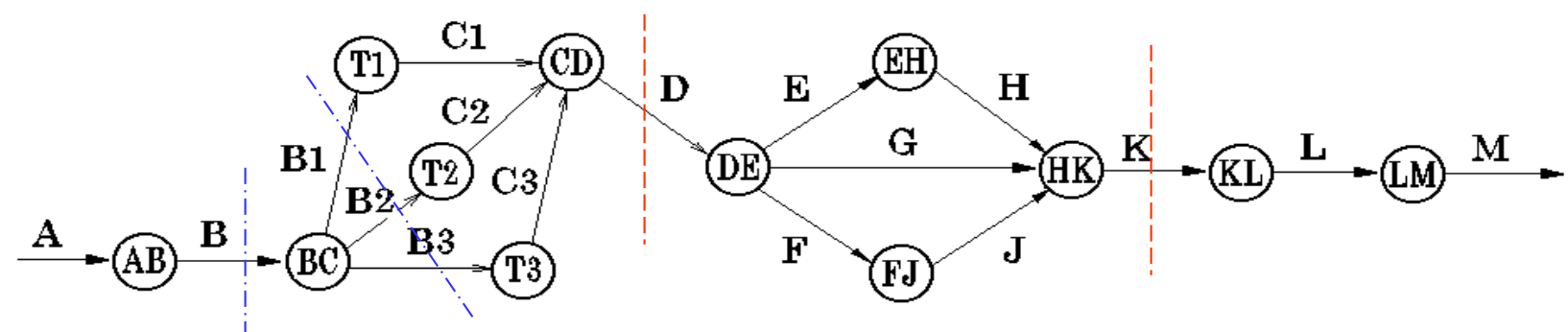
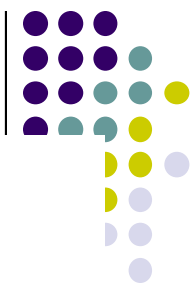


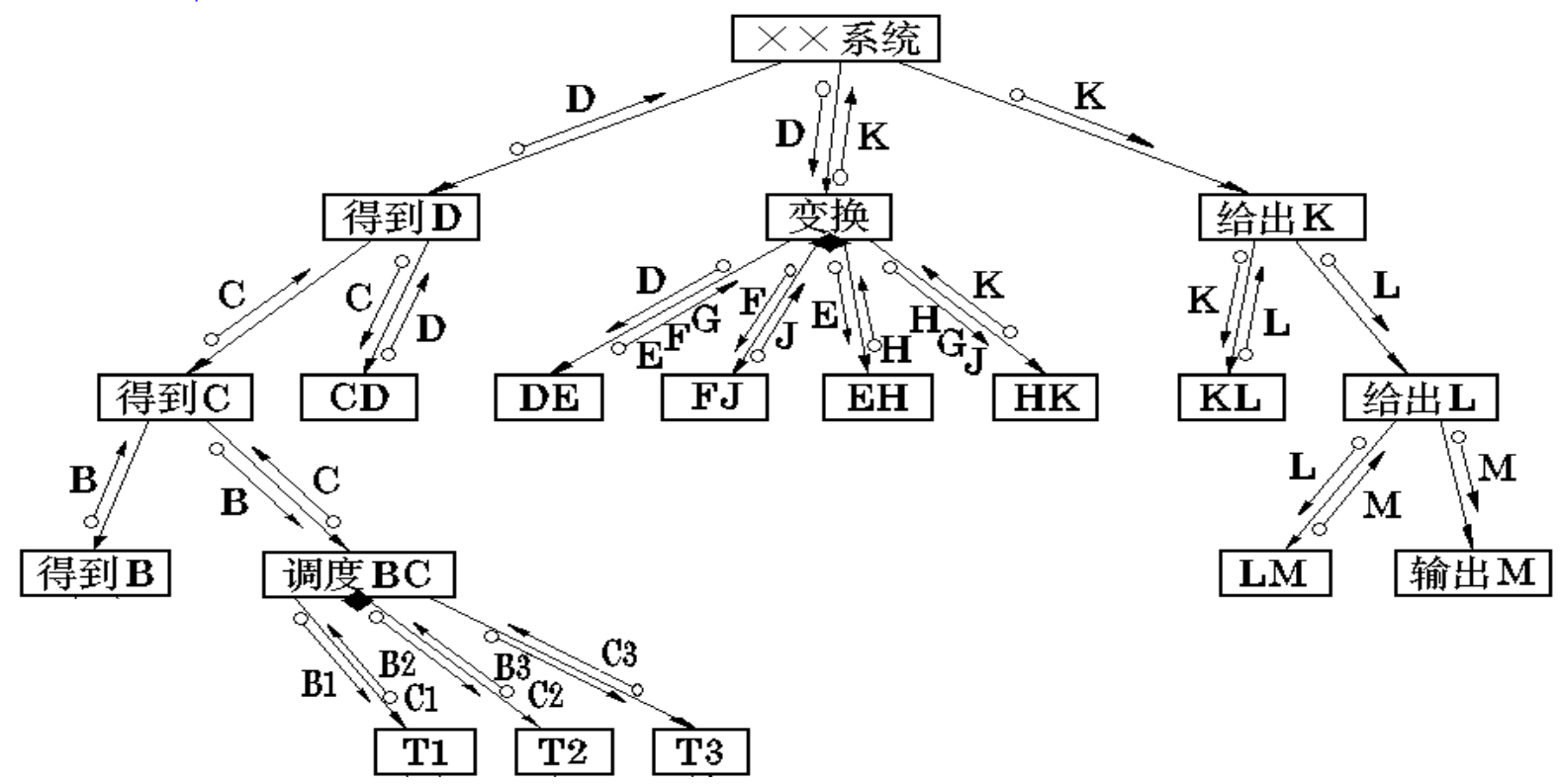
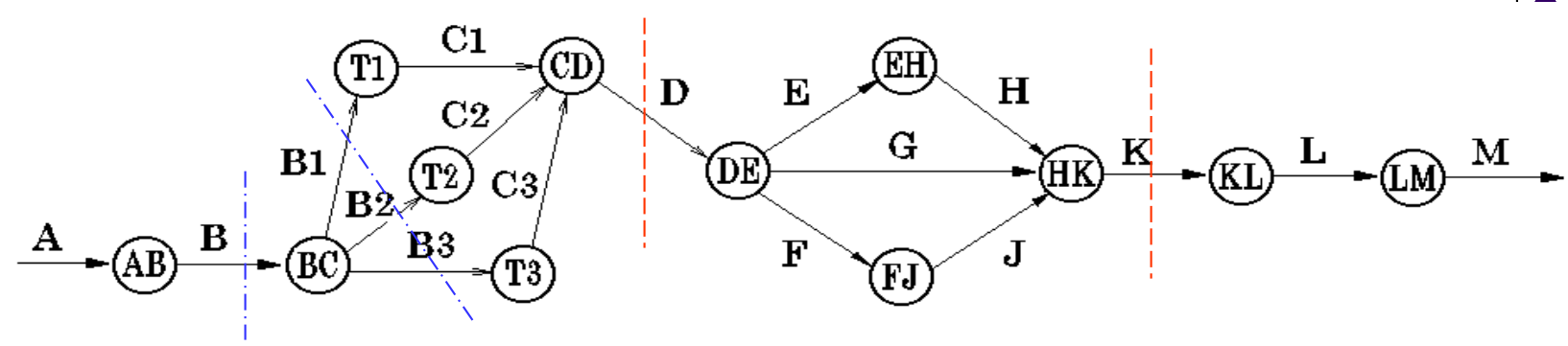


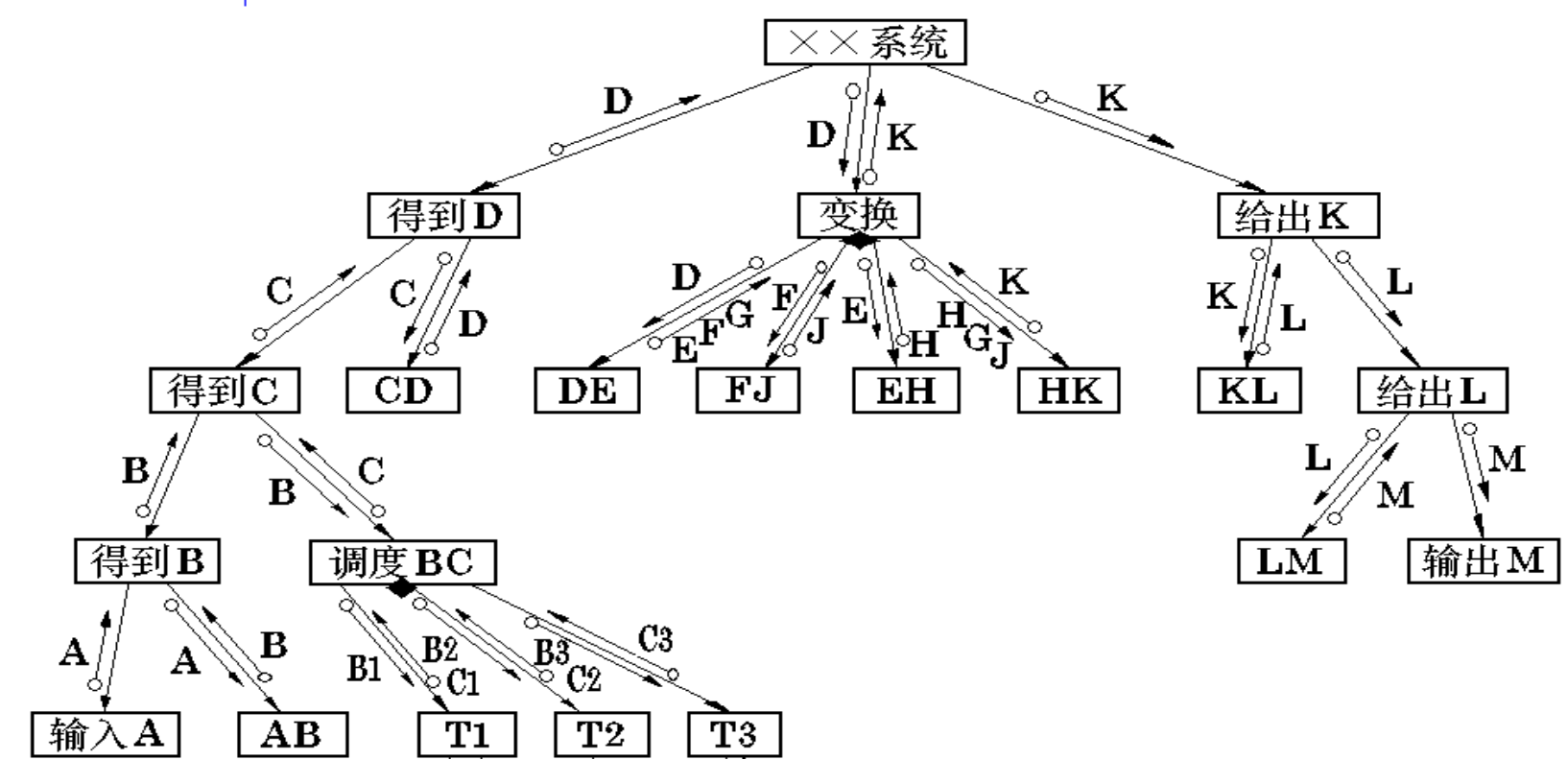
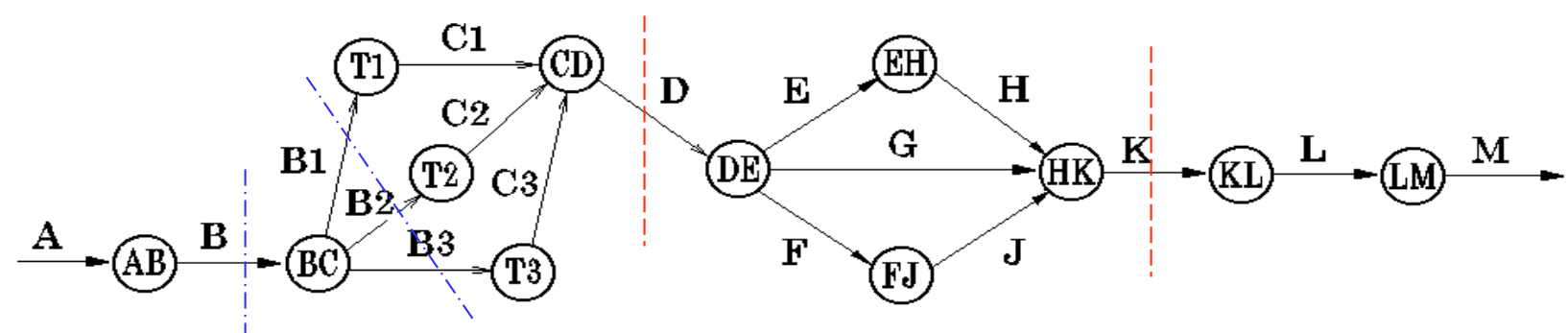


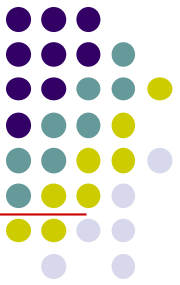










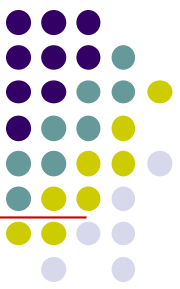


4.3.5 软件模块结构的改进方法

■ 改进软件结构提高模块独立性

设计出软件的初步结构后，应审查分析，通过分解和合并，力求降低耦合提高内聚。

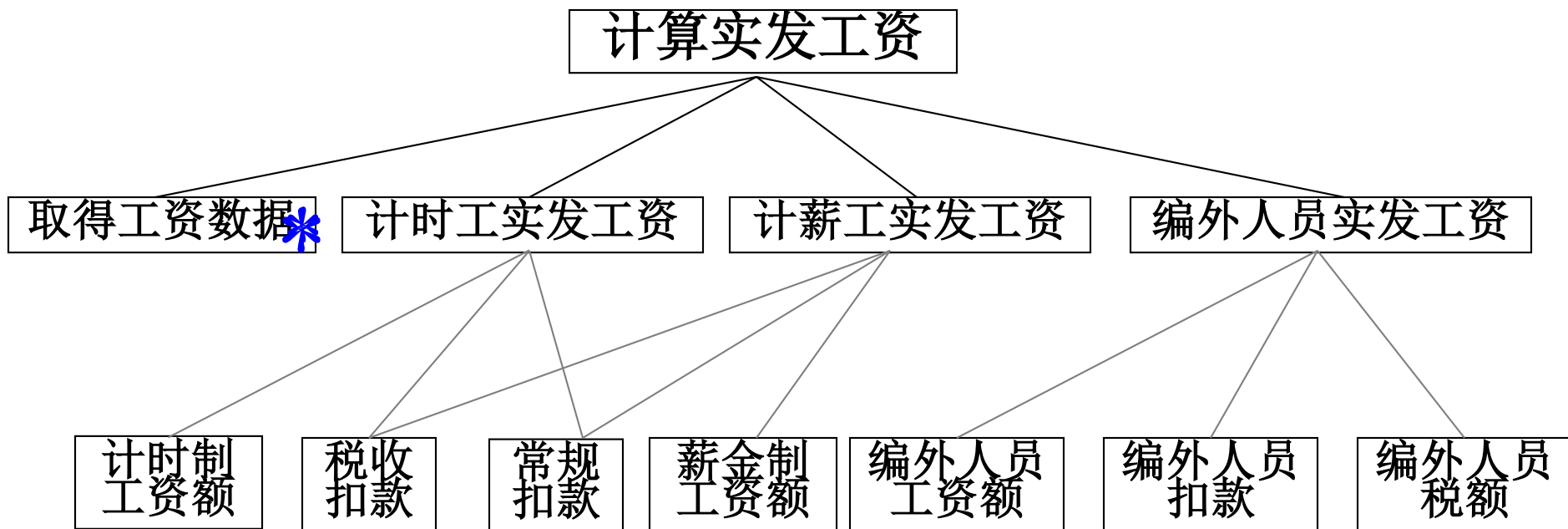
如：分解模块使控制耦合变成数据耦合。



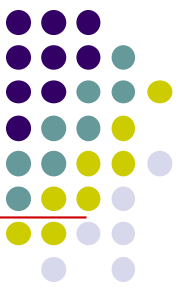
4.3.5 软件模块结构的改进方法

■ 模块的作用域应在控制域之内

- 模块的**控制域**包括它本身及其所有的从属模块。 ▶
- 模块的**作用域**是指模块内一个判定的作用范围，
凡是受这个判定影响的所有模块都属于这个判定的作用范围。 ▶
 - 整个模块是否执行，依赖于判定的结果
 - 上述模块的下属模块
 - 模块内有部分功能的执行依赖于判定的结果



- 整个模块是否执行，依赖于判定的结果
- 上述模块的下属模块
- 模块内有部分功能的执行依赖于判定的结果



4.3.5 软件模块结构的改进方法

■ 作用域/控制域原则

- 如果一个判定的作用范围包含在这个判定所在模块的控制范围之内，则这种结构是简单的。
- 违反原则的解决方法
 - 1)将系统中有较大影响的判定从低层次上移到足够高的上层模块中。
 - 2)受判定影响的模块调整并下移到控制域之内。

计算实发工资

取得工资数据*

计时工实发工资

计薪工实发工资

编外人员实发工资

计时制
工资额

税收
扣款

常规
扣款

薪金制
工资额

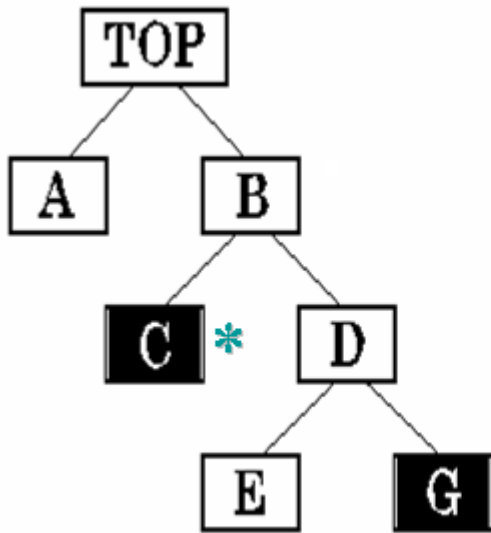
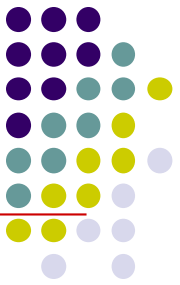
编外人员
工资额

编外人员
扣款

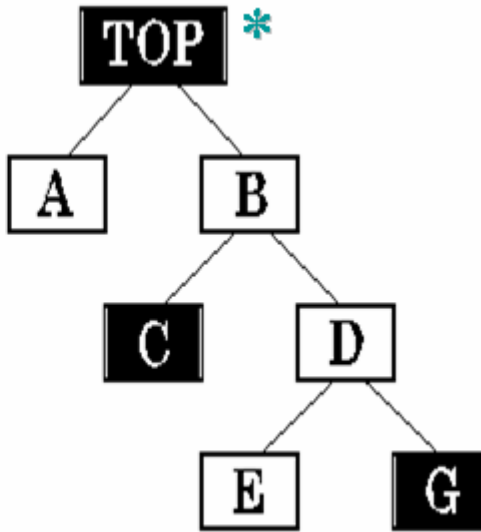
编外人员
税额

- 整个模块是否执行，依赖于判定的结果
- 上述模块的下属模块
- 模块内有部分功能的执行依赖于判定的结果

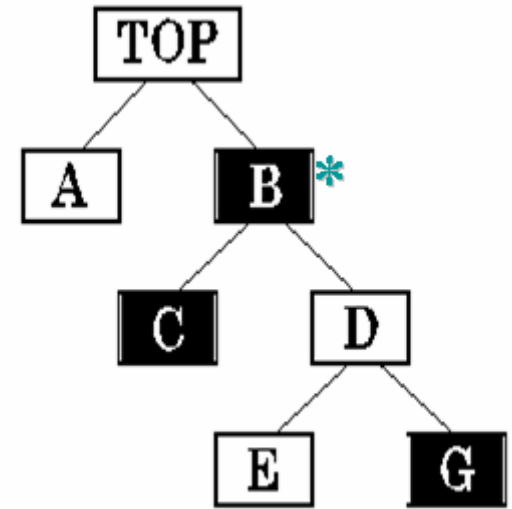
4.3.5 软件模块结构的改进方法



不符合作用域/控制域规则。
图中黑框表示判定的作用域。



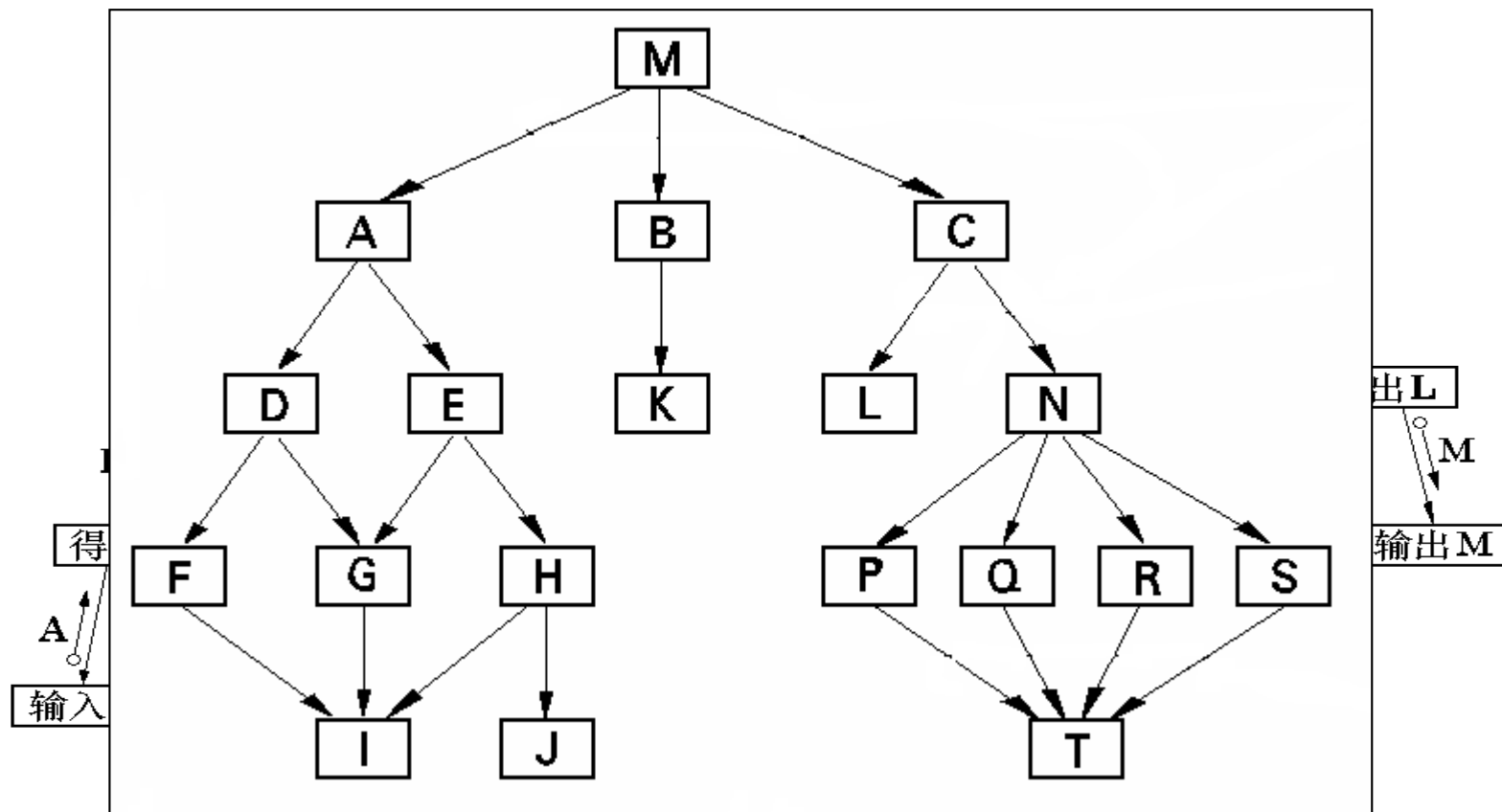
符合规则；
但判定所在模块TOP层次太高；
需要经过不必要的信号传送，增加了数据的传送量。

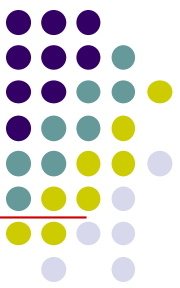


符合规则；
只有一个判定分支有一个不必要的穿越；
较好的结构。

4.3.5 软件模块结构的改进方法

- 系统结构图的深度、宽度、扇出、扇入应适当



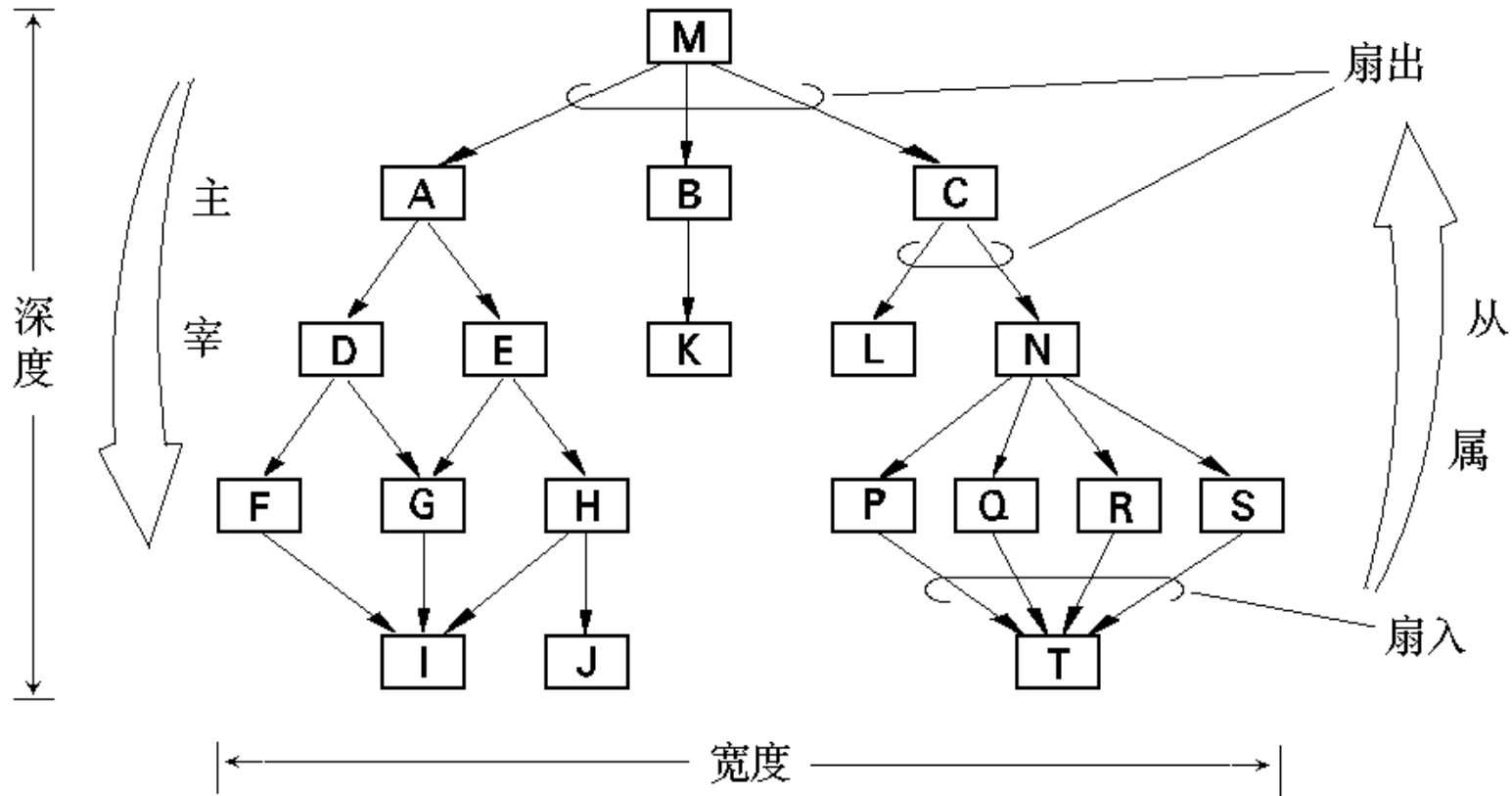


4.3.5 软件模块结构的改进方法

■ 系统结构图的深度、宽度、扇出、扇入应适当

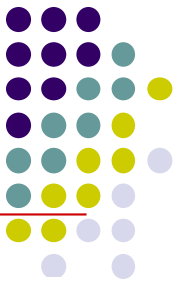
- **深度**：软件结构中控制的层数。
它粗略标志一个系统的大小和复杂程度。
- **宽度**：软件结构中同一层次上的模块总数的最大值，
宽度越大，系统越复杂。
- **扇出**：一个模块直接控制（调用）的下级模块数目。
适当的扇出为 $2 \sim 5$ ，最多不要超过9，
平均扇出是 $3 \sim 4$ 。
- **扇入**：调用（或控制）一个给定模块的模块个数。

4.3.5 软件模块结构的改进方法



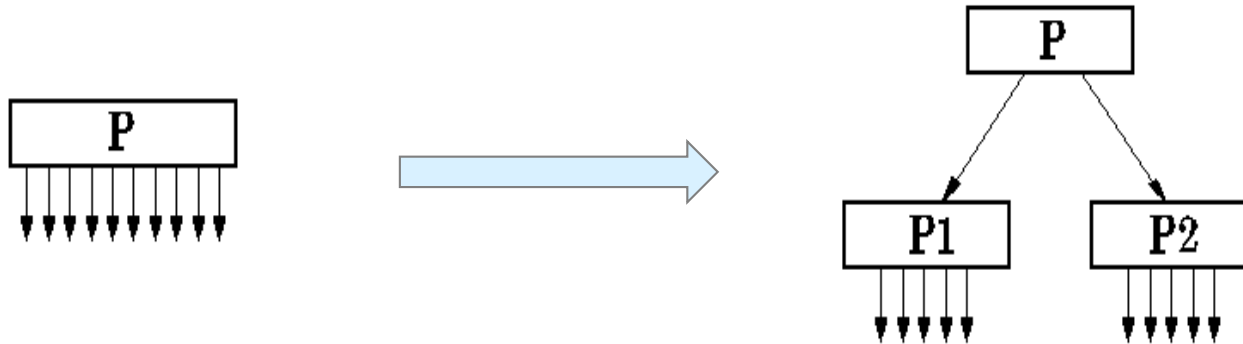
设计良好的软件模块结构，通常上层扇出比较高，中层扇出较少，底层扇入到有高扇入的公共模块中。

4.3.5 软件模块结构的改进方法



■ 尽可能减少高扇出结构，随着深度增大扇入。

- 如果一个模块的**扇出数过大**，就意味着该模块过分复杂，需要协调和控制过多的下属模块。
- 应当适当**增加中间层次的控制模块**。

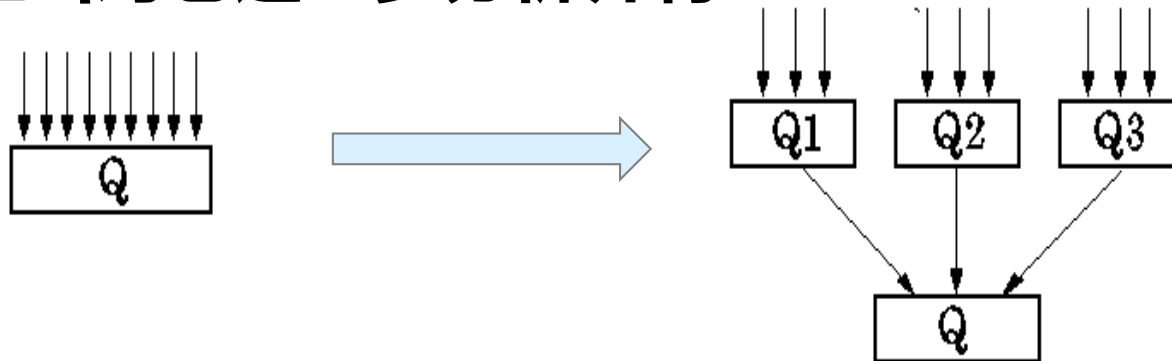


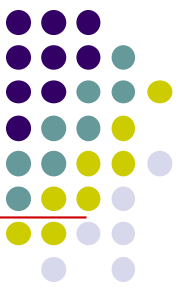


4.3.5 软件模块结构的改进方法

■ 尽可能减少高扇出结构，随着深度增大扇入。

- 一个模块的扇入数越大，则共享该模块的上级模块数目越多。扇入大，一般是有好处的。
- 但如果一个模块的**扇入数太大**，如超过8，而它又不是公用模块，说明该模块可能具有多个功能。在这种情况下应当对它进一步分析并将其**功能分解**





4.3.5 软件模块结构的改进方法

■ 模块规模应该适中

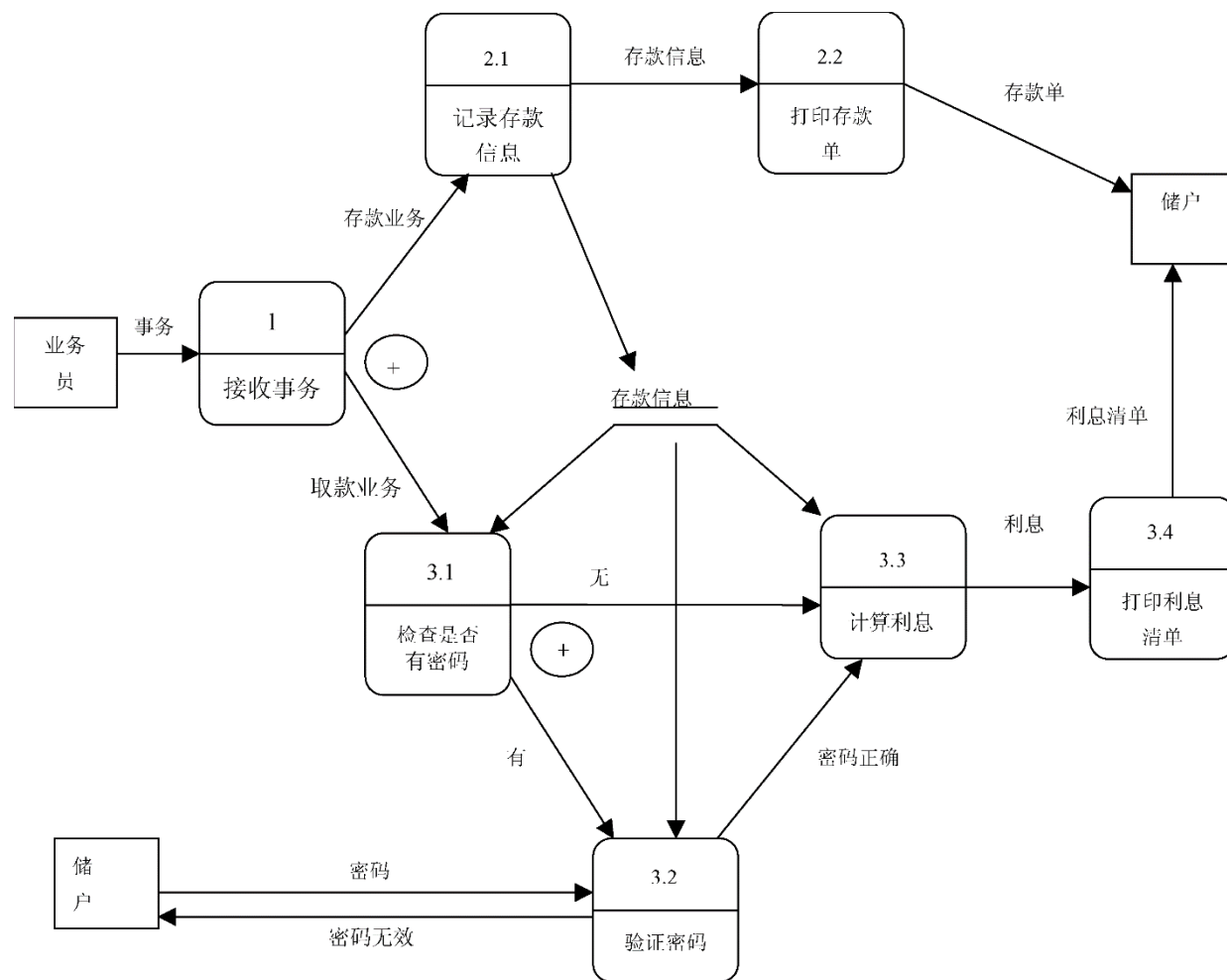
- 通常规定其语句行数在50 ~ 100左右，保持在一页纸之内，最多不超过500行。
- 过大：如果包含了多个功能则分解。
- 过小：合并到上级模块，但如果该模块是功能内聚模块或是共享模块或上级模块很复杂，则不宜合并。

4.3.5 软件模块结构的改进方法

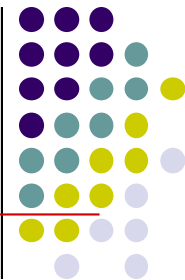
■ 实例研究

➤ 针对第3章例3.1的银行储蓄系统，开发软件的结构图。

第1步：对银行储蓄系统的数据流图进行复查并精化，得到如图所示的数据流图。



4.3.5 软件模块结构的改进方法

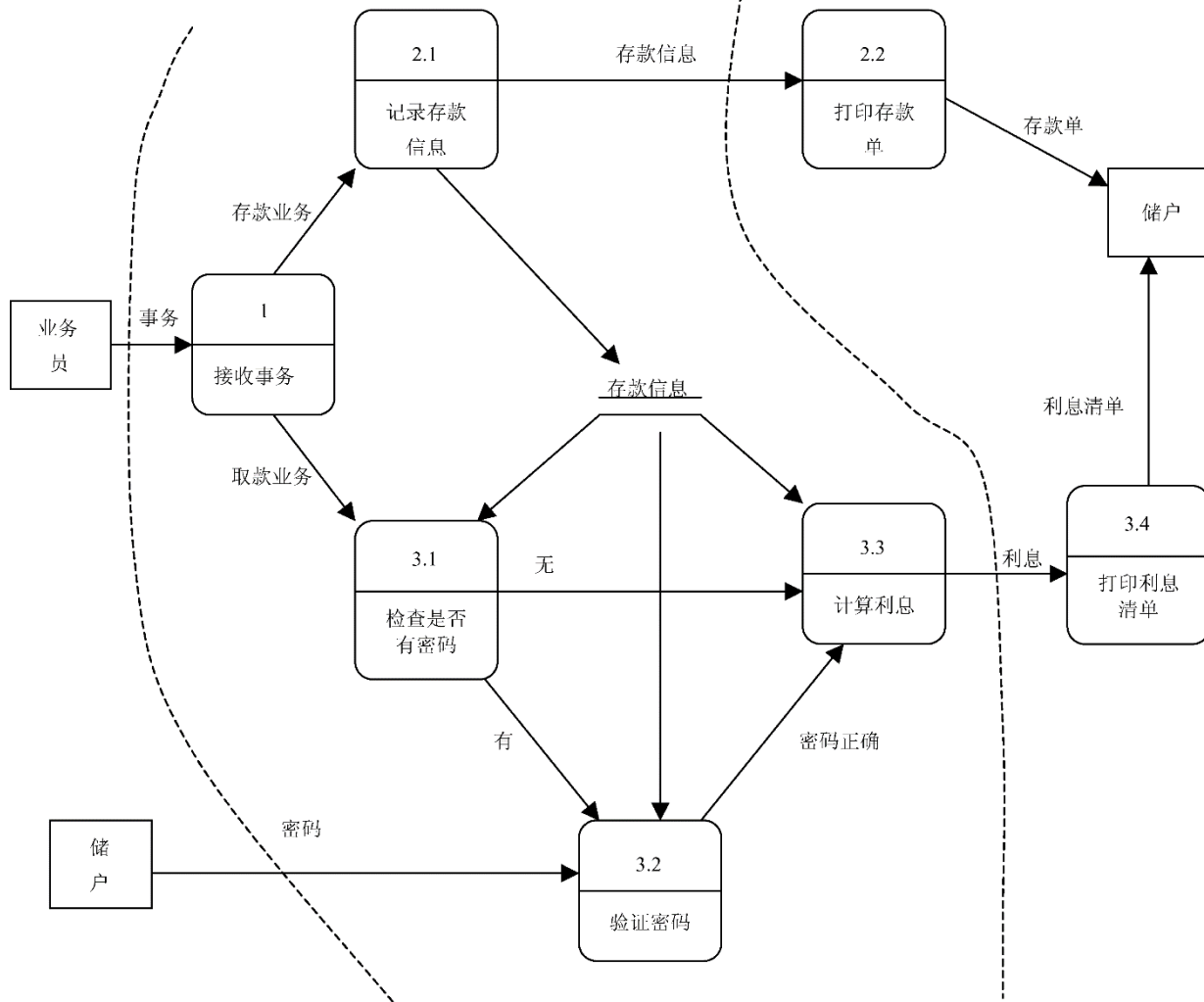


第2步：确定数据流图具有变换特性还是事务特性。

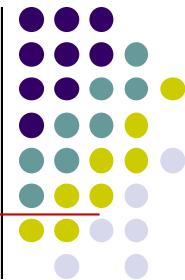
通过对精化后的数据流图进行分析，可以看到整个系统是对存款及取款两种不同的事务进行处理，因此具有事务特性。

4.3.5 软件模块结构的改进方法

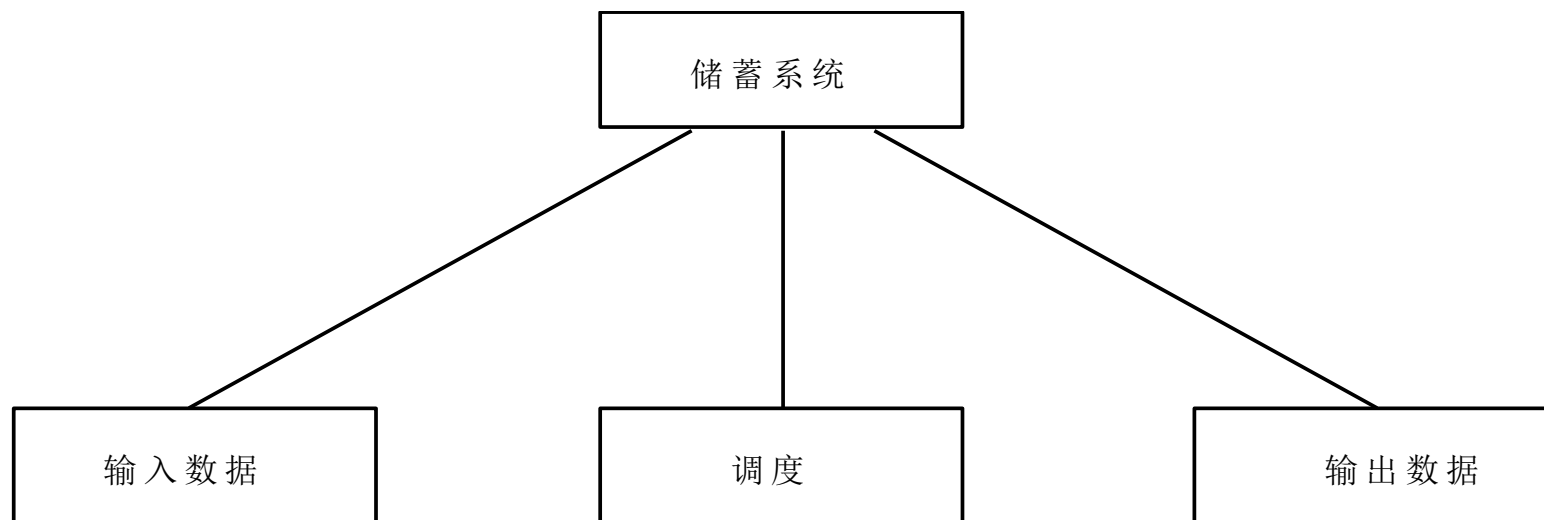
第3步：确定输入流和输出流的边界，如图所示。



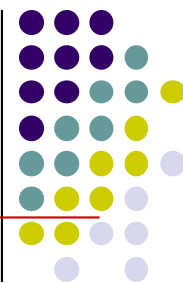
4.3.5 软件模块结构的改进方法



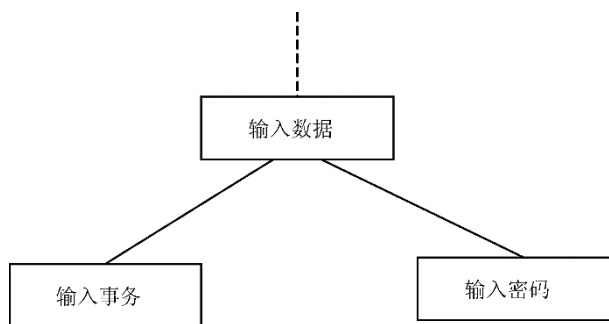
第4步：完成第一级分解。分解后的结构图如图所示。



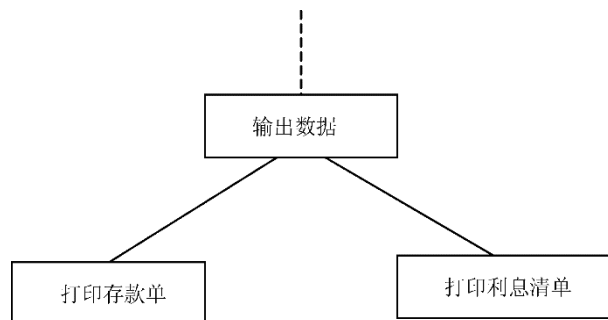
4.3.5 软件模块结构的改进方法



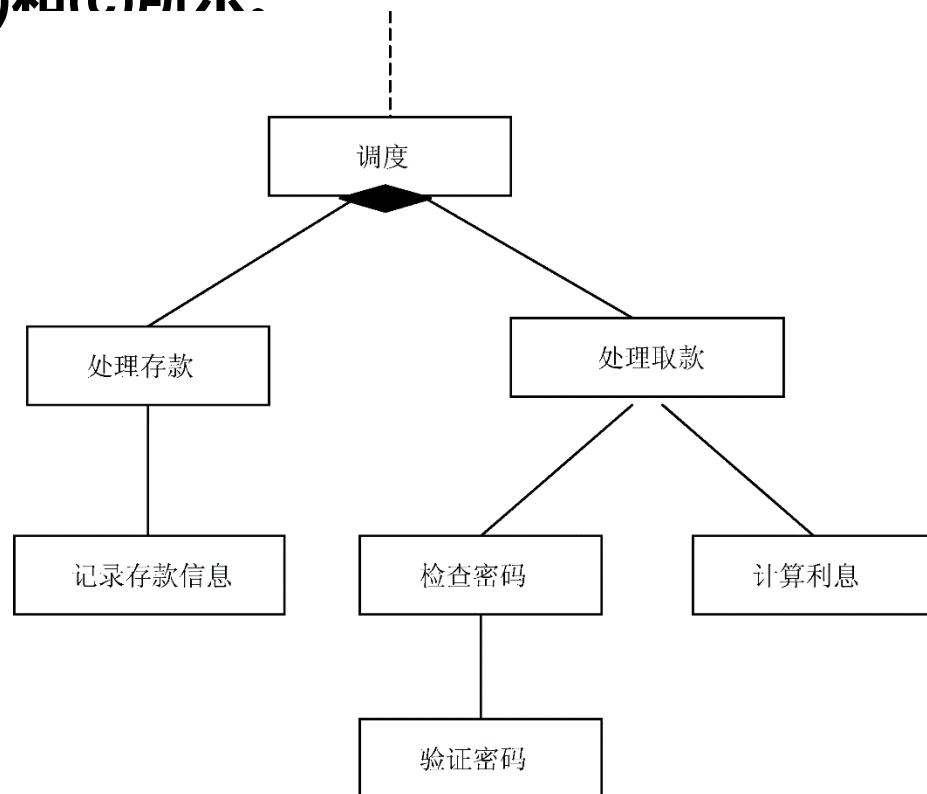
第5步：完成第二级分解。对上图中的“输入数据”、“输出数据”和“调度”模块进行分解，得到未经精化的输入结构、输出结构和事务结构，分别如图(a)、(b)和(c)所示。



(a) 未经精化的输入结构

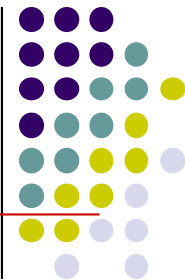


(b) 未经精化的输出结构

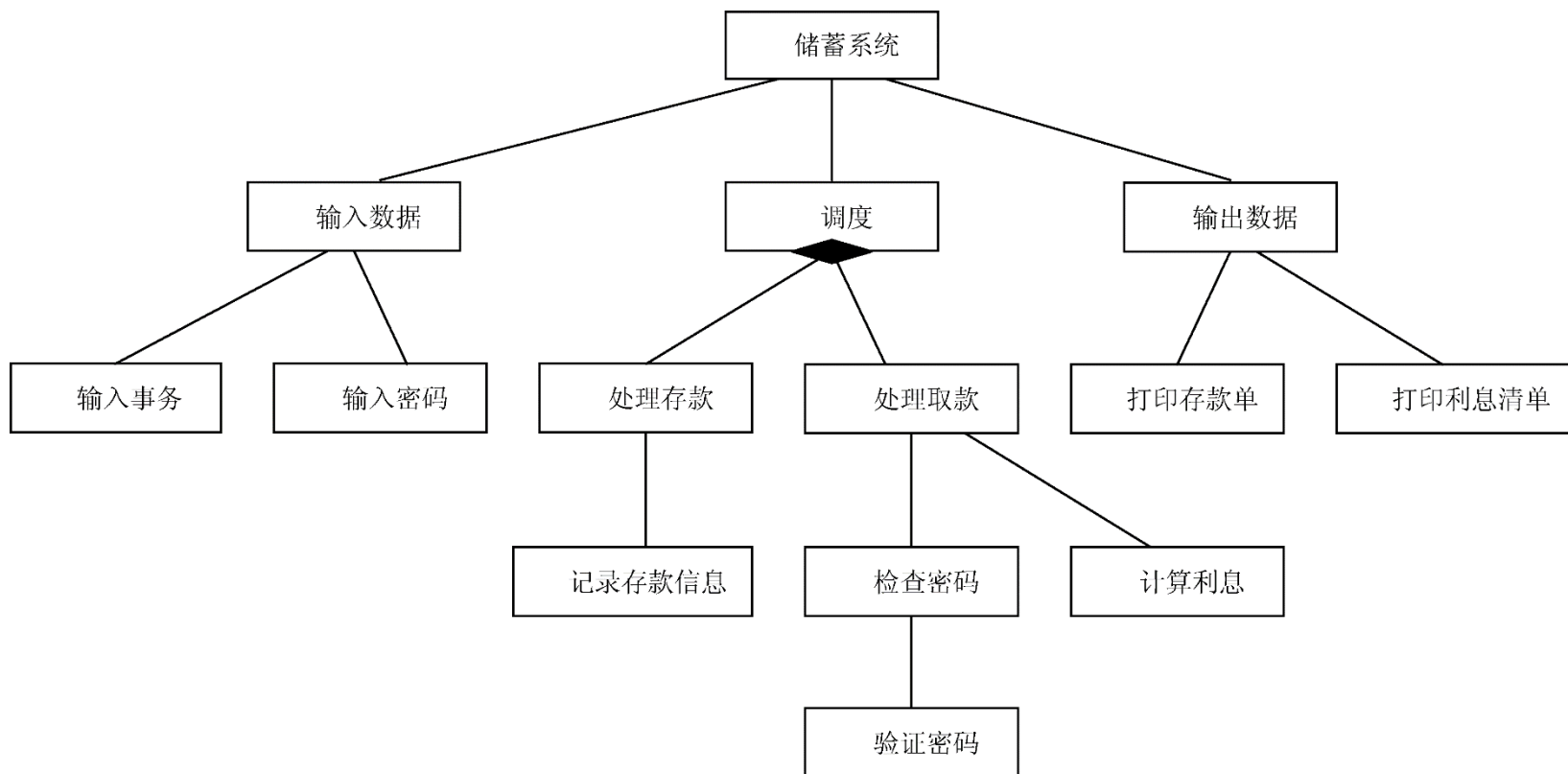


(c) 未经精化的事务结构

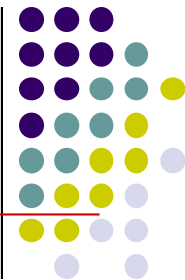
4.3.5 软件模块结构的改进方法



第5步：完成第二级分解。将上面的3部分合在一起，得到初始的软件结构，如图所示。

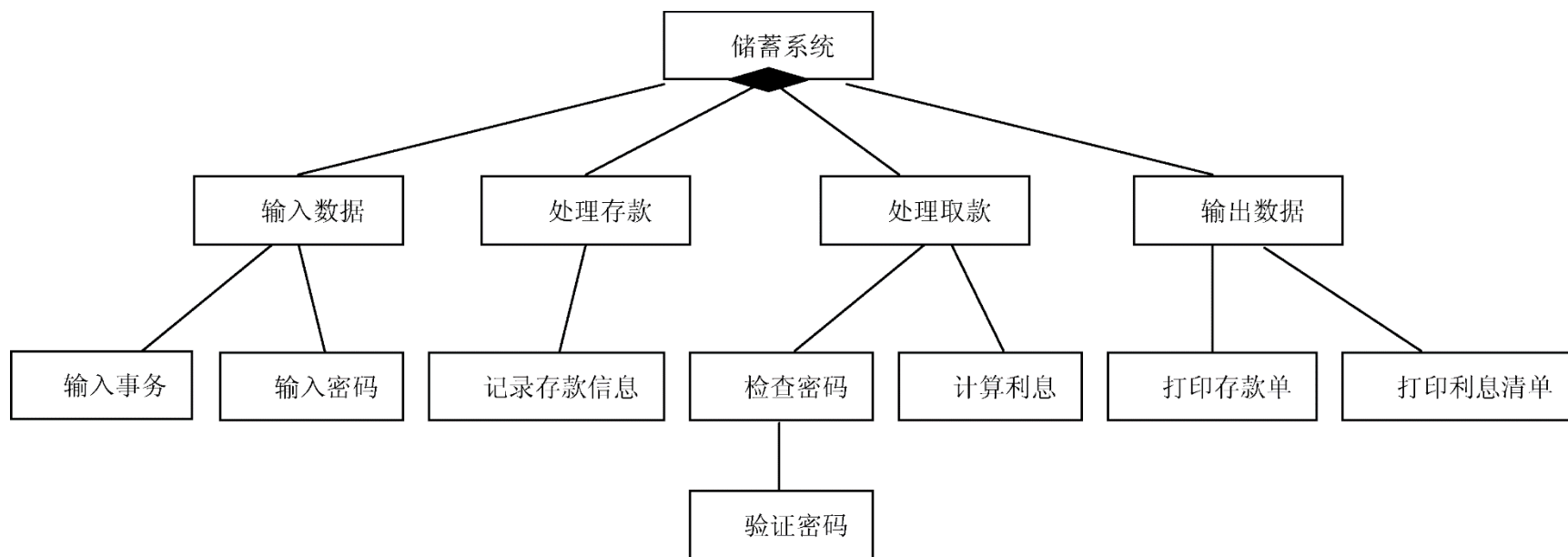


4.3.5 软件模块结构的改进方法

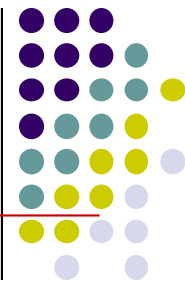


第6步：对软件结构进行精化。

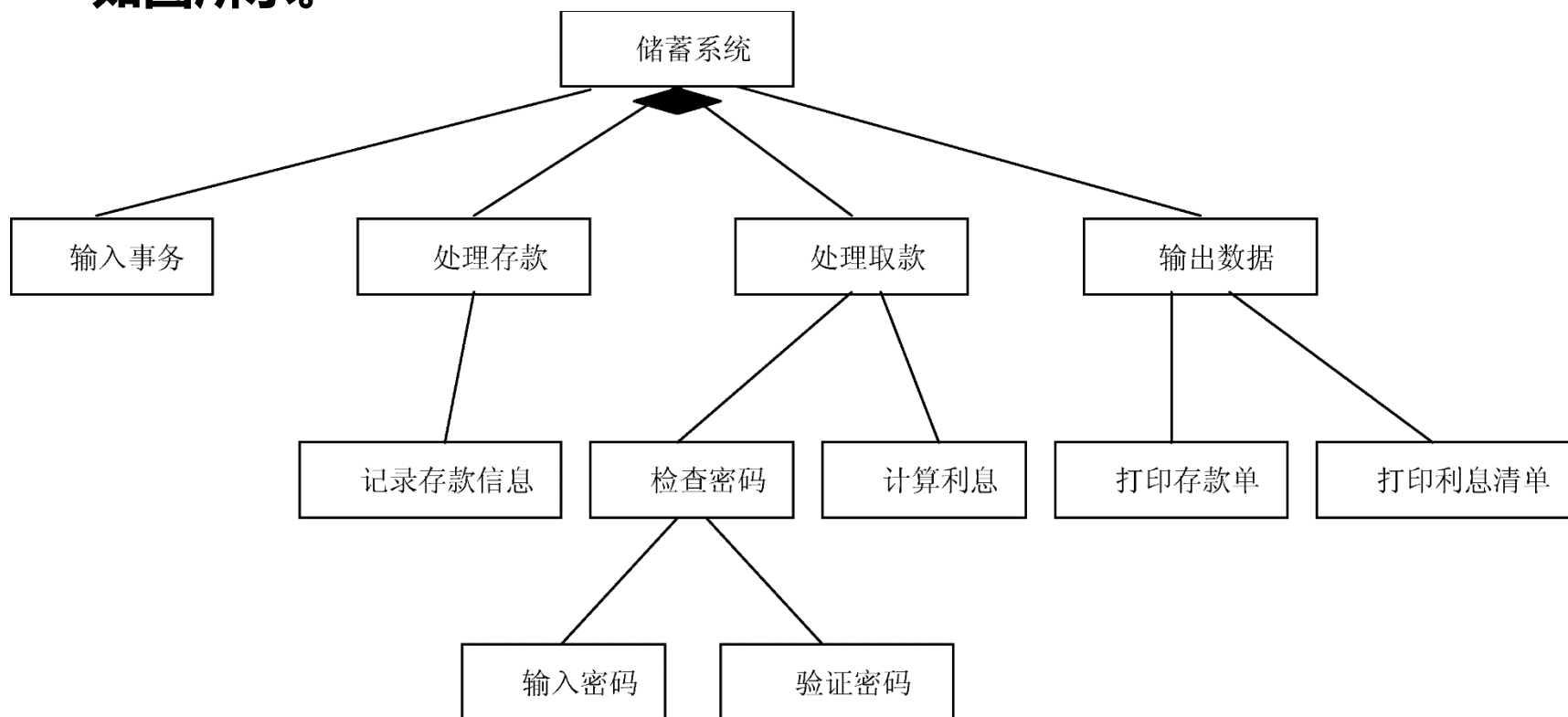
(1) 由于调度模块下只有两种事务，因此，可以将调度模块合并到上级模块中，如图所示。



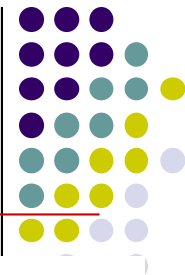
4.3.5 软件模块结构的改进方法



(2) “检查密码”模块的作用范围不在其控制范围之内（即“输入密码”模块不在“检查密码”模块的控制范围之内），需对其进行调整，如图所示。

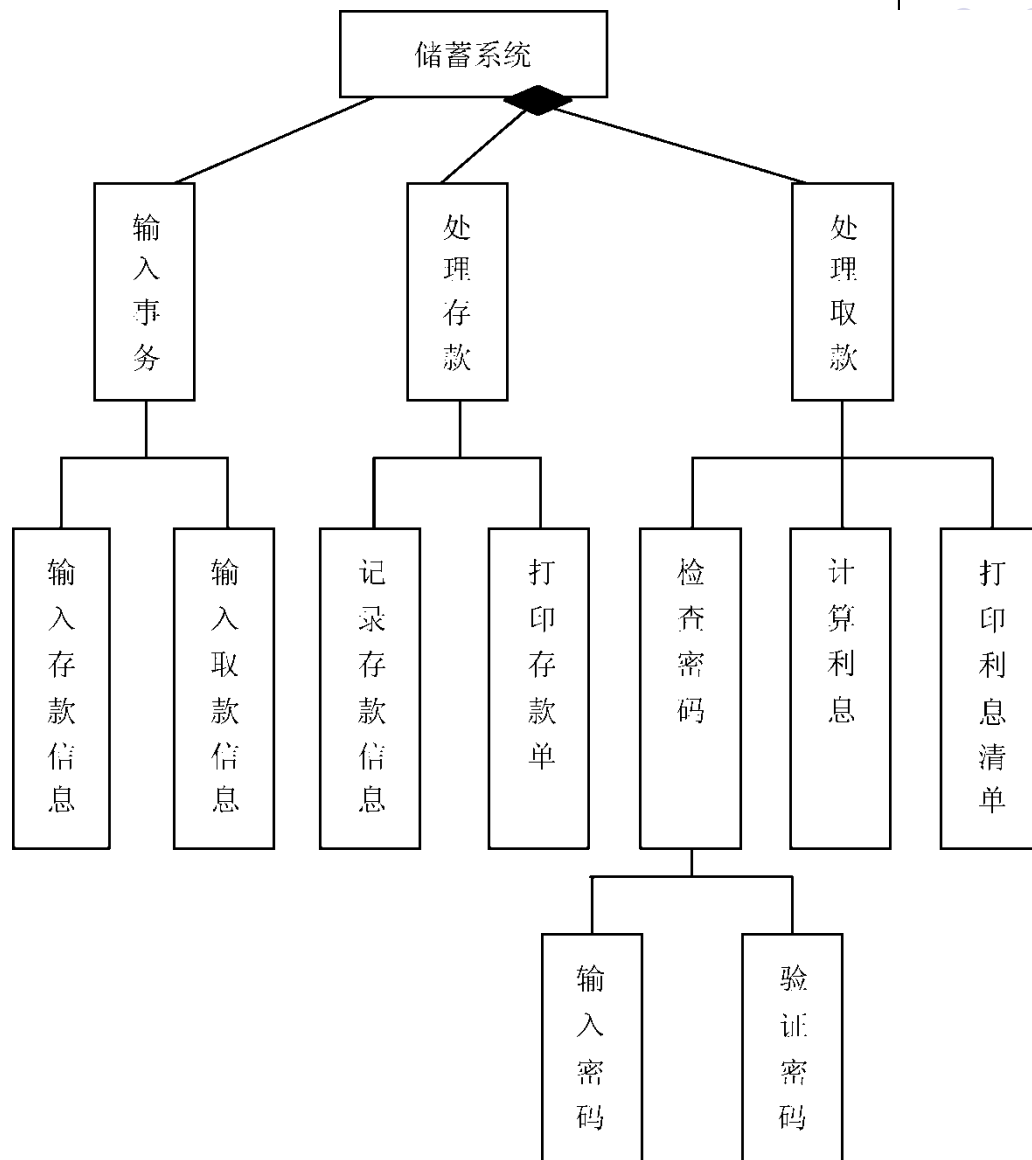


4.3.5 软件模块结构的改进方法

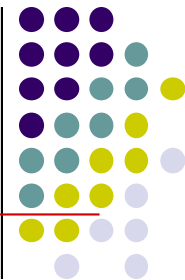


**(3) 提高模块的独立性，
并对“输入事务”模块进行细化。**

**也可以将“检查密码”
功能合并到其上级模块中。**



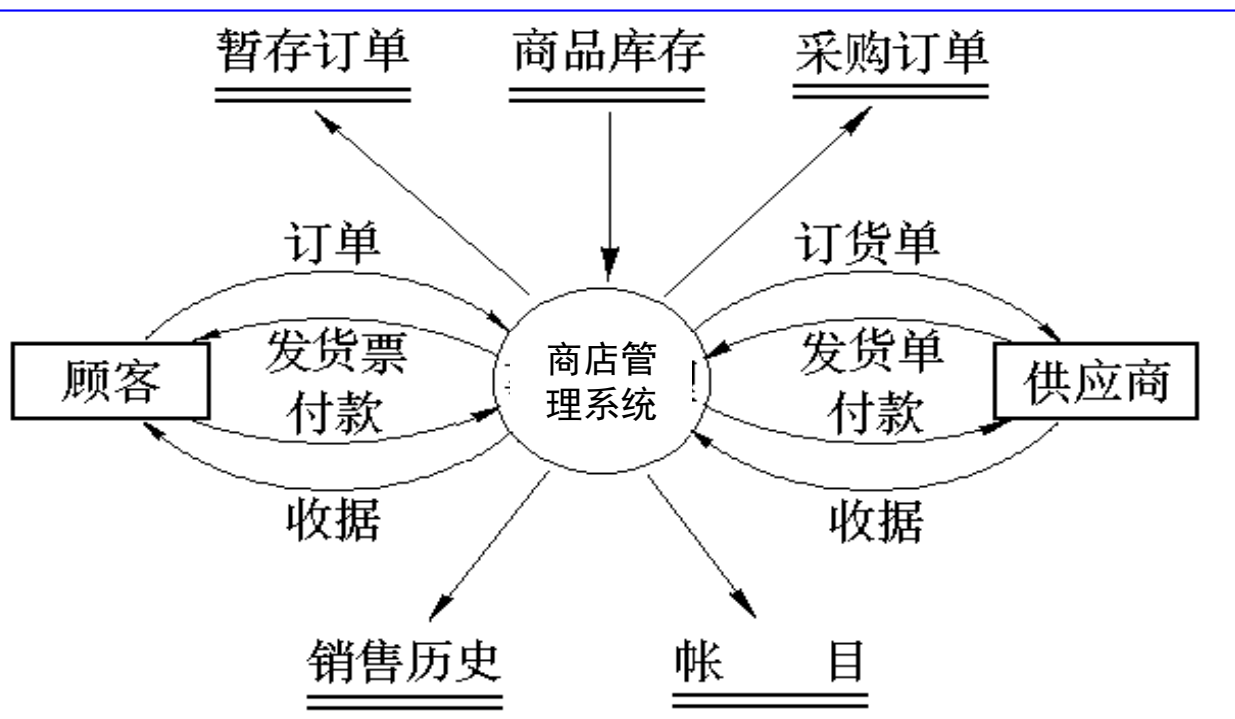
商店业务处理系统软件结构设计



顶层流图

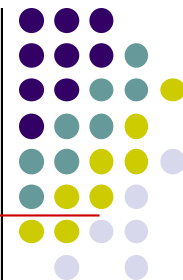


顶层模块



商店业务管理系统

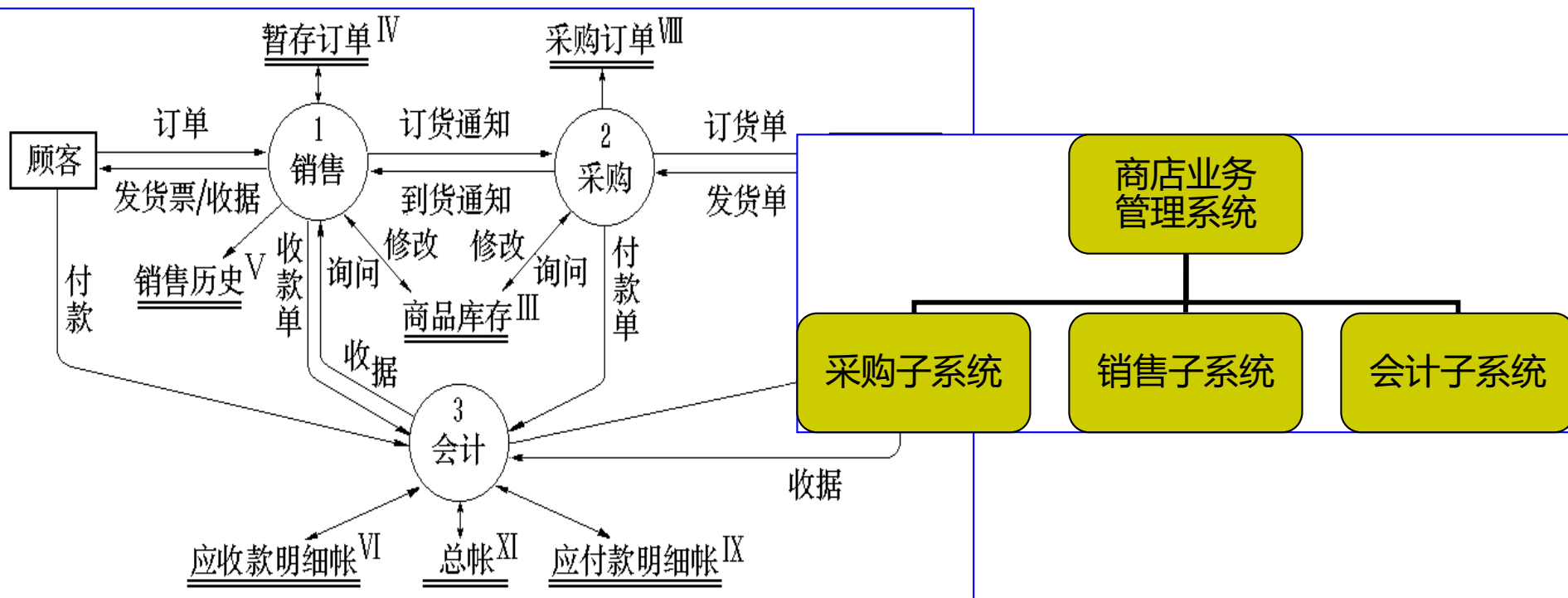
第一层数据流图



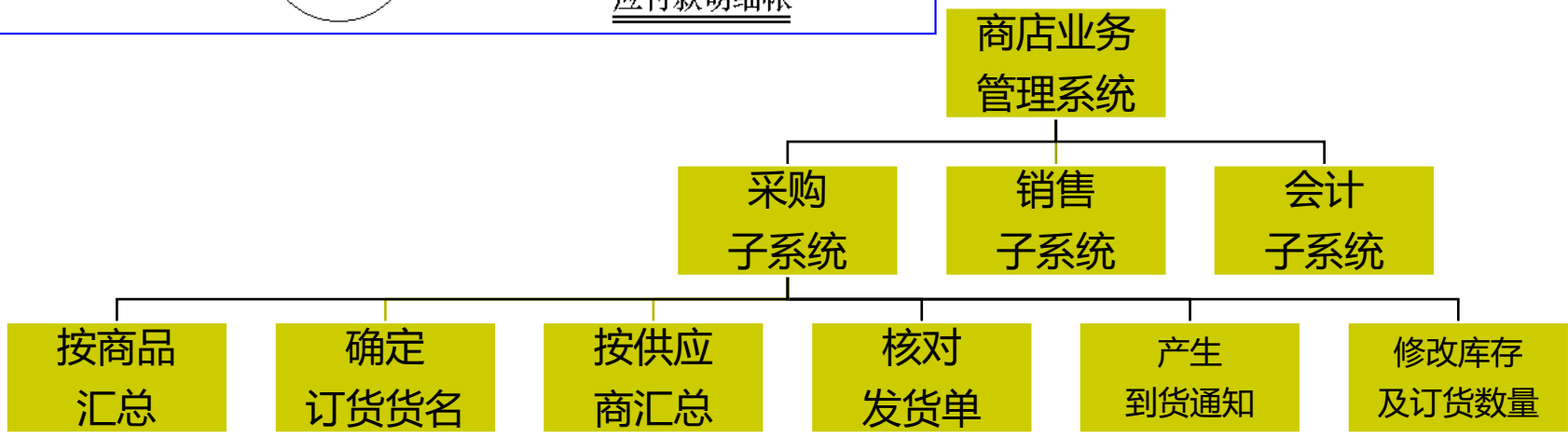
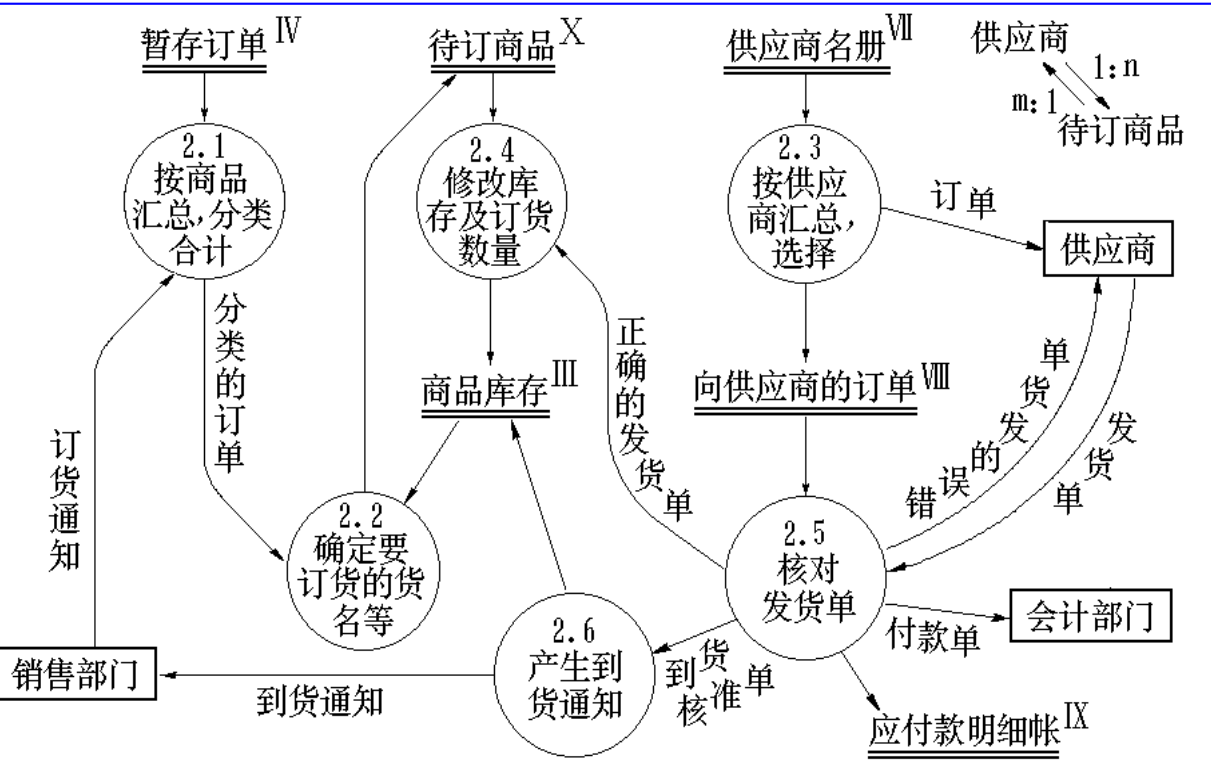
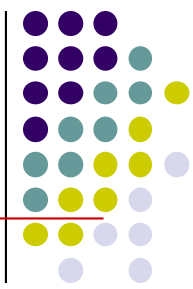
一层流图



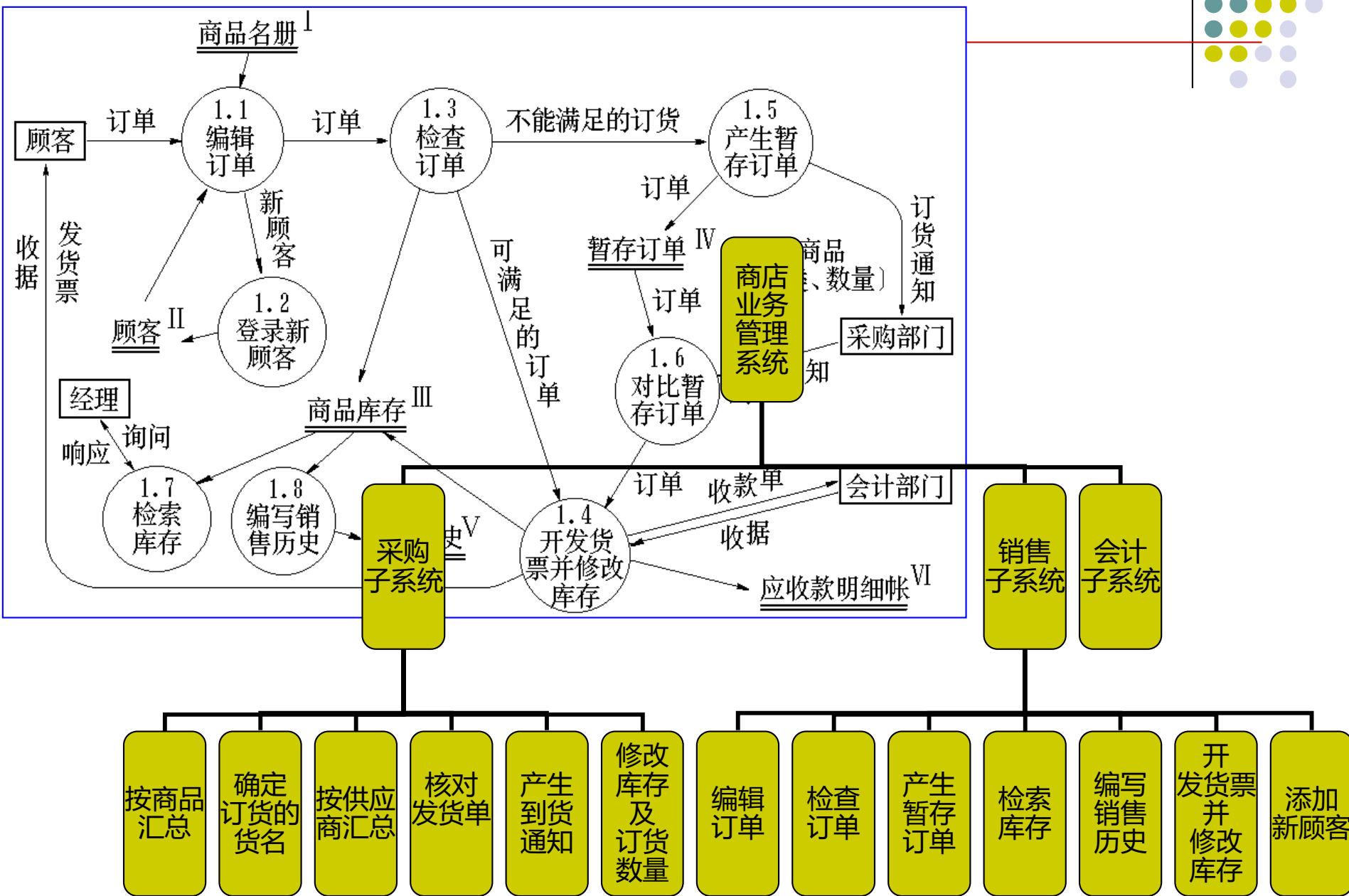
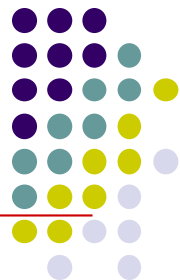
一层模块



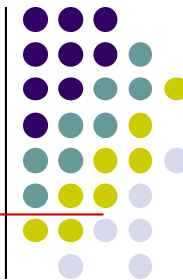
采购细化——二层模块设计



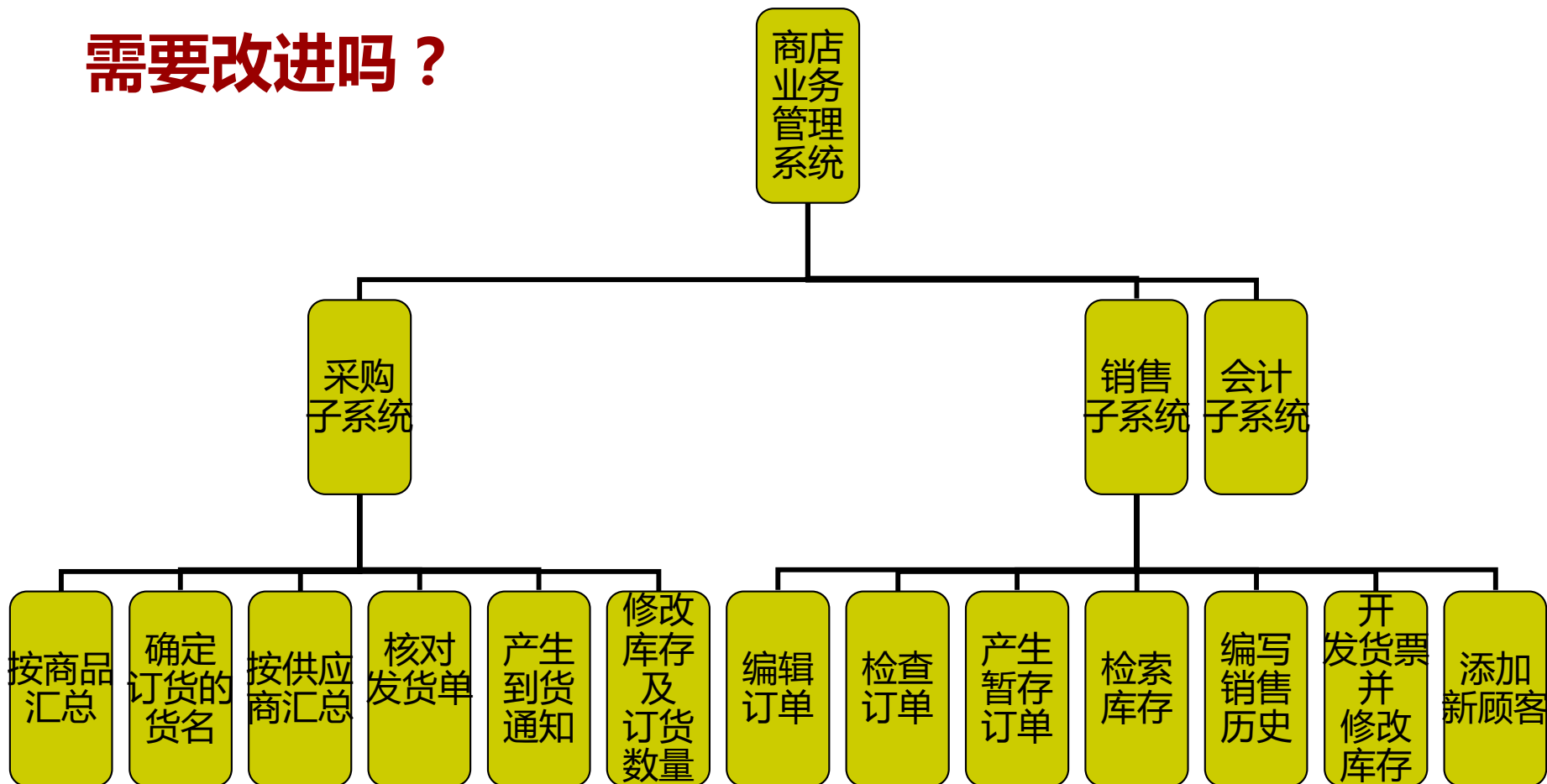
销售细化——二层模块设计



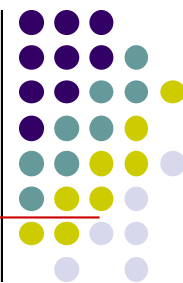
初始结构图



需要改进吗？

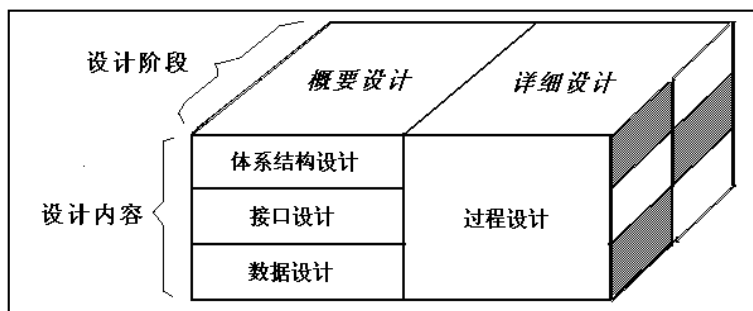


4.4 接口设计

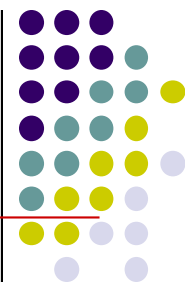


■ 接口设计概述

- 接口设计的依据是数据流图中的自动化系统边界。
- 接口设计主要包括3个方面：
 - 模块或软件构件间的接口设计；
 - 软件与其他软硬件系统之间的接口设计；
 - 软件与人（用户）之间的交互设计。
- **人机交互（用户）界面是人机交互的主要方式**



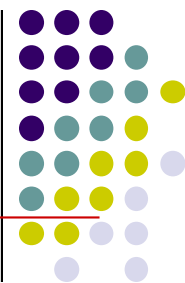
4.4 接口设计



■ 用户界面应具备的特性

- **可使用性**：使用简单、界面一致、拥有HELP帮助功能、快速的系统响应和低的系统成本、**具有容错能力**等。
- **灵活性**：考虑到用户的特点、能力和知识水平，应当使用户接口**满足不同用户的要求**。
- **可靠性**：用户界面的可靠性是指无故障使用的间隔时间。用户界面应能保证用户正确、可靠地使用系统，保证有关程序和数据的安全性。

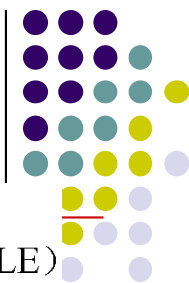
4.4 接口设计



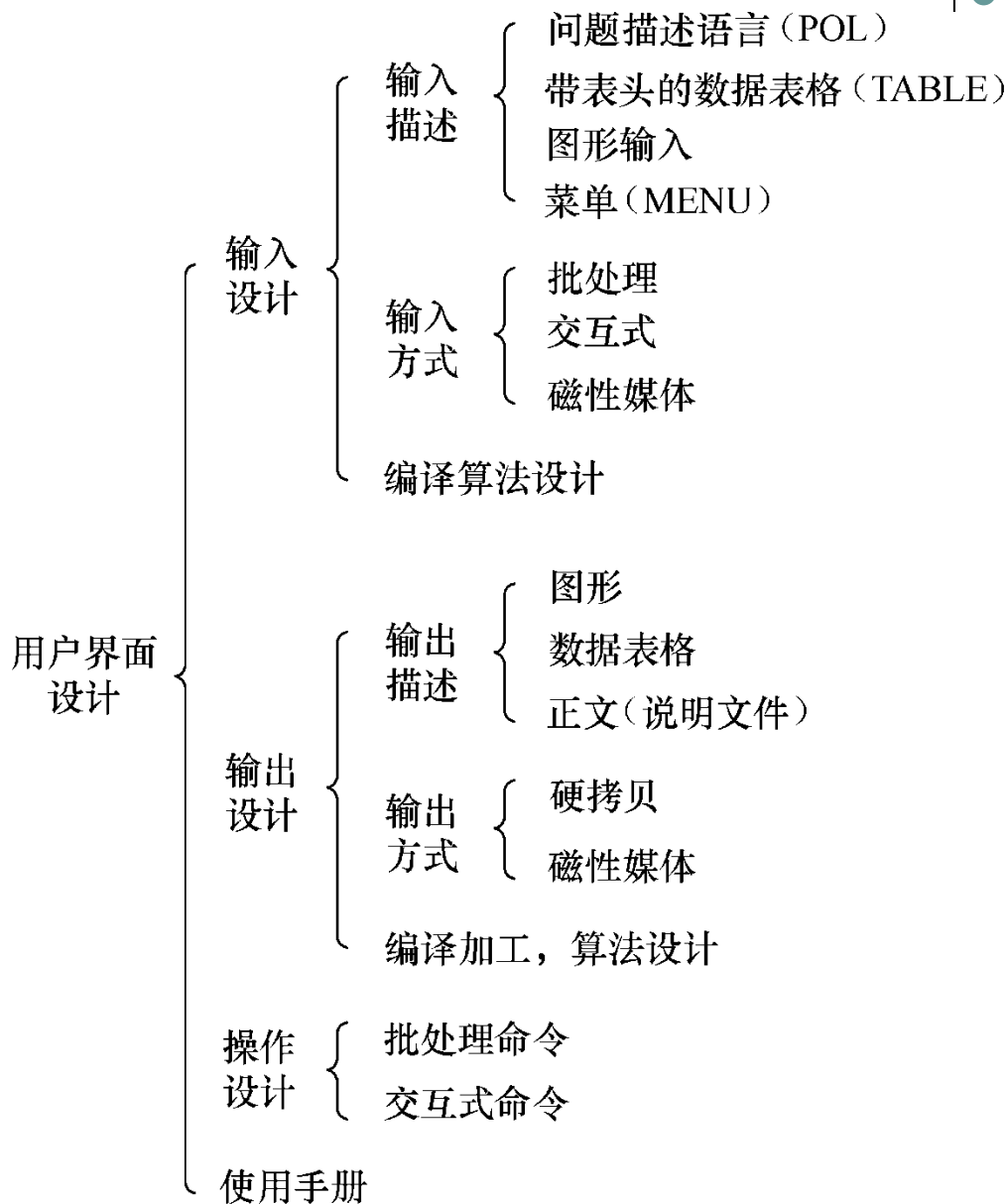
■ 用户类型

- **外行型**：以前从未使用过计算机系统的用户。
- **初学型**：尽管对新的系统不熟悉，但对计算机还有一些使用经验的用户。
- **熟练型**：对一个系统有相当多的经验，能够熟练操作的用户。
- **专家型**：这一类用户了解系统内部的构造，有关于系统工作机制的专业知识，具有维护和修改基本系统的能力。专家型需要为他们提供能够修改和扩充系统能力的复杂界面。

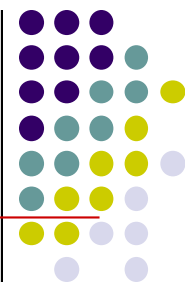
4.4 接口设计



■ 界面设计类型



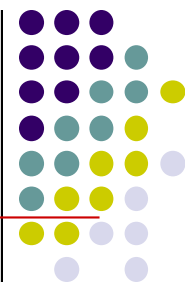
4.4 接口设计



■ 界面设计类型

- 在选用界面形式的时候，应当考虑每种类型的优点和限制，可以从以下几个方面来考察：
 - (1) **使用的难易程度**：对于没有经验的用户，该界面使用的难度有多大。
 - (2) **学习的难易程度**：学习该界面的命令和功能的难度有多大。
 - (3) **操作速度**：在完成一个指定操作时，该界面在操作步骤、击键和反应时间等方面效率有多高。

4.4 接口设计



■ 界面设计类型

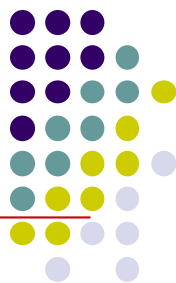
- (4) **复杂程度**：该界面提供了什么功能、 能否用新的方式组合这些功能以增强界面的功能。
- (5) **控制**：人机交互时，是由计算机还是由人发起和控制对话。
- (6) **开发的难易程度**：该界面设计是否有难度、 开发工作量有多大。

尽管有很多技术出色、聪明过人的软件开发人员，但是他们未必开发得出“易用”的和“美观”的软件。主要原因有：

国内绝大多数大学的**计算机学科教育存在缺陷**：没有开设人机工程学、美学、心理学这些必修课。由于学生们接受的教育几乎全是科学与技术，他们根本不知道怎样才能设计出易用、美观的用户界面，很多人甚至想都没有想过。当他们毕业后真正参与软件产品开发时，只好**凭着个人的经验与感觉设计软件的用户界面**，这样产生的界面往往得不到大众用户的认可。

开发人员在设计用户界面方面不仅存在先天的教育缺陷，更加糟糕的是还常常犯“**错位**”的毛病，即**他以为只要自己感觉用户界面漂亮、使用起来方便，那么用户也一定会满意**。俗话说“王婆卖瓜，自卖自夸”。当开发人员向用户展示软件时，常会得意地讲：“这个软件非常好用，我操作给你看，……是很好用吧！蛮漂亮的吧！”

用户界面设计工作



·结构设计 Structure Design

结构设计是界面设计的骨架。通过对用户研究和任务分析，制定出产品的整体架构。在结构设计中，目录体系的逻辑分类和语词定义是用户易于理解和操作的重要前提。

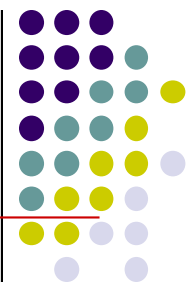
·交互设计 Interactive Design

交互设计的目的是使产品让用户能简单使用。任何产品功能的实现都是通过人和机器的交互来完成的。因此，人的因素应作为设计的核心被体现出来。

·视觉设计 Visual Design

在结构设计的基础上，参照目标群体的心理模型和任务达成进行视觉设计。包括色彩、字体、页面等。视觉设计要达到用户愉悦使用的目的。

出错信息处理



- 错误应在视觉或听觉上加以提示。
- 使用用户可以理解的术语描述问题。
即使使用用户的语言，而非技术的语言。
- 错误不能指责用户。
- 提供有助于从错误中恢复的建设性意见。
- 指出错误可能造成的后果。

A screenshot of an email login interface. At the top left is a blue circular icon with a white '@' symbol. To its right is the title '电子邮箱登录' (Email Login). Below the title is a red error message: '必须填写用户名和密码' (Must fill in username and password). There are two input fields: '用户名' (Username) and '密码' (Password). Below the password field is a checkbox with a green checkmark and the text '在此电脑上记住用户名' (Remember username on this computer). At the bottom is a blue button with the text '登录' (Login).

出错处理

发现错误

- 提供对**输入数据进行校验**的功能。
- 对于在某些情况下不应该使用的菜单项和命令按钮，将其“**失效**”（屏蔽）可以有效防止该项

减少错误

功能被错

- 提供Und

- 执行破坏

网上教学 - Microsoft Internet Explorer

地址: http://202.204.68.123:8080/new_wsjsx/exam/main.jsp

华北电力大学
网上教学系统
North China Electric Power University

您当前位置: 题库管理 当前用户: 徐燕

题库管理 > 修改多项选择题

课程	计算机文化基础	章节	计算机基础知识	知识点	知识点1	题型	多项选择题
难度	难	题号	30	日期	2004/07/14		

试题内容:

TCP/IP协议大Internet网络系统描述具有四层功能的网络模型。即：链路层、网络层及()。

A. 关系层
B. 应用层
C. 表示层
D. 传输层
E. .

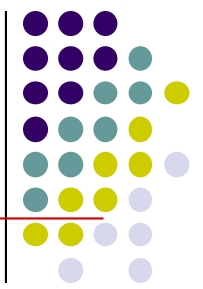
试题答案: ☐ A ☒ B ☐ C ☒ D ☐ E

动作:

华北电力大学计算机系 2003 copyright by ncepubj computer All rights reserved.

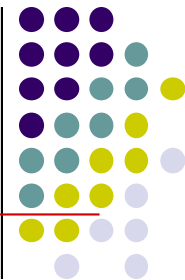
人。

保持一致性

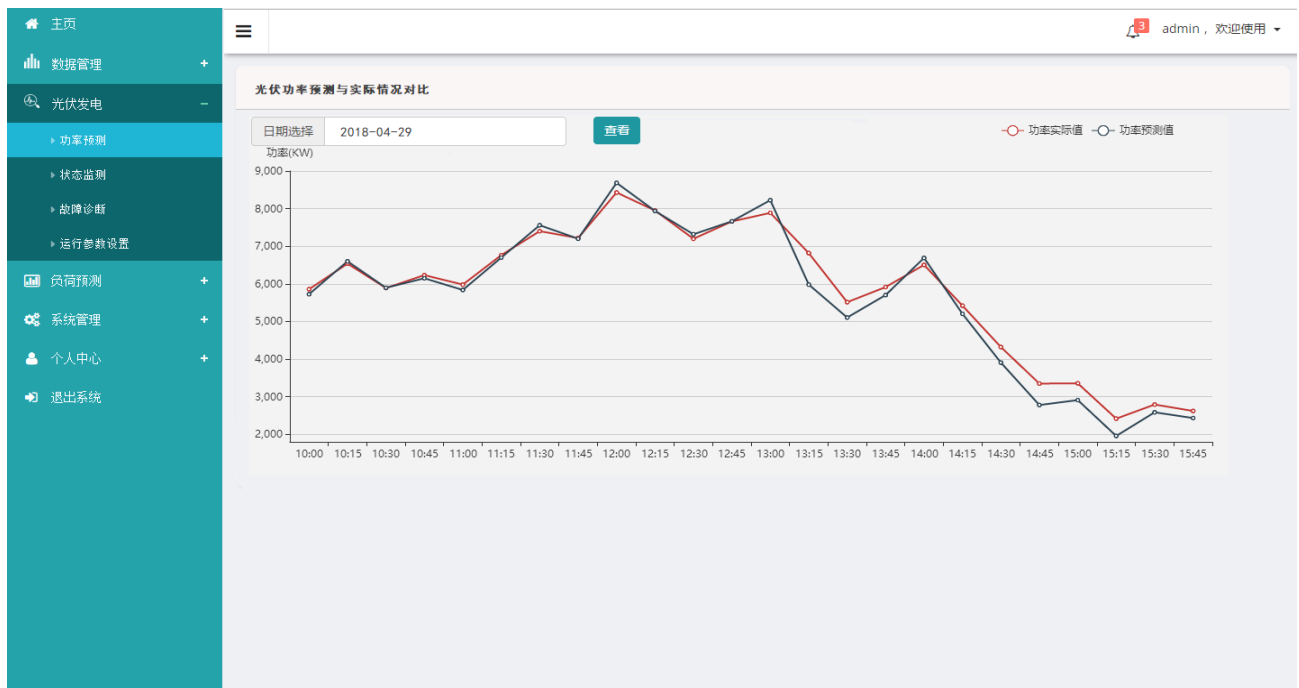


- 同一类型软件的用户界面应当有一定程度的相似性；
- 一个软件的用户界面中，同类的界面元素应当有相同的视感和相同的操作方式；
- **“风格一致”能够减少用户的记忆量、减少出错几率，并且迅速积累操作经验。**

信息显示



- ❖ 可以用多种不同方式“显示”信息。
用文字、图片和声音；
按位置、移动和大小；
使用颜色、分辨率和省略。
- ❖ 只显示与当前工作内容有关的信息。
- ❖ 不要用数据淹没用户，应该用便于用户迅速地吸取信息的方式来表示数据。

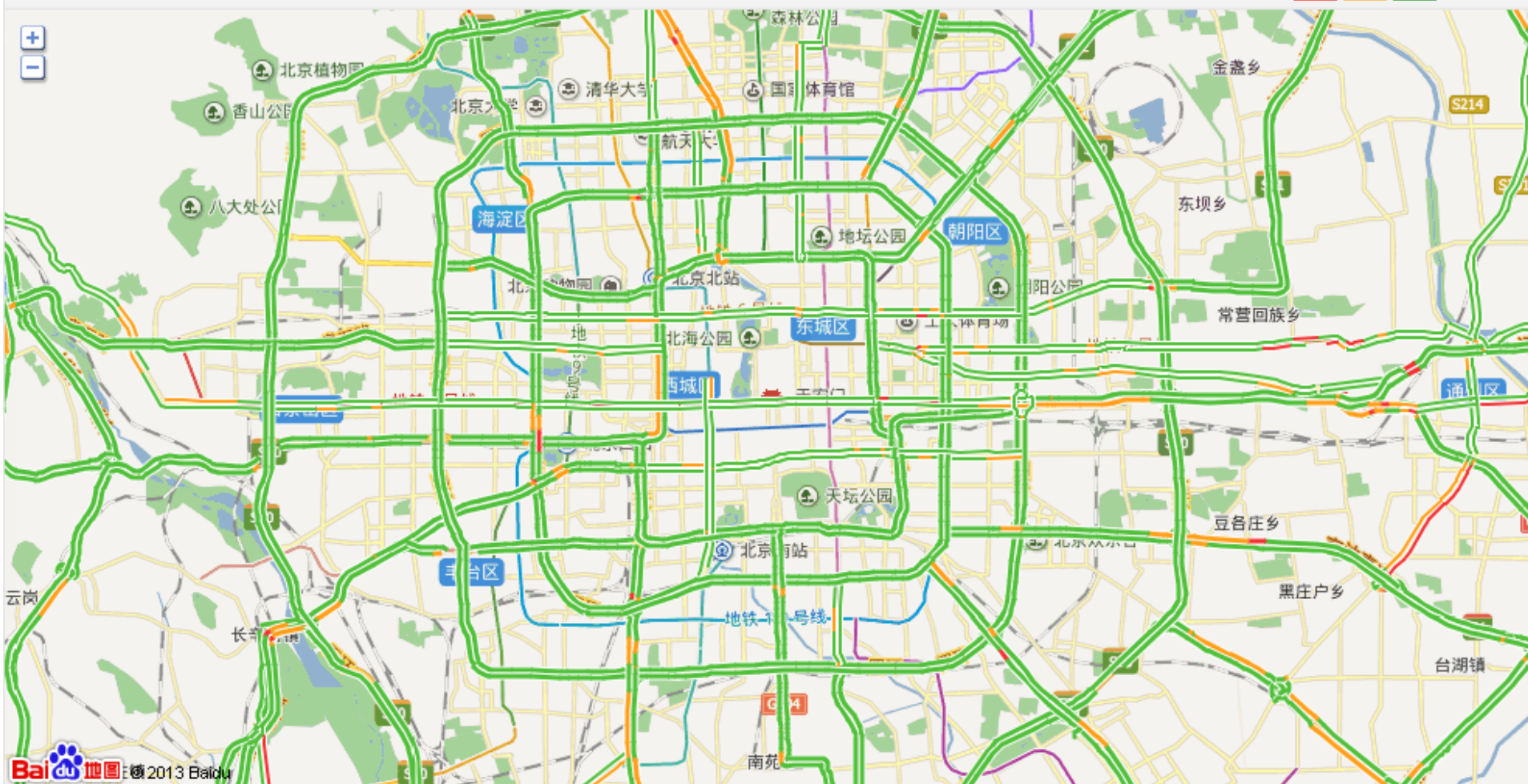


北京实时交通路况信息



北京实时路况信息 当前时间 21:12

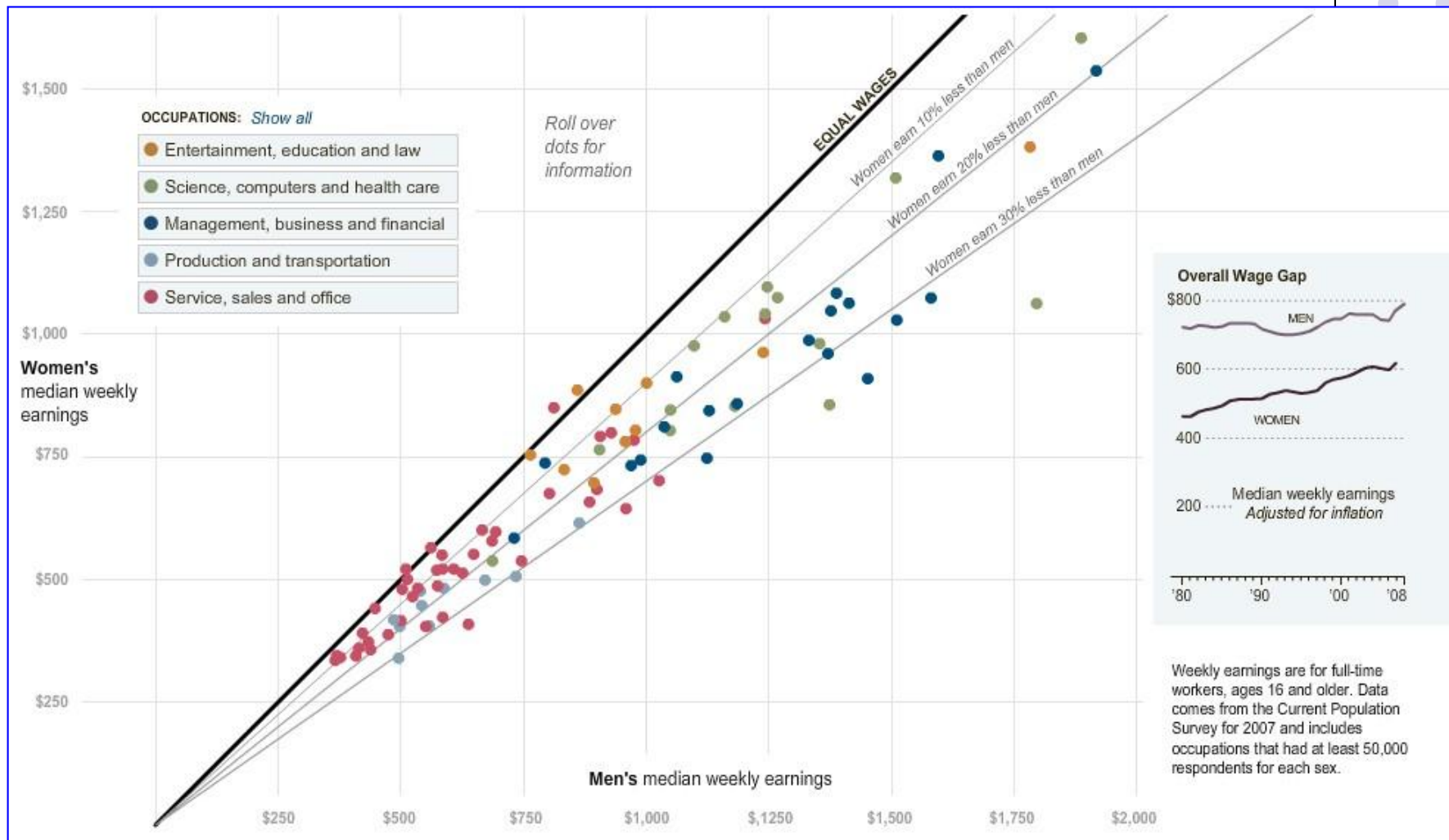
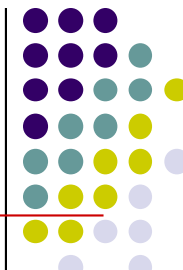
路况状态 **拥挤** 缓行 畅通 缩小地图



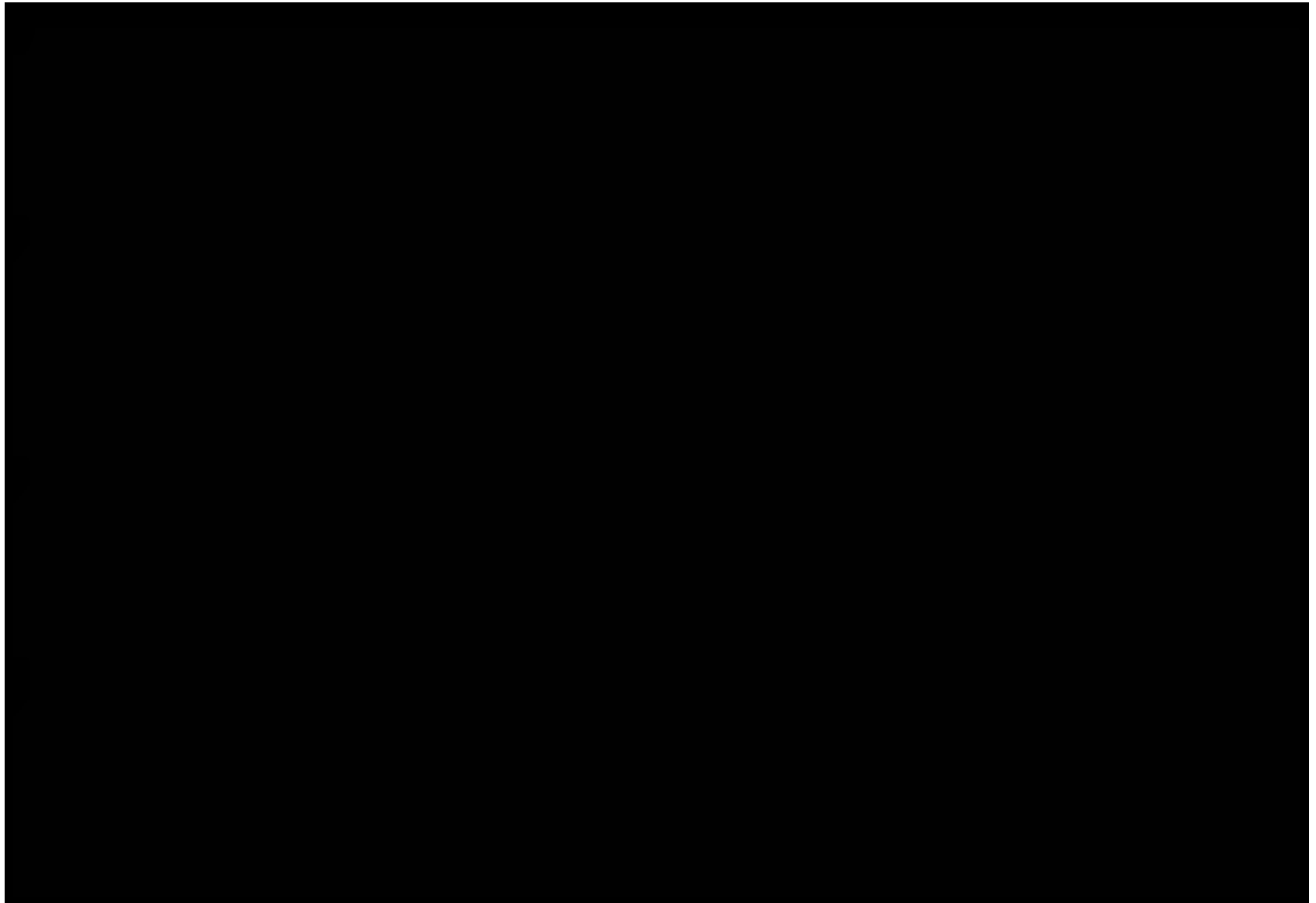
Baidu 地图 2013 Baidu

实时路况查询为您提供实时路况 北京实时路况 上海实时路况 广州实时路况 深圳实时路况 杭州实时路况 长春实时路况 沈阳实时路况 大连实时路况 天津实时路况 青岛实时路况 乌鲁木齐

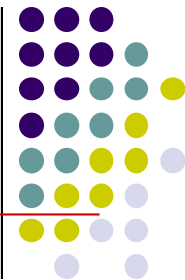
男女薪酬对比



感知海洋



数据输入



- **尽量减少用户的输入动作。**
- **保持信息显示和数据输入之间的一致性。**
- **允许用户自定义输入。**
- **交互应该是灵活的，并且可调整成用户最喜欢的输入方式。**

:: 填写个人资料 ::

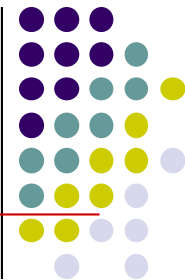
请填写个人资料：（注意带有*的项目必须填写）

* 密码		密码可使用长度为6-14的任何字符，并区分英文字母大小写
* 密码确认		请再输入一次密码
* 密码提示问题		例如：我的哥哥是谁？ 当您忘记密码时可以通过密码提示问题和答案找回密码
* 密码提示答案		注意： 密码提示问题答案长度不少于6位
* 出生日期	19 年 01 月 日	此项信息用以找回密码，请如实填写
安全码		安全码长度为6-16位任何字符，区分大小写。能为密码修复提供更好的安全性，建议设置
* 姓名		请输入真实的姓名
* 性别	☐ 男 ☐ 女	
* 有效证件号码		证件号码作为取回帐号的最后手段，一旦输入，不可更改
电子邮箱		请输入您常用的其它电子邮箱
* 联系电话		请输入区号和真实的电话，以便我们与您联系
* 所在省份	请选择	
* 所属行业	请选择	
* 教育水平	请选择	
您的职业(选填)	请选择	
您的收入水平(选填)	请选择	

个人声明

我愿意其他人可以搜索到我的如下资料：☒ 姓名，联系方式 ☐ 其他已登记的信息

4.5 数据设计

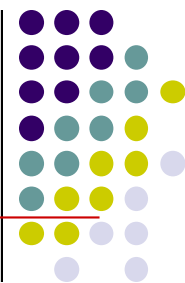


■ 文件设计

➤ 以下几种情况适合于选择文件存储。

- (1) 数据量较大的非结构化数据，如多媒体信息。
- (2) 数据量大，信息松散，如历史记录、档案文件等。
- (3) 非关系层次化数据，如系统配置文件。
- (4) 对数据的存取速度要求极高的情况。
- (5) 临时存放的数据。

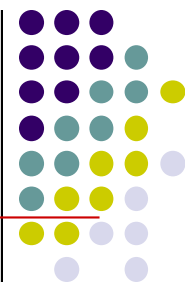
4.5 数据设计



■ 文件设计

- 一般要根据文件的特性，来确定文件的组织方式。
- (1) **顺序文件**：这类文件分两种，一种是连续文件，另一种是串联文件。
- (2) **直接存取文件**：可根据记录关键字的值，通过计算直接得到记录的存放地址。
- (3) **索引顺序文件**：其基本数据记录按顺序文件组织，记录排列顺序必须按关键字值升序或降序安排，且具有索引部分，索引部分也按同一关键字进行索引。

4.5 数据设计



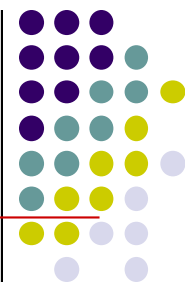
■ 文件设计

(4) **分区文件**：这类文件主要用于存放程序。它由若干称为成员的顺序组织的记录组和索引组成。

每一个成员就是一个程序，由于各个程序的长度不同，所以各个成员的大小也不同，需要利用索引给出各个成员的程序名、开始存放位置和长度。

(5) **虚拟存储文件**：这是基于操作系统的请求页式存储管理功能而建立的索引顺序文件。

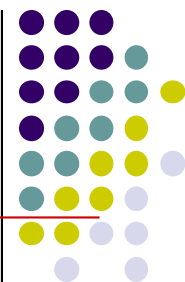
4.5 数据设计



■ 数据库设计

- 根据数据库的组织，可以将数据库分为网状数据库、层次数据库、关系数据库、面向对象数据库、文档数据库、多维数据库等。
- 关系数据库最成熟，应用也最广泛，一般情况下，大多数设计者都会选择关系数据库。
- 在结构化设计方法中，很容易将结构化分析阶段建立的实体—关系模型映射到关系数据库中。

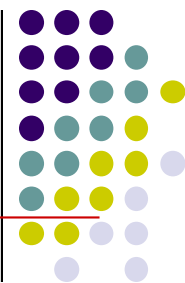
4.5 数据设计



■ 数据对象实体的映射

- 一个数据对象（实体）可以映射为一个表或多个表，当分解为多个表时，可以采用**横切**和**竖切**的方法。
- **竖切**常用于实例较少而属性很多的对象。通常将经常使用的属性放在主表中，而将其他一些次要的属性放到其他表中。
- **横切**常常用于记录与时间相关的对象。往往在主表中只记录最近的对象，而将以前的记录转到对应的历史表中。

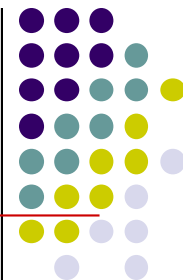
4.5 数据设计



■ 关系的映射

- **一对一关系的映射**：可以在两个表中都引入外键，进行双向导航。也可以将两个数据对象组合成一张单独的表。
- **一对多关系的映射**：可以将关联中的“一”端毫无变化地映射到一张表，将关联中表示“多”的端上的数据对象映射到带有外键的另一张表，使外键满足关系引用的完整性。
- **多对多关系的映射**：为了表示多对多关系，关系模型必须引入一个关联表，将两个数据实体之间的多对多关系转换成两个一对多关系。

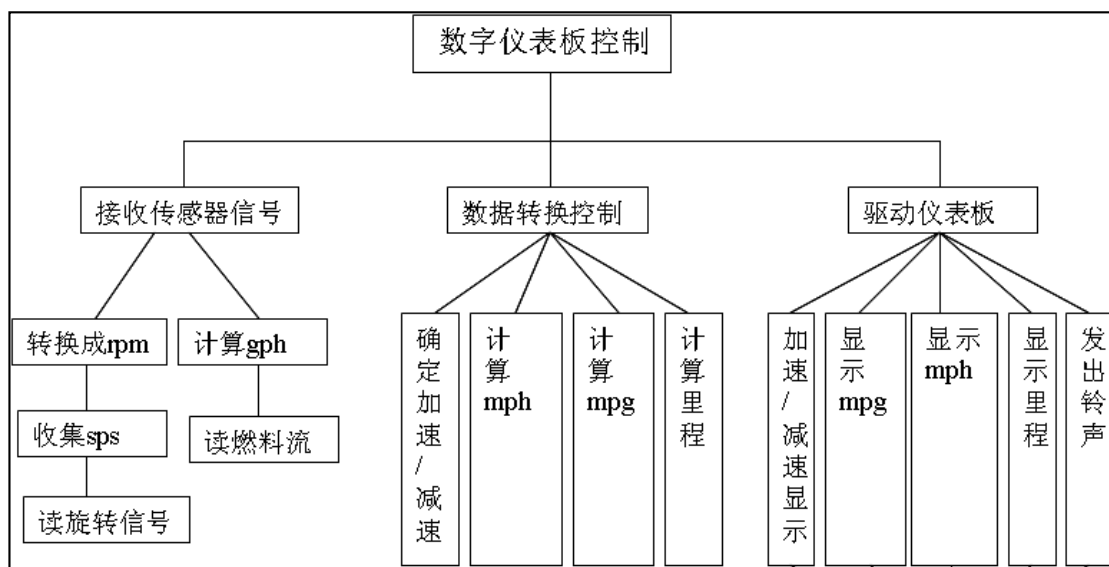
4.6 过程设计



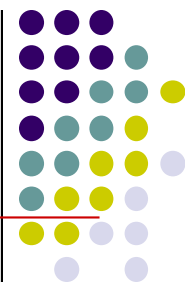
● 过程设计的任务

“怎样具体地实现这个系统”

- 确定每个模块的实现算法
- 选定某种过程的表达形式来描述算法

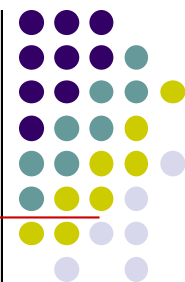


4.6 过程设计



- 过程描述工具：表达过程规格说明的工具
- 过程描述工具分为3类
 - 图形工具：把过程的细节用图形方式描述出来，如程序流程图、N-S图、PAD图、决策树等。
 - 表格工具：用一张表来表达过程的细节。这张表列出了各种可能的操作及其相应的条件，即描述了输入、处理和输出信息，如决策表。
 - 语言工具：用某种类高级语言（称为伪代码）来描述过程的细节，如很多数据结构教材中使用类Pascal、类C语言来描述算法。

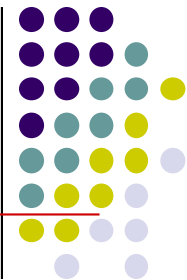
4.6.1 结构化程序设计



■ 结构化程序设计的概念与原则

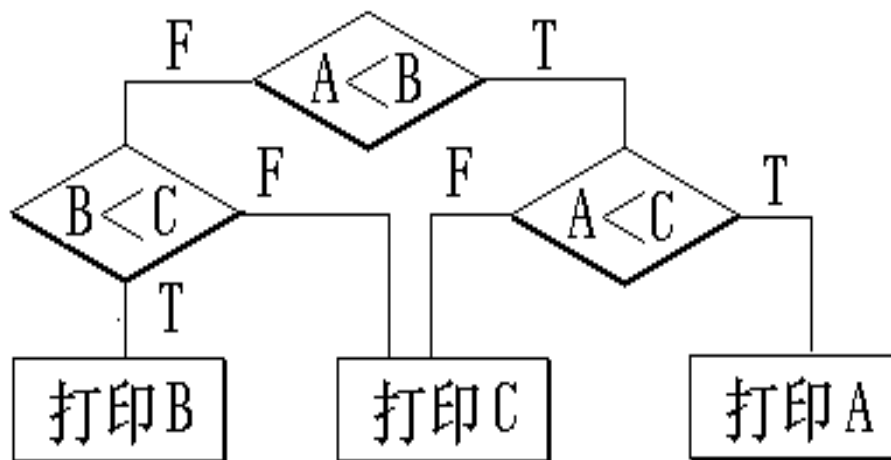
- 由于软件开发和维护中存在的一系列严重问题，于20世纪60年代爆发了软件危机。
- 为了克服软件危机，引发了程序设计的一场革命，诞生了结构化程序设计方法。
 - 最早由E. W. Dijkstra提出：建议从高级语言中取消GOTO语句。

4.6.1 结构化程序设计



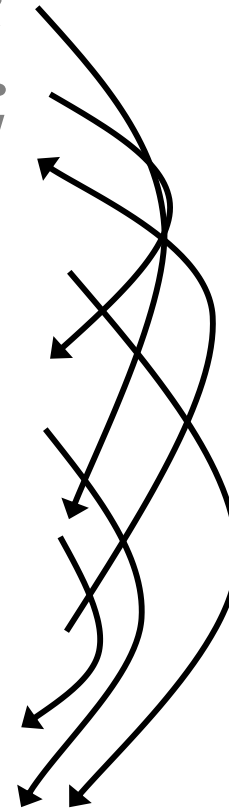
➤ 关于GOTO 语句的争论

例：打印A, B, C三数中最小者程序



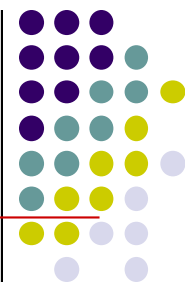
程序1

```
        if ( A < B )    goto l20 ;  
        if ( B < C )    goto l10 ;  
l00  write ( C ) ;  
        goto l40 ;  
l10  write ( B ) ;  
        goto l40 ;  
l20  if ( A < C )    goto l30 ;  
        goto l00 ;  
l30  write ( A ) ;  
l40  end
```



goto 语句出现了6次，程序可读性很差。

4.6.1 结构化程序设计



■ HackerRank平台调查

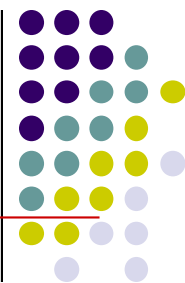
- 2018 年 11 月 5 日至 11 月 27 日，HackerRank 在社区发起了程序员技能调查。
- 哪些事情是让程序员觉得最恼火呢？
 - 在初级程序员中，74% 的开发者认为是糟糕的文档，54% 的开发者认为是面条式代码。
 - 在中高级程序员中，情况有所变化，面条式代码和未合理规划优先级几乎是并列排在首位（两者均为 63%）。55%的开发者认为是糟糕的文档。

程序2

```
if ( A < B && A < C )  
    printf(“%f”,A);  
else  
    if ( B < C )  
        printf(“%f”,B);  
    else  
        printf(“%f”,C);
```

结构清晰，程序可读性好。

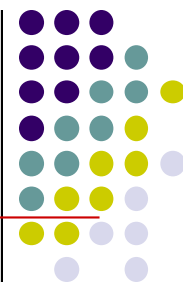
4.6.1 结构化程序设计



■ 结构程序设计的概念

- 1966年，Bohm和Jacopini**证明**：只用三种基本的控制结构“顺序”、“选择”和“循环”就能实现任何单入口和单出口的没有“死循环”的程序。
- 如果一个程序的代码块仅仅通过顺序、选择和循环这三种基本控制结构进行连接，并且每个代码块只有一个入口和一个出口，则称这个程序是结构化的。

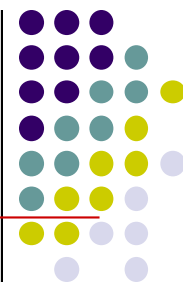
4.6.1 结构化程序设计



■ 结构程序设计的主要原则

- (1)使用语言中的顺序、选择、重复等有限的基本控制结构表示程序逻辑。**
- (2)选用的控制结构只准许有一个入口和一个出口。**
- (3)程序语句组成容易识别的块（ Block ） ， 每块只有一个入口和一个出口。**
- (4)复杂结构应该用基本控制结构进行组合嵌套来实现。**
- (5)语言中没有的控制结构，可用一段等价的程序段模拟，但要求该程序段在整个系统中应前后一致。**

4.6.1 结构化程序设计

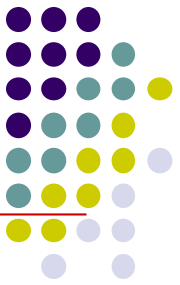


■ 结构程序设计的主要原则

(6) 严格控制GOTO语句，仅在下列情形才可使用：

- 用非结构化的程序设计语言去实现结构化的构造。
- 若不使用GOTO语句就会使程序功能模糊。
- 在某种可以改善而不是损害程序可读性的情况下。例如，在查找结束时，文件访问结束时，出现错误情况要从循环中转出时，使用布尔变量和条件结构来实现就不如用GOTO语句来得简洁易懂。

(7) 在程序设计过程中，尽量采用自顶向下(Top - Down)、逐步细化(Stepwise Refinement)的原则，由粗到细，一步步展开。



4.6.2 程序流程图

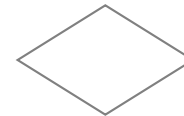
- 程序流程图也称为程序框图
- 程序流程图使用的基本符号



起止端点



处理



判断

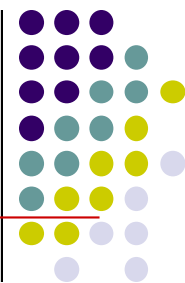


数据输入输出



控制流线

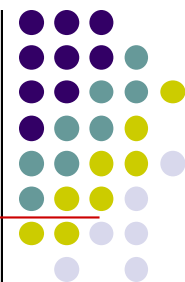
4.6.2 程序流程图



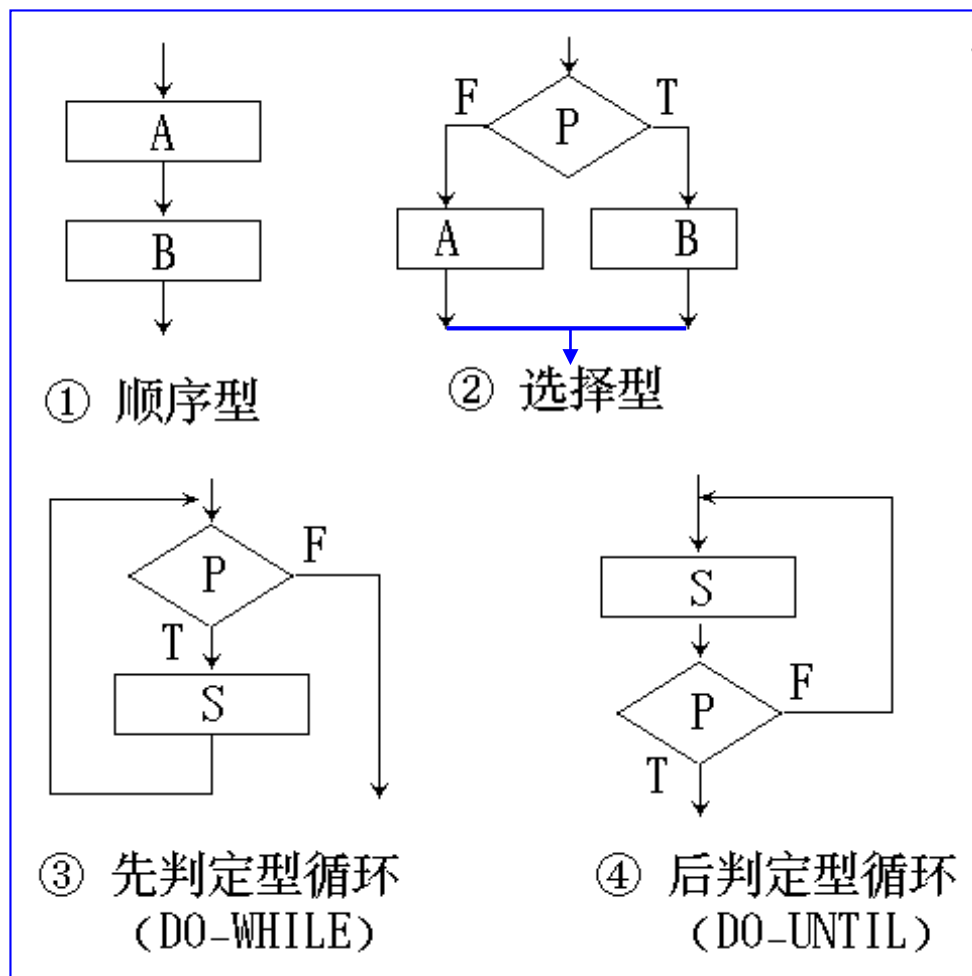
- 程序流程图的基本控制结构

- (1) **顺序型**：几个连续的加工步骤依次排列构成。
- (2) **选择型**：由某个逻辑判断式的取值决定选择两个加工中的一个。
- (3) **先判定 (while) 型循环**：在循环控制条件成立时，重复执行特定的加工。
- (4) **后判定 (until) 型循环**：重复执行某些特定的加工，直至控制条件成立。
- (5) **多情况 (case) 型选择**：列举多种加工情况，根据控制变量的取值，选择执行其一。

4.6.2 程序流程图

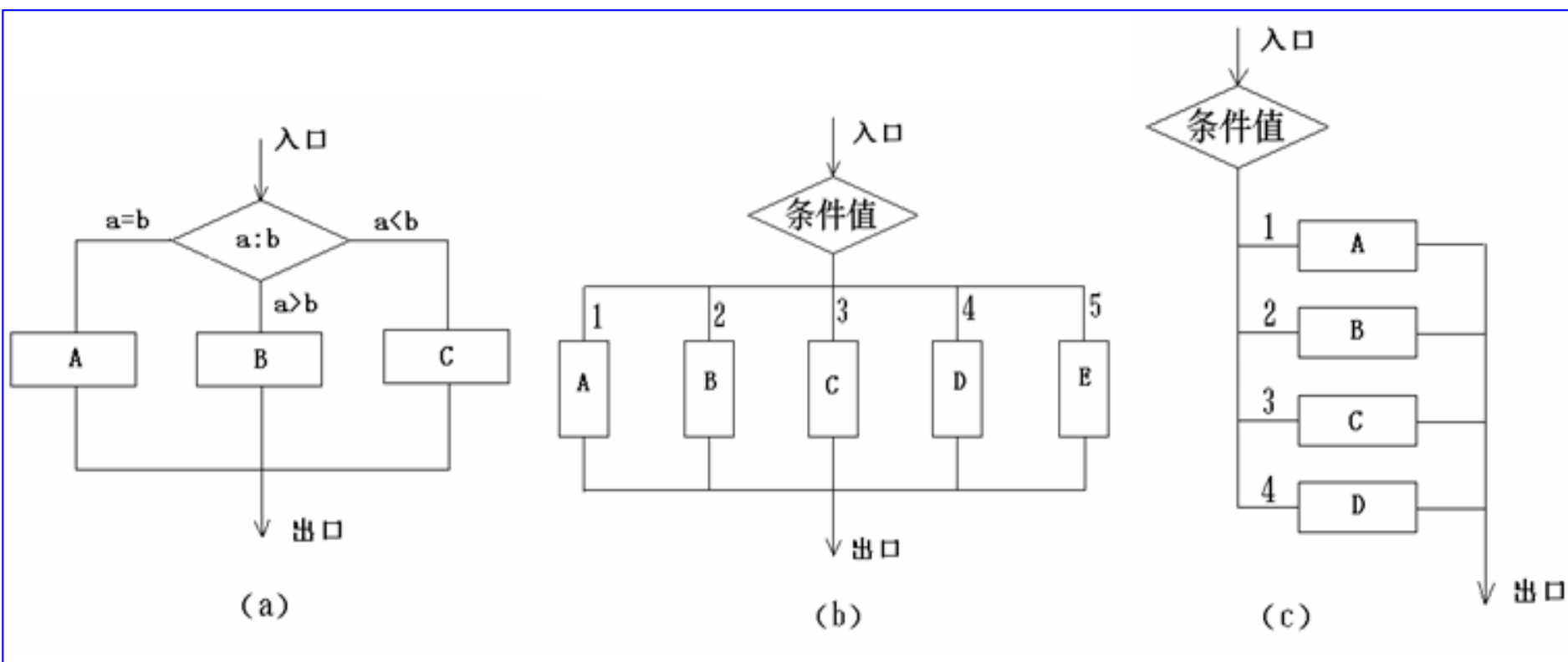


■ 程序流程图的基本控制结构



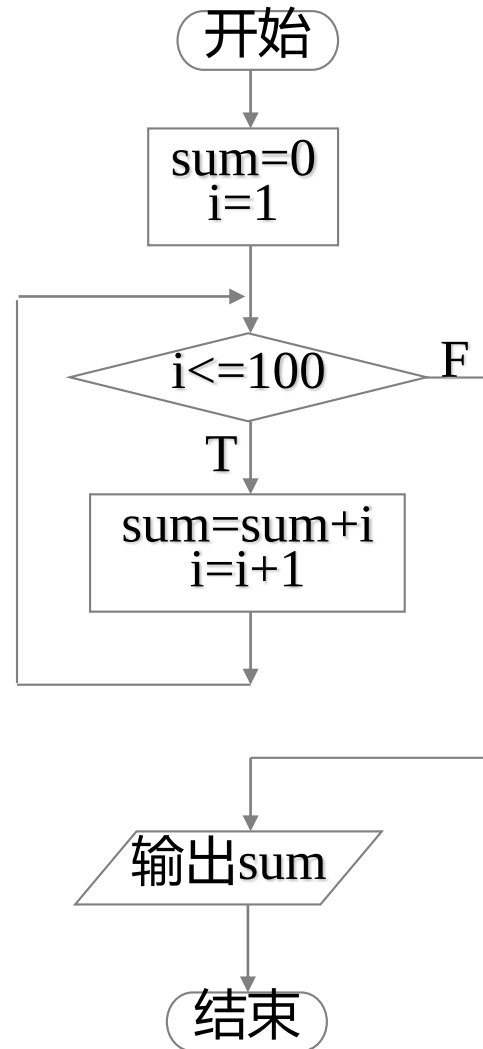
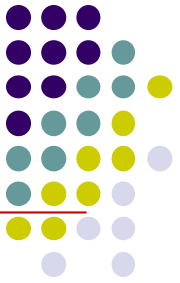
4.6.2 程序流程图

■ 程序流程图的基本控制结构

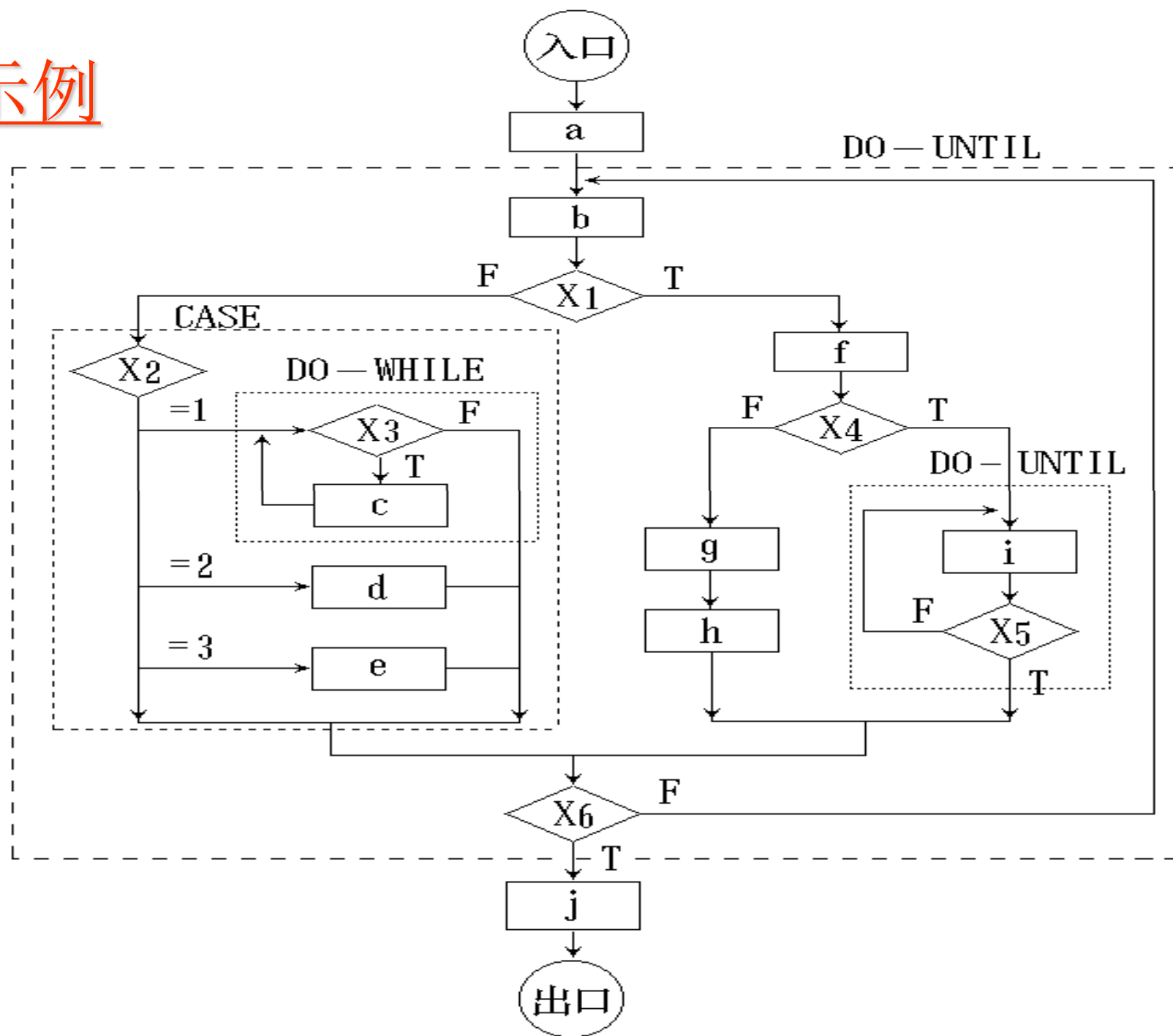


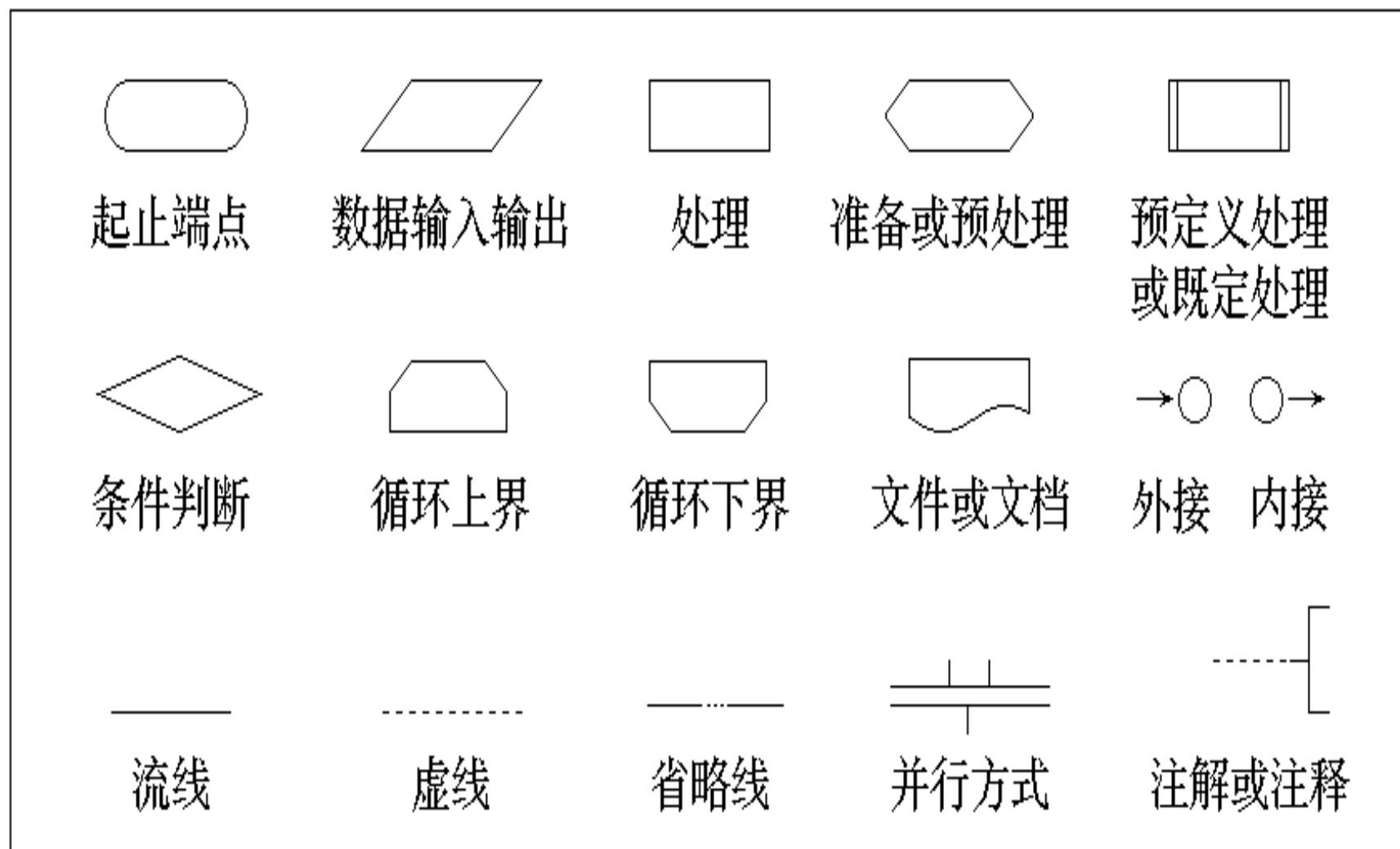
多出口判断

4.6.2 程序流程图



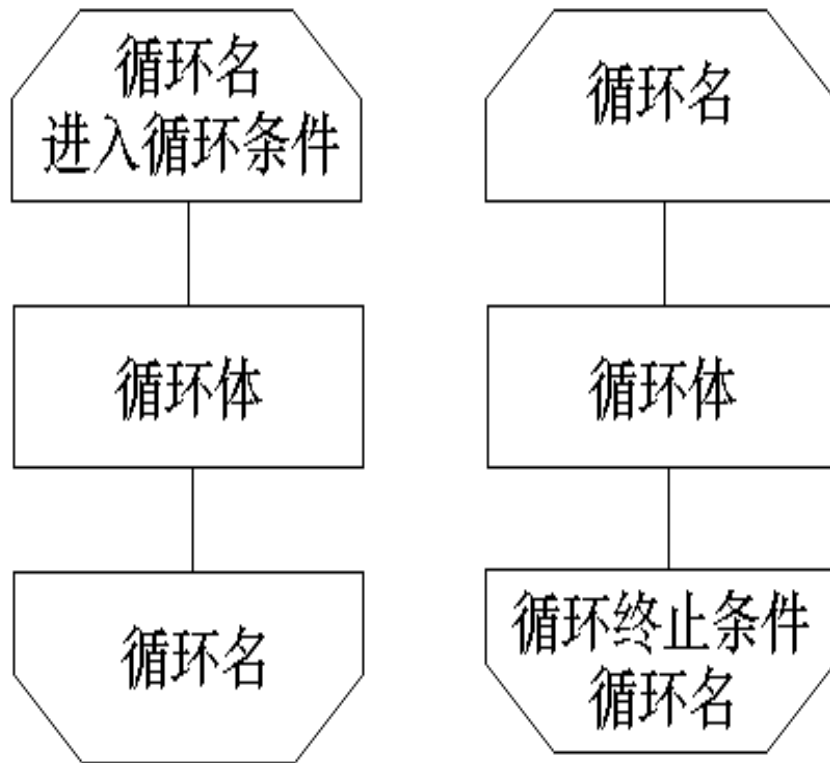
示例





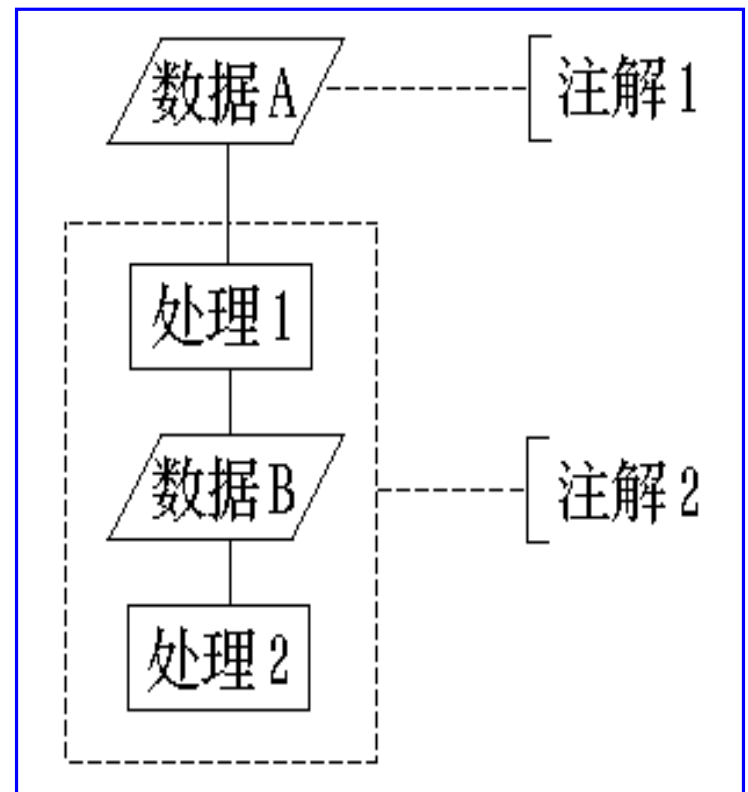
程序流程图的标准符号

循环的标准符号

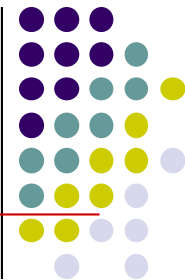


(a) while 型循环 (b) until 型循环

注解的使用



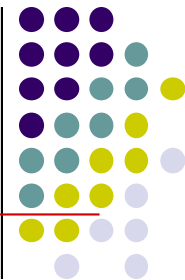
4.6.2 程序流程图



■ 程序流程图的特点

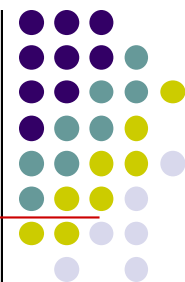
- 优点：直观、清晰，易于掌握。
- 缺点：
 - 流程图所使用的符号不够规范，常常使用一些习惯性用法。
 - 表示程序控制流程的箭头可以不受任何约束，随意转移控制。
 - 使程序员过早考虑程序的控制流程，而不考虑全局的结构。

4.6.3 N-S图

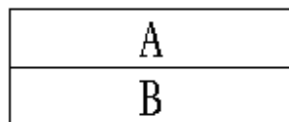


- Nassi和Shneiderman 提出了一种符合结构化程序设计原则的图形描述工具，叫做**盒图**（ box-diagram ），也叫做N-S图。
- 在N-S图中，为了表示5种基本控制结构，规定了5种图形构件。

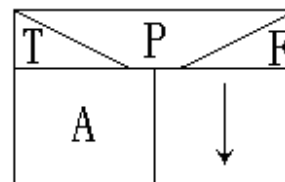
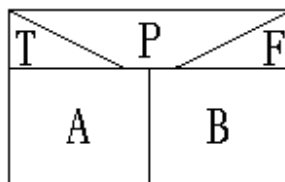
4.6.3 N-S图



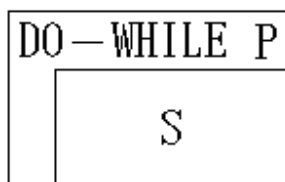
■ N-S图的基本控制结构



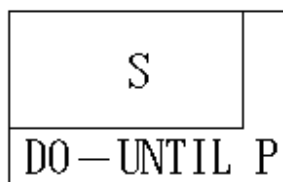
(a) 顺序型



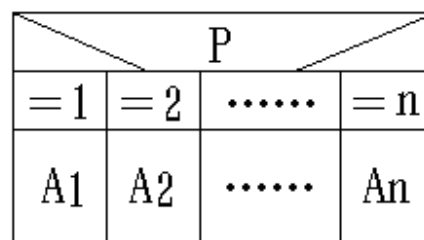
(b) 选择型



(c) WHILE 重复型

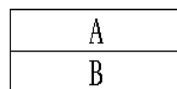


(d) UNTIL 重复型

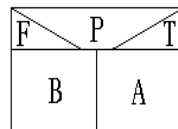


(e) 多分支选择型
(CASE 型)

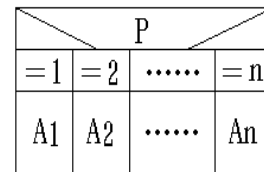
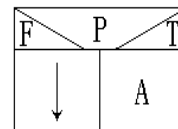
示例



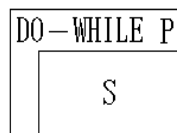
① 顺序型



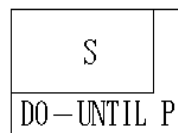
② 选择型



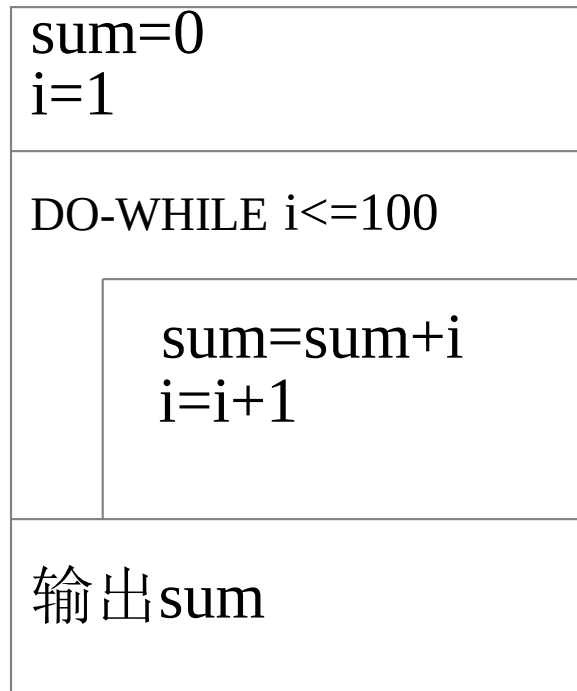
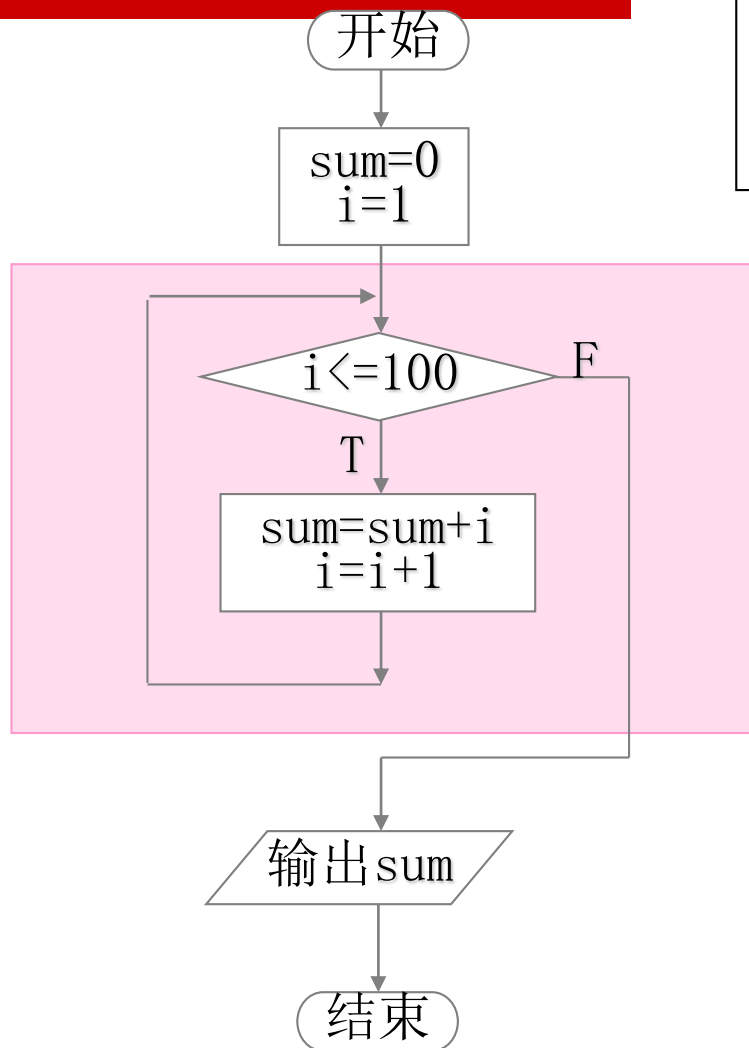
⑤ 多分支选择型
(CASE型)



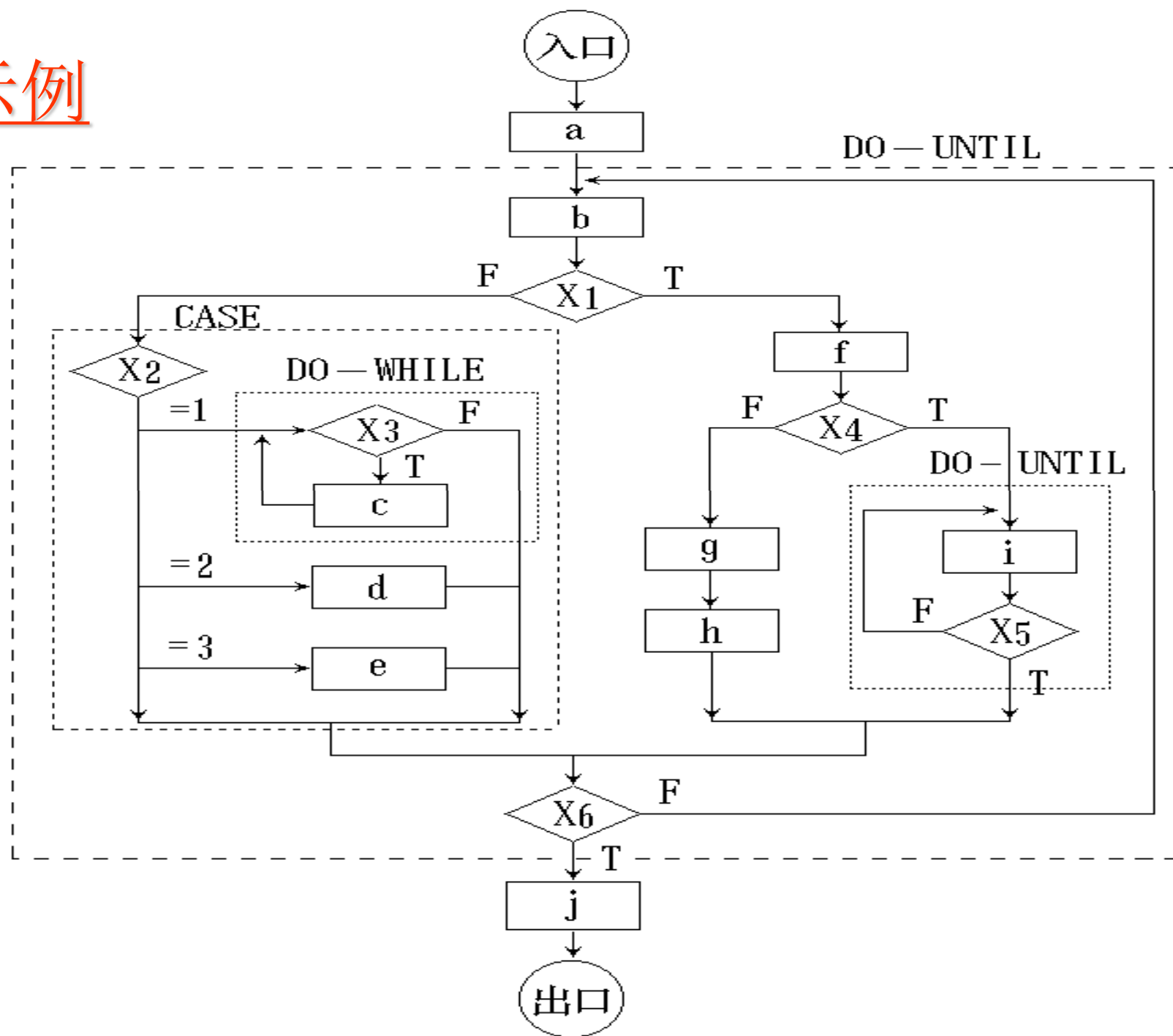
③ WHILE 重复型



④ UNTIL 重复型



示例



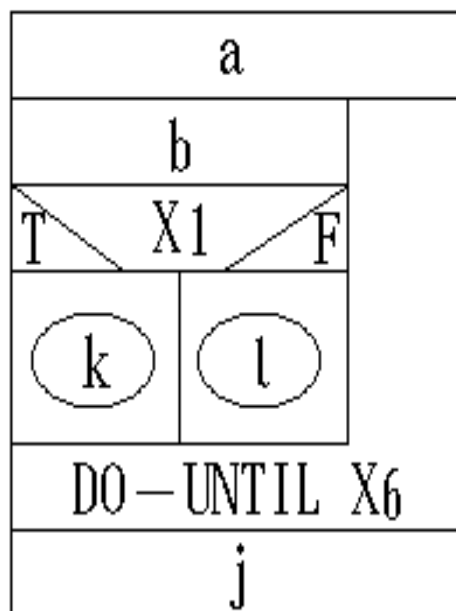


a						
b						
T \		X1			/ F	
f			X2 \			
T \		X4		/ F		
				= 1	= 2	= 3
i		X5	g	DO — WHILE		d
				X3	c	
DO — UNTIL		h		e		
DO — UNTIL X6						
j						

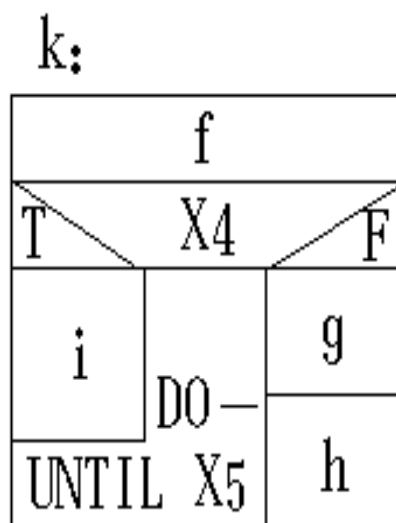


4.6.3 N-S图

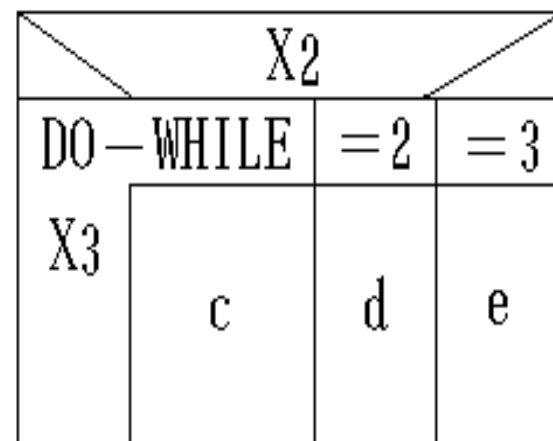
■ N-S图的嵌套定义形式



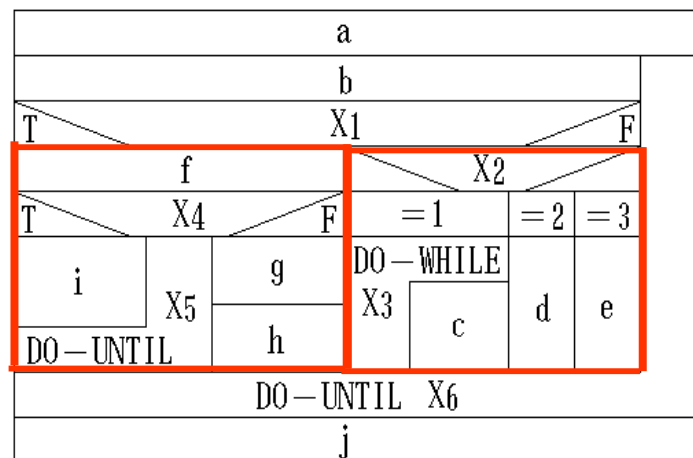
(a)



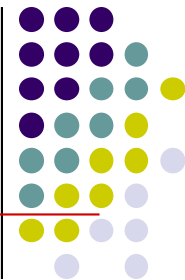
(b)



(c)



4.6.3 N-S图



■ N-S图的特点

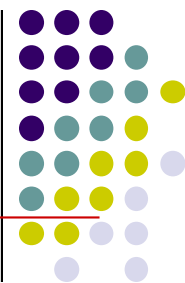
● 优点

- 功能域明确（即一个特定控制结构的作用域）
- 不可任意转移控制，符合结构化程序设计要求。
- 容易确定局部数据或全局数据的作用域。
- 容易表现嵌套关系，也可以表示模块的层次结构。

● 缺点

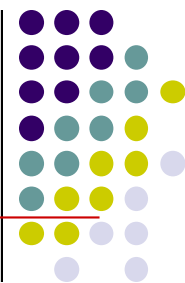
- 有时画图困难，嵌套太多时，内层方框越来越小。
- 这种情况可采用嵌套定义解决。

4.6.4 PAD图

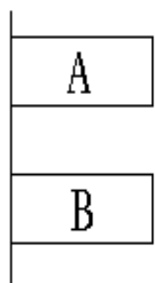


- PAD (problem analysis diagram) 是日本日立公司提出，由程序流程图演化来的，用结构化程序设计思想表现程序逻辑结构的图形工具。
- 用二维树形结构的图来表示程序的控制流，将这种图翻译成程序代码比较容易。现在已为ISO认可。

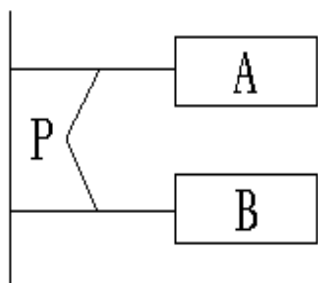
4.6.4 PAD图



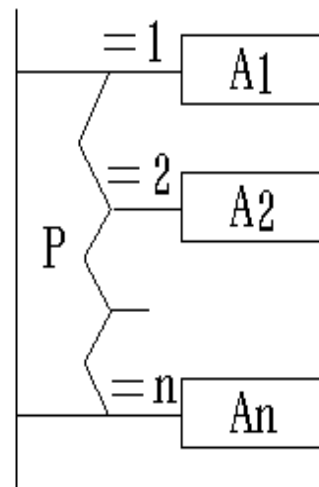
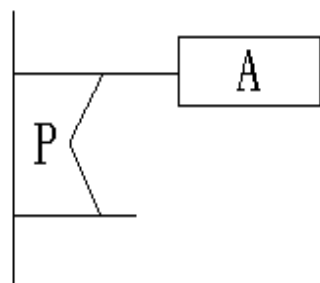
- PAD也设置了五种基本控制结构的图式，并允许递归使用。



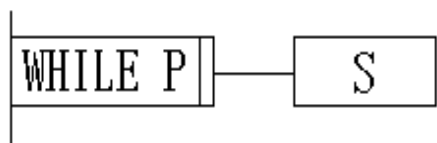
① 顺序型



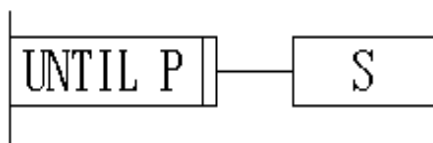
② 选择型



⑤ 多分支选择型
(CASE型)

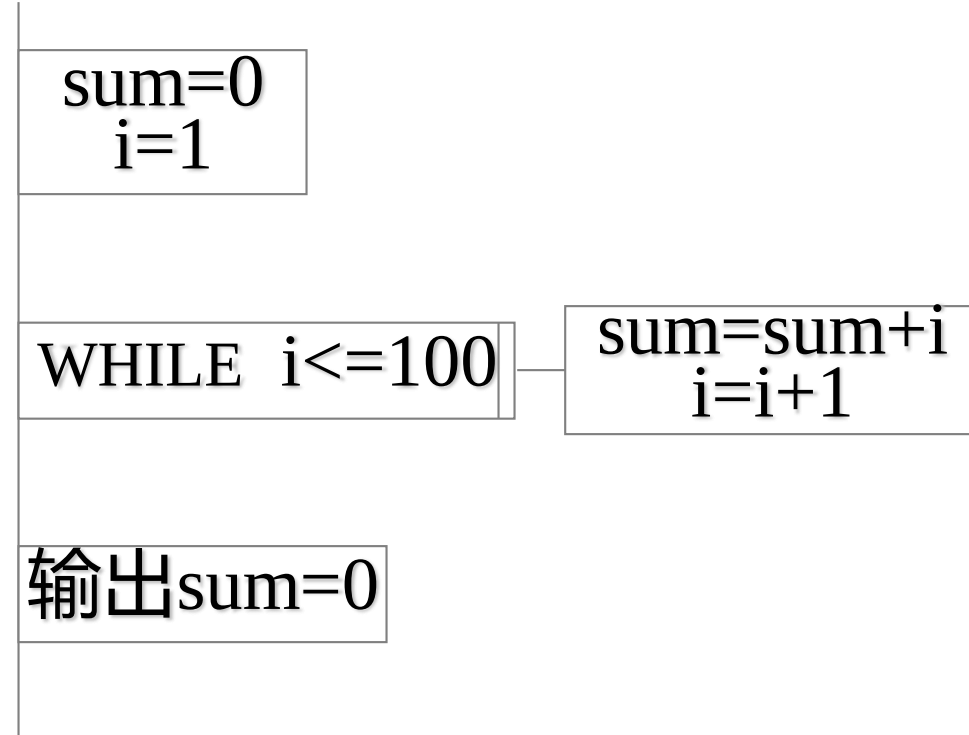
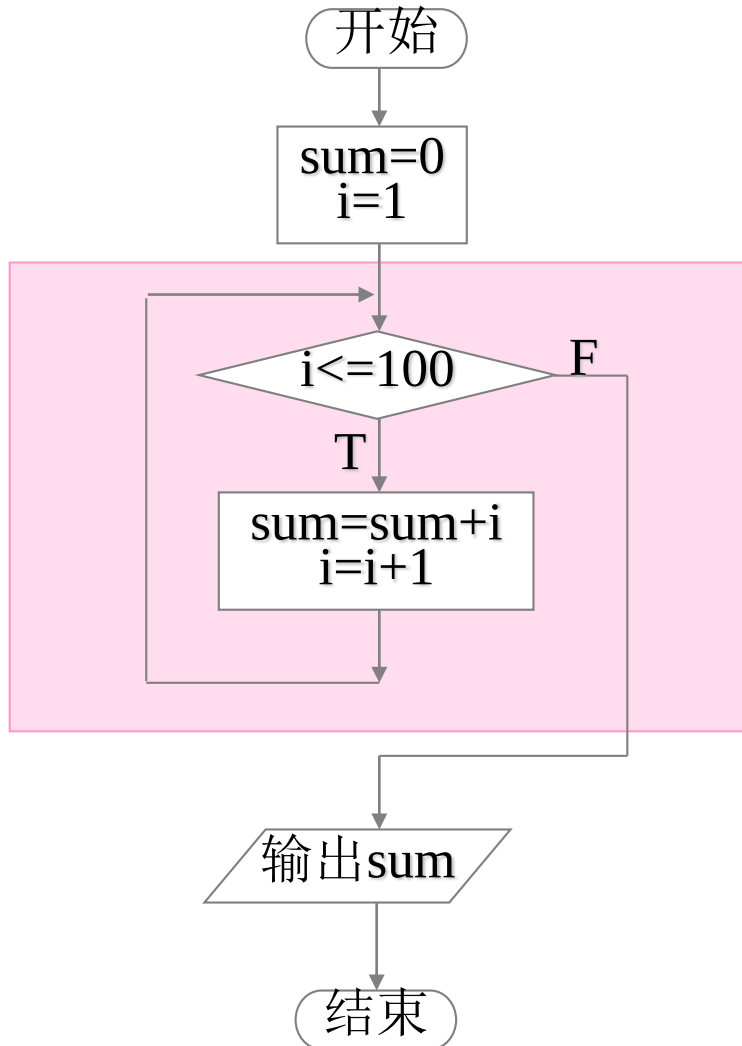
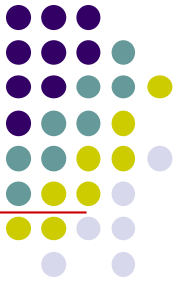


③ WHILE 重复型

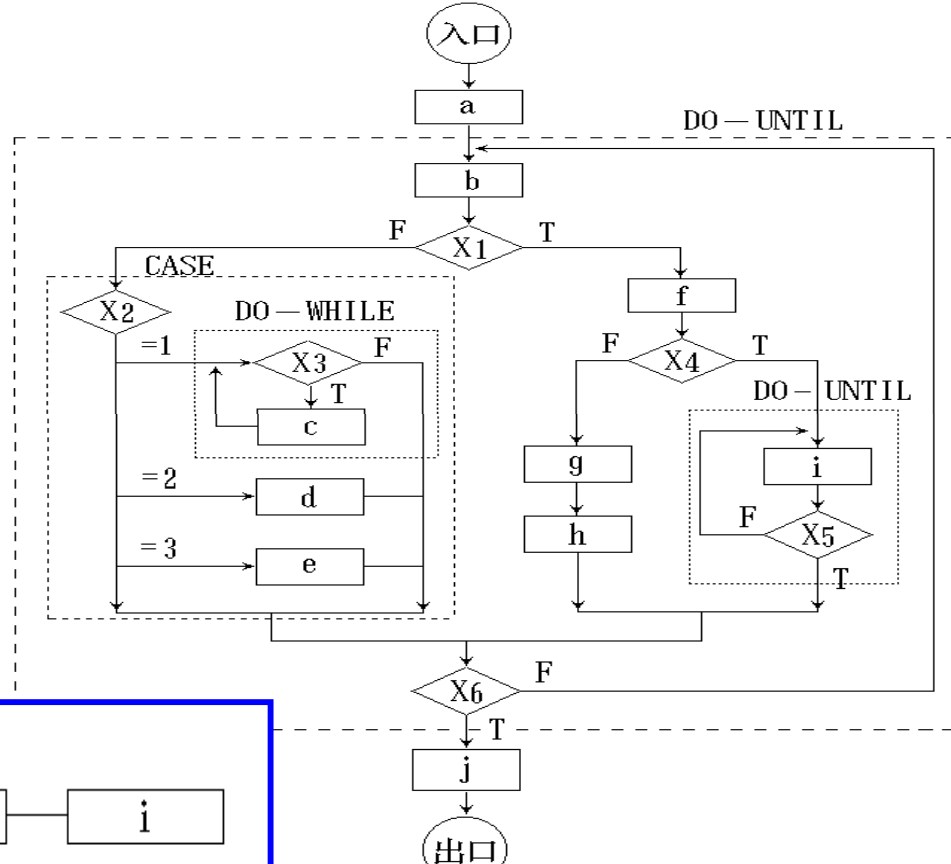


④ UNTIL 重复型

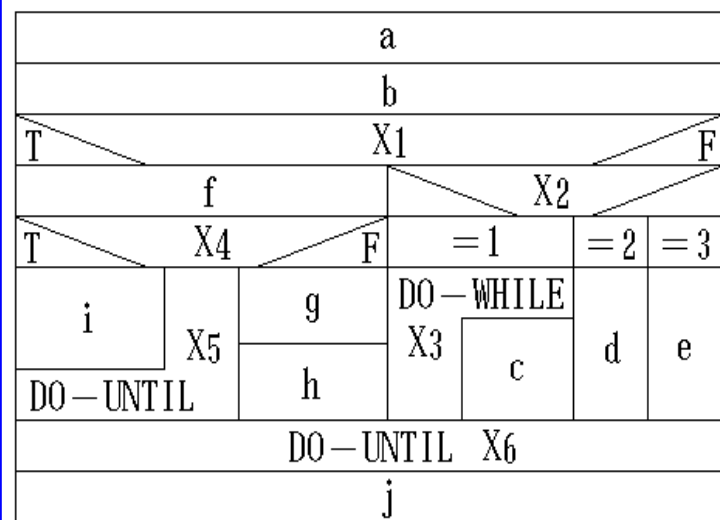
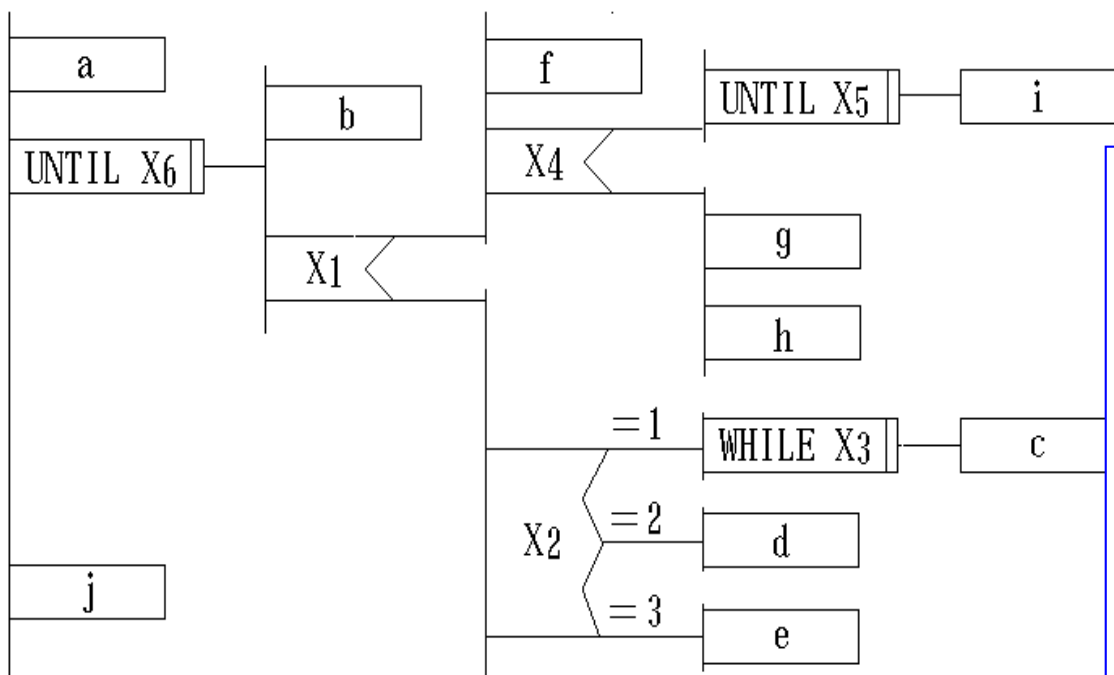
4.6.4 PAD图



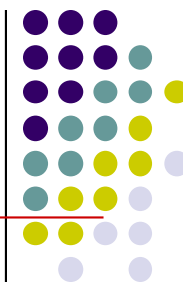
示例



PAD描述的示例



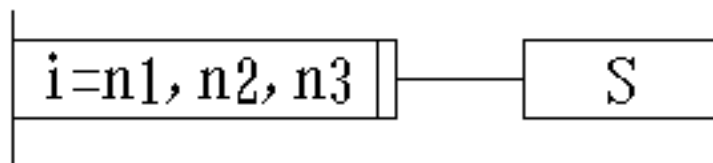
4.6.4 PAD图



- PAD的扩充控制结构
 - 对应于增量型循环结构

for $i := n1$ to $n2$ step $n3$ do

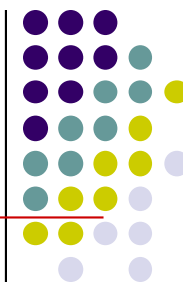
在PAD中有相应的循环控制结构



(a) FOR 重复型

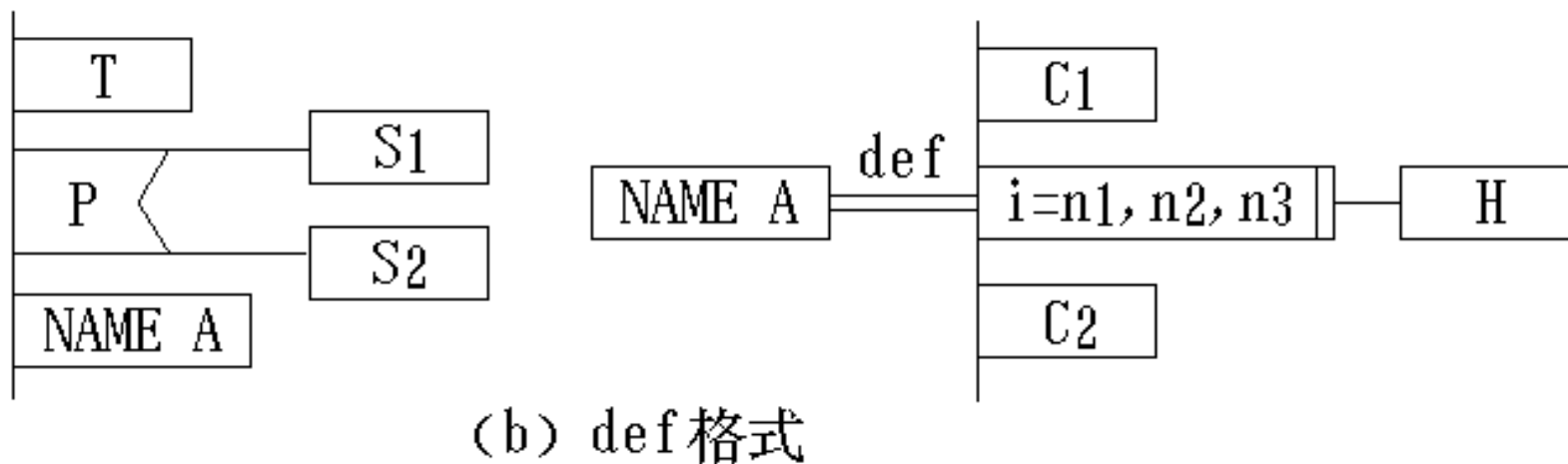
其中, $i=n1, n2, n3$ 表示
 $i:=n1$ to $n2$ step $n3$

4.6.4 PAD图

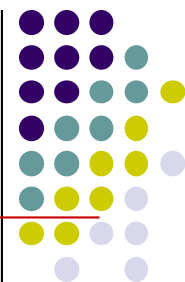


■ PAD的扩充控制结构

- PAD图支持自顶向下，逐步求精的方法。

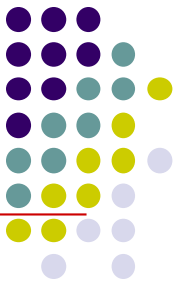


4.6.4 PAD图



■ PAD图的特点

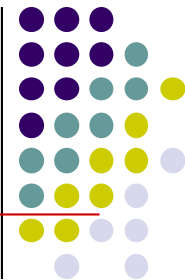
- 用PAD图设计的程序必然是结构化程序。
- PAD图描绘的程序结构十分清晰
PAD图中竖线表示层次，竖线的总条数就是程序的层次数。
- 执行次序
以PAD图为基础，按照机械的变换规则就可以写出结构化的程序来，可用软件工具自动完成。
- PAD图支持自顶向下，逐步求精的方法。



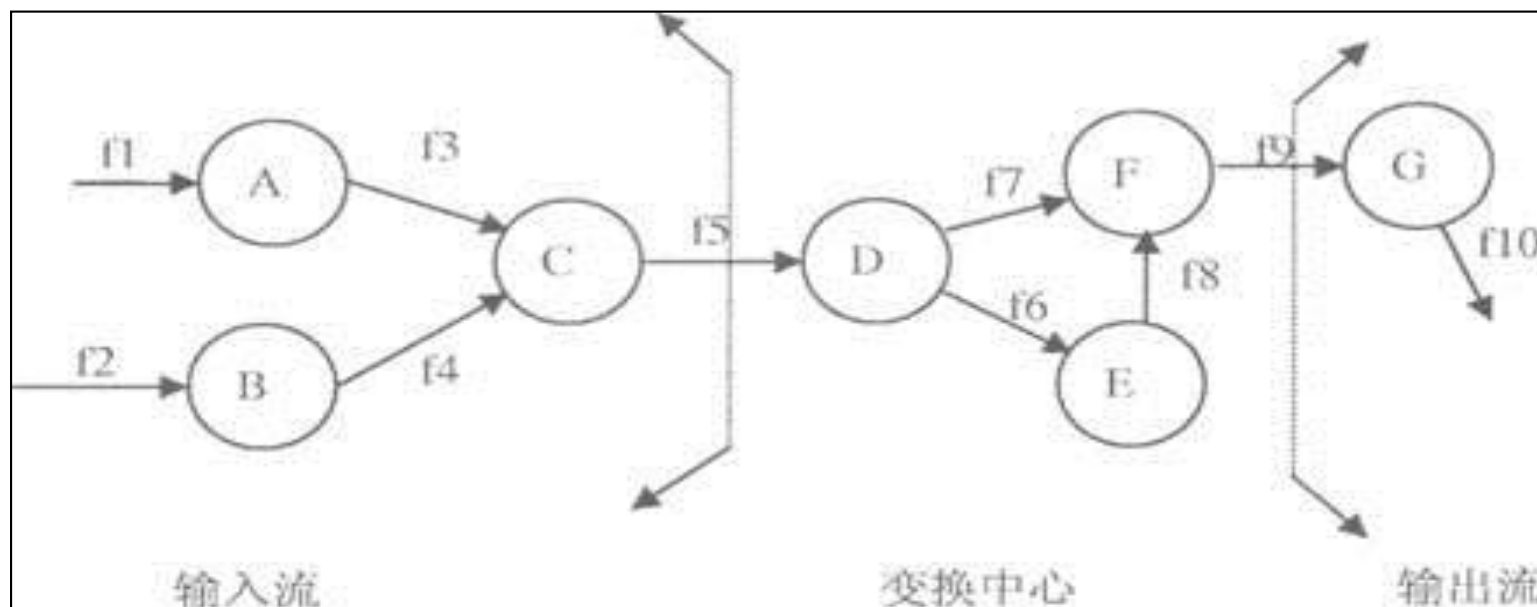
4.7 软件设计规格说明

- **国家标准GB/T 8567—2006 《计算机软件文档编制规范》**
- **有关软件总体设计的文档**
 - 《系统/子系统设计(结构设计)说明(SSDD)》
 - 《软件(结构)设计说明 (SDD) 》
 - 《数据库设计说明 (DBDD) 》
 - 《接口设计说明 (IDD) 》

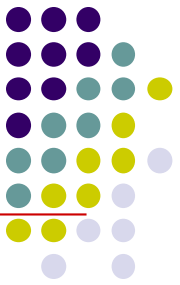
练习



根据下列变换型的数据流图，画出对应的软件结构图。



练习



根据下列伪代码，画出程序流程图、盒图、PAD图。

```
begin
s1;
if x>1 then s2
else s3;
for n=1 to 10
do s4;
if y>1 then s5;
end;
```